

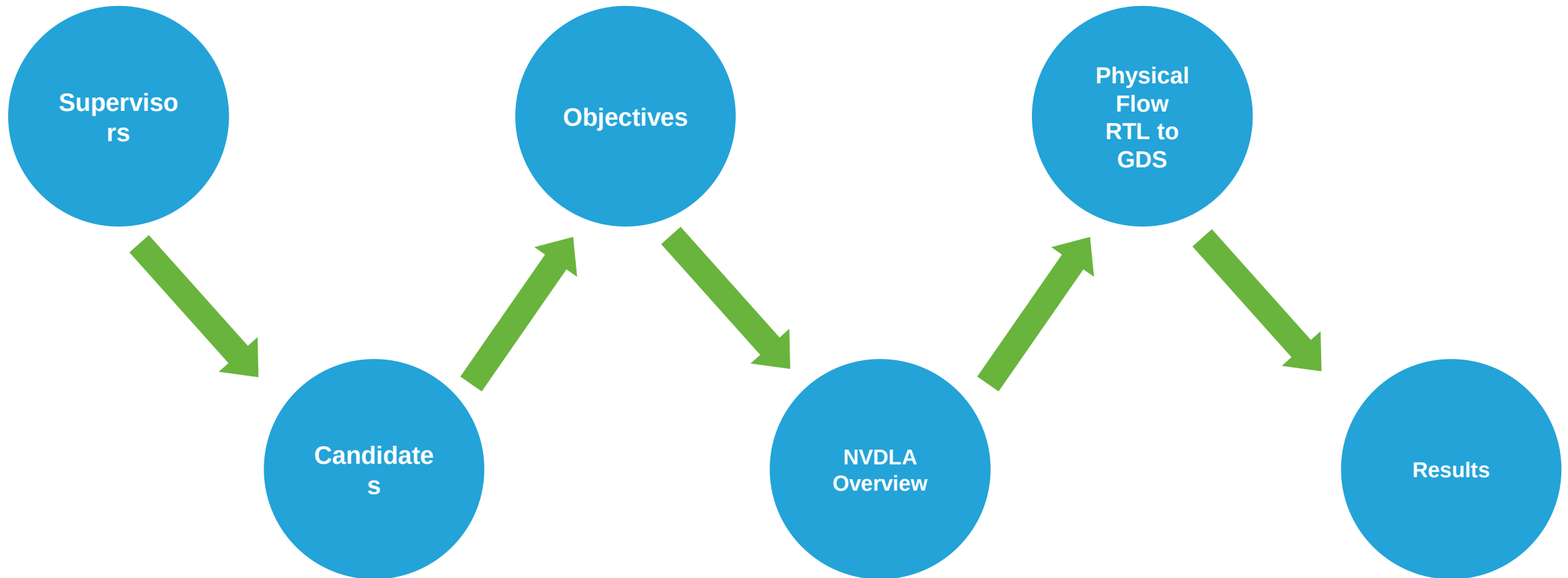


NVIDIA®

NVIDIA Deep Learning Accelerator

Physical Implementation







Eng. Islam Samir



Eng. Mariam



Eng. Bassem



Eng. Momen



Eng.Ahmed El-Gammal



Eng.Mohamed Hossam



Eng.Momen Ashraf



Eng.Mohamed Emam



Eng.Aasmaa Anwar



Eng.Ahmed Abdullah

1. Large-Scale Design

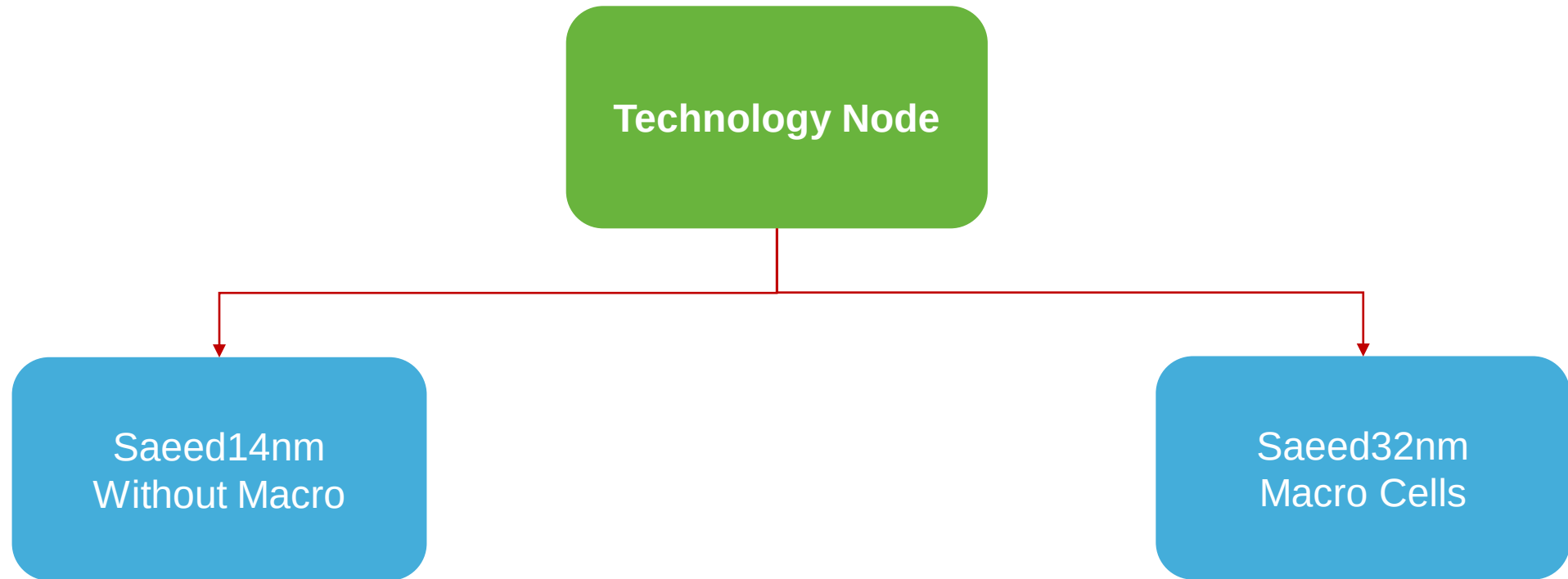
NVDLA provides a large and complex design, allowing us to identify and overcome the challenges that typically arise during a complete RTL-to-GDS physical implementation flow.

2. Macro Integration

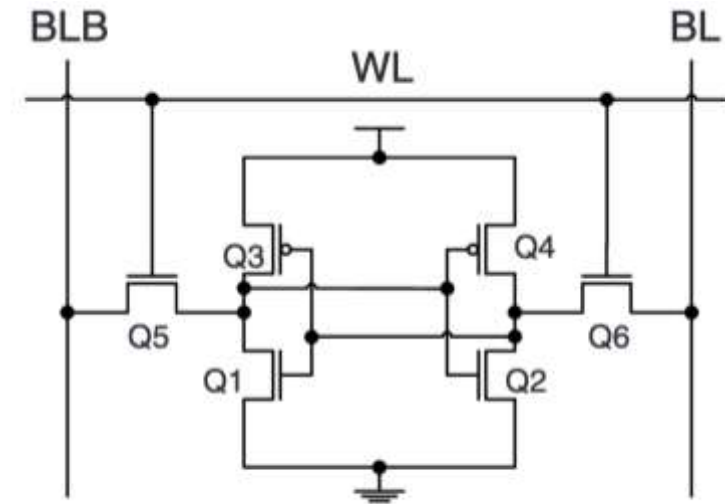
The design contains multiple macros, providing practical experience in macro placement, power connectivity, routing, congestion, timing, and other macro-related physical implementation challenges.

3. Real-World Design:

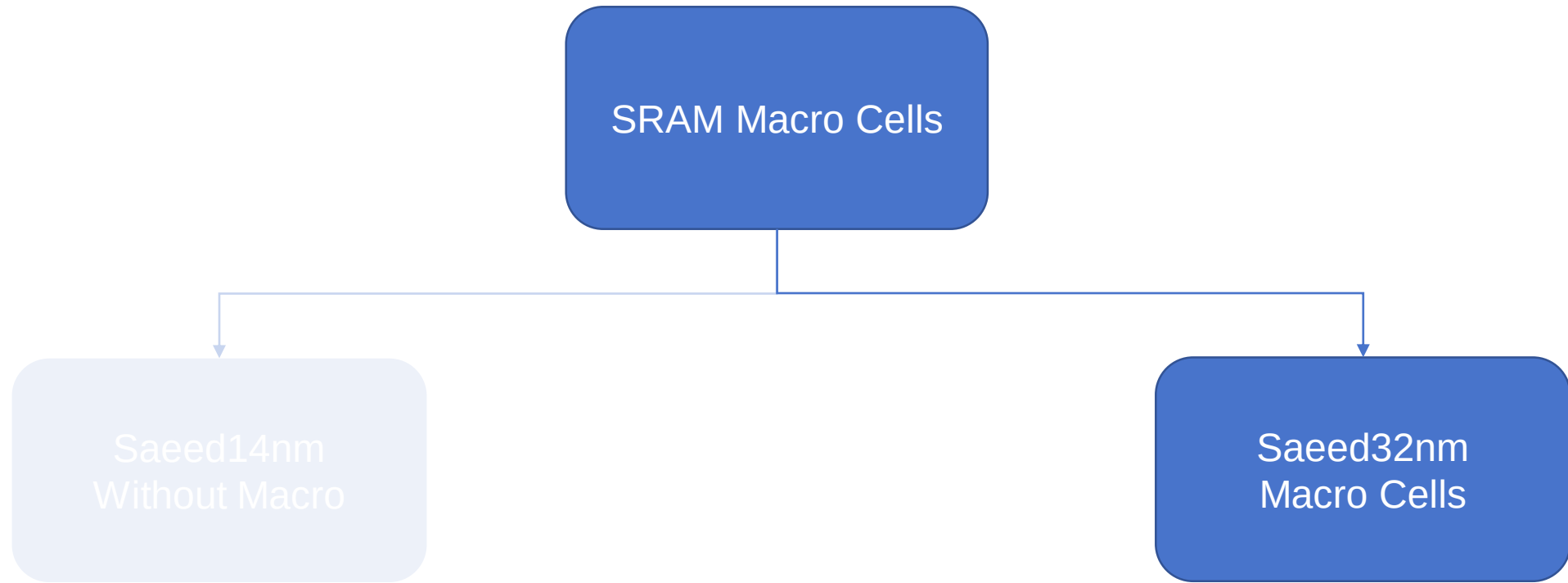
NVDLA is an actual **Deep Learning Accelerator** architecture designed for AI workloads and has been integrated into real-world applications.



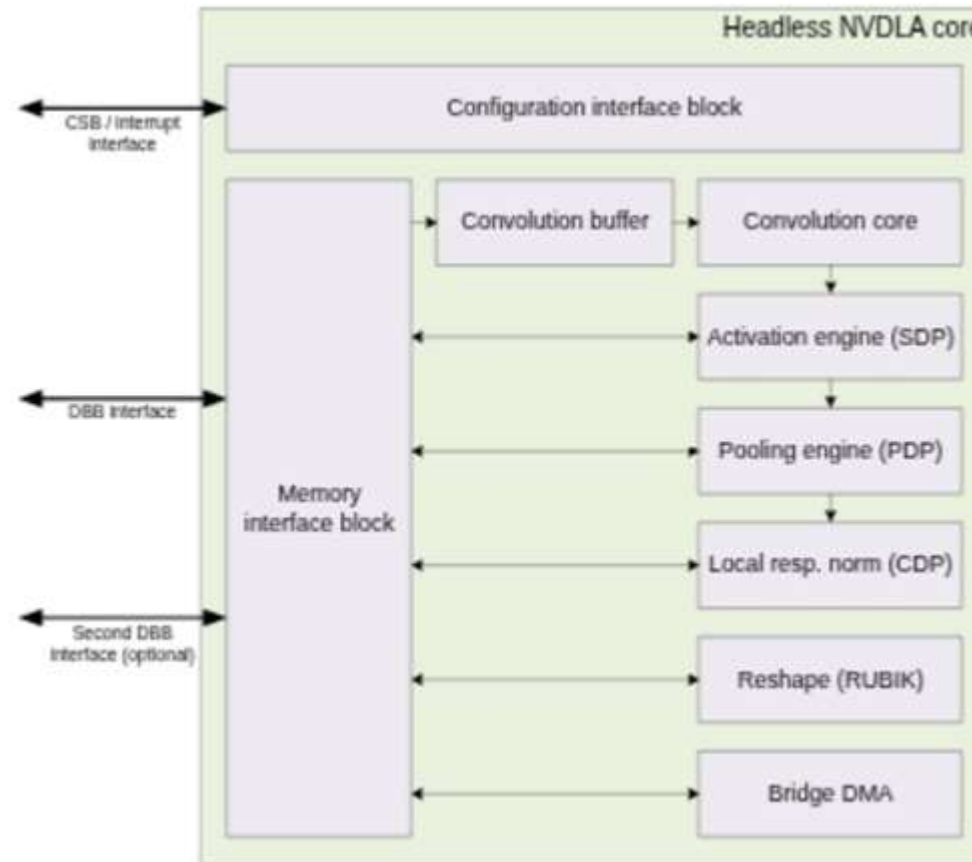
- **Open-source SRAM** compiler framework that generates layout, netlists, timing/power models, and P&R abstracts.
- Works with both open-source PDKs (e.g., SkyWater 130, GF 180) and commercial PDKs if you configure the tech rules



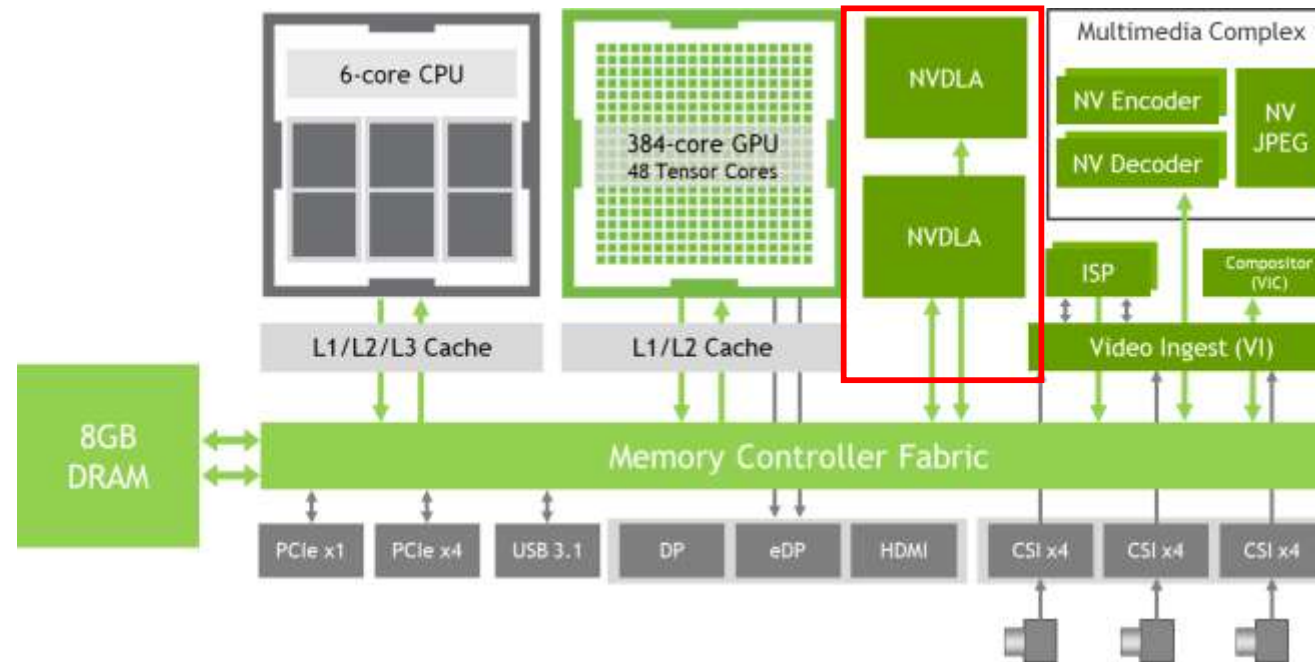
SRAM unit Cell



- **NVDLA** is an open-source, scalable AI accelerator architecture designed for neural network inference. It provides dedicated hardware to accelerate key operations such as **convolution, pooling, activation, and data reshaping**, enabling faster and more efficient AI data processing.



- **NVDLA** hardware-accelerated operations reduce the computational workload on the CPU/GPU and enable faster and more efficient processing of AI workloads.



NVIDIA Jetson Xavier

NVIDIA Jetson AGX Xavier integrates NVDLA as a dedicated deep-learning accelerator and is used in commercial and real-world applications, including drones, robotics, autonomous machines, and other AI-enabled systems



NVIDIA Jetson Xavier



Drones

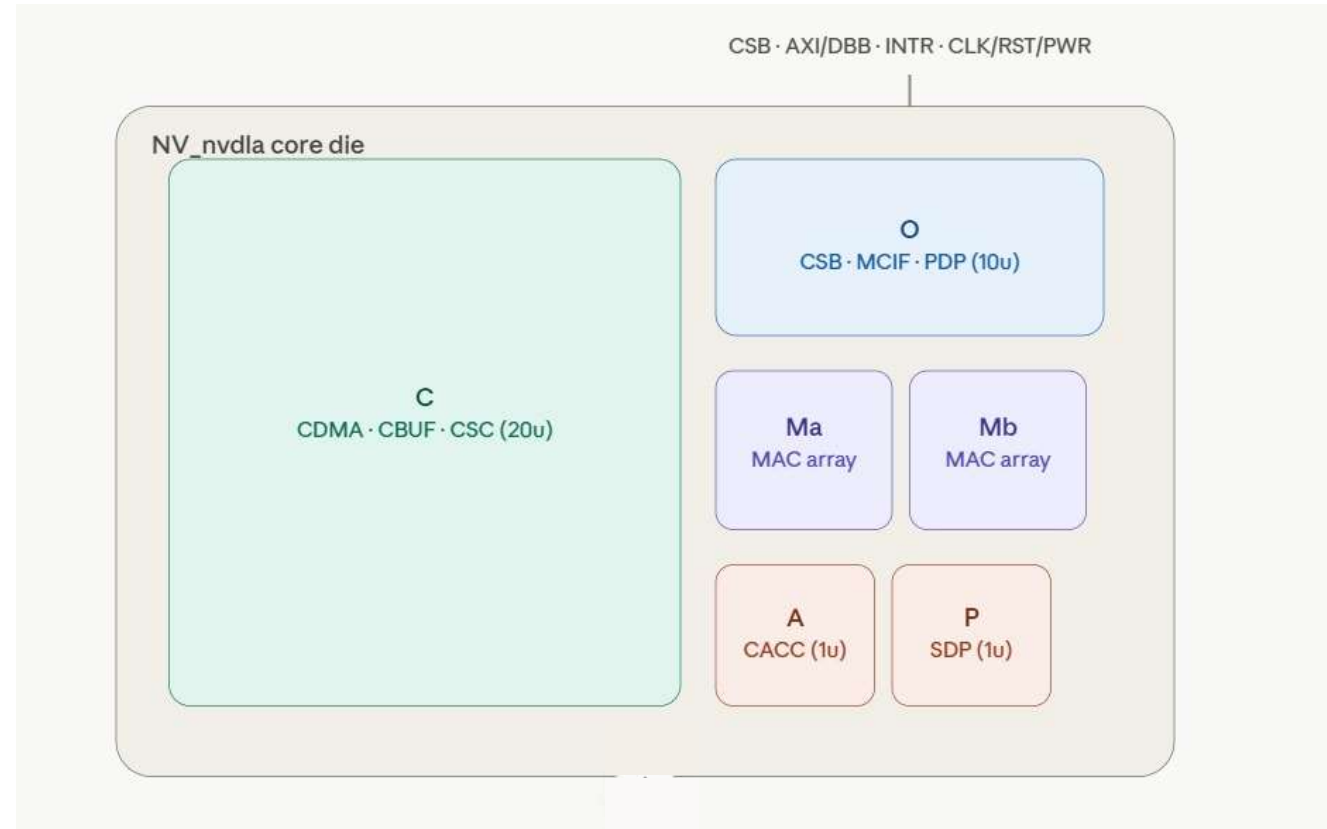


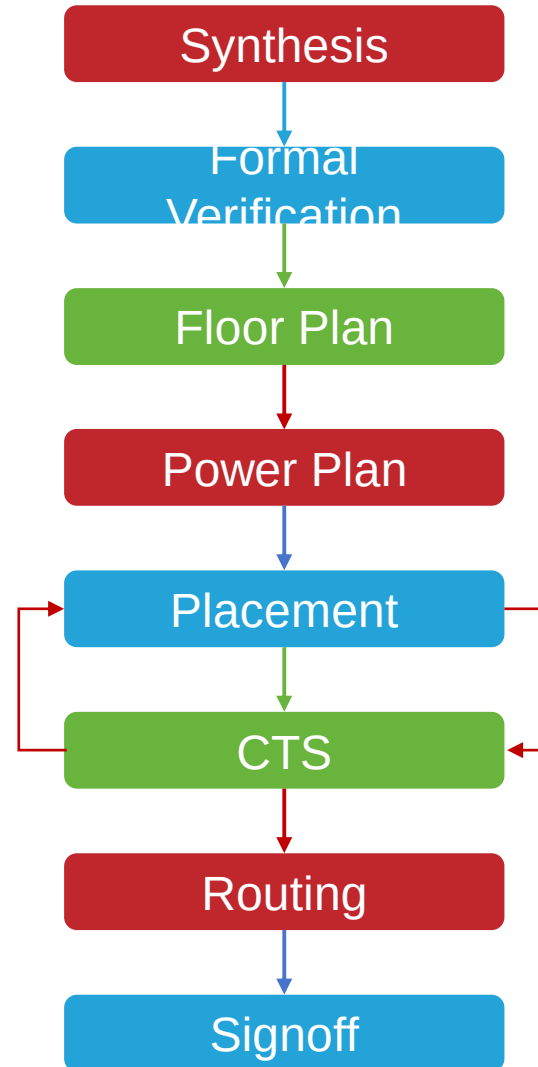
Medical

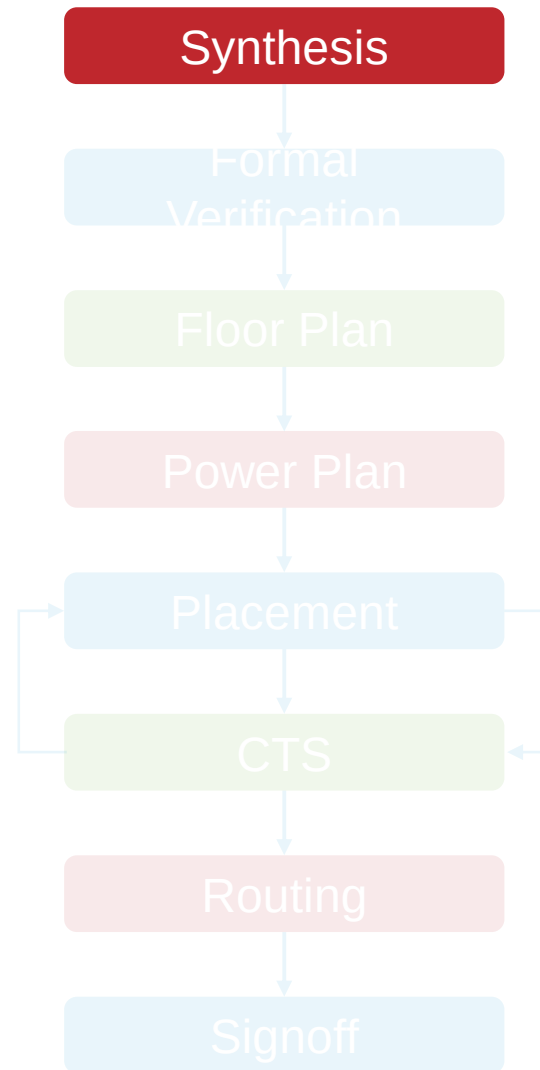


Robotics

The **NVDLA open-source** project is divided into **six partitions**, with each member responsible for implementing one partition through the complete RTL-to-GDS physical implementation flow.

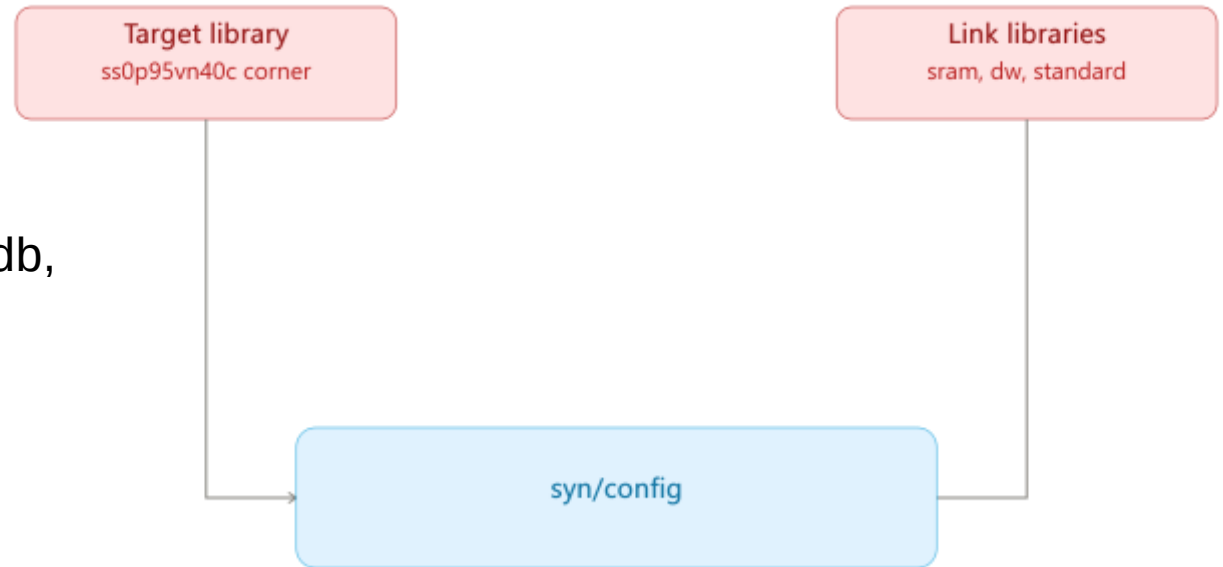






1. **Libs**
2. Memory mapping
3. Constraints
4. Formality
5. MMMC and library creation

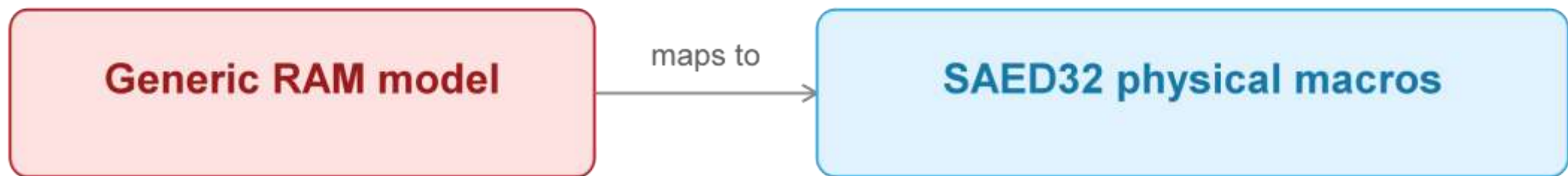
- **Target library** : saed32rvt_ss0p95vn40c.db
- **Link libraries** : saed32sram_ss0p95vn40c.db,
dw_foundation.sldb, standard.sldb



- Libs
- **Memory mapping**
- Constraints
- Formality
- MMMC and library creation

Why we needed it ?

- The original open-source NVDLA RTL uses generic/behavioral RAM models (vmod/rams/synth/), which are vendor-agnostic wrappers. However, real ASIC manufacturing requires actual physical SRAM macro blocks from a foundry/library (Synopsys SAED32 32nm SRAMs).



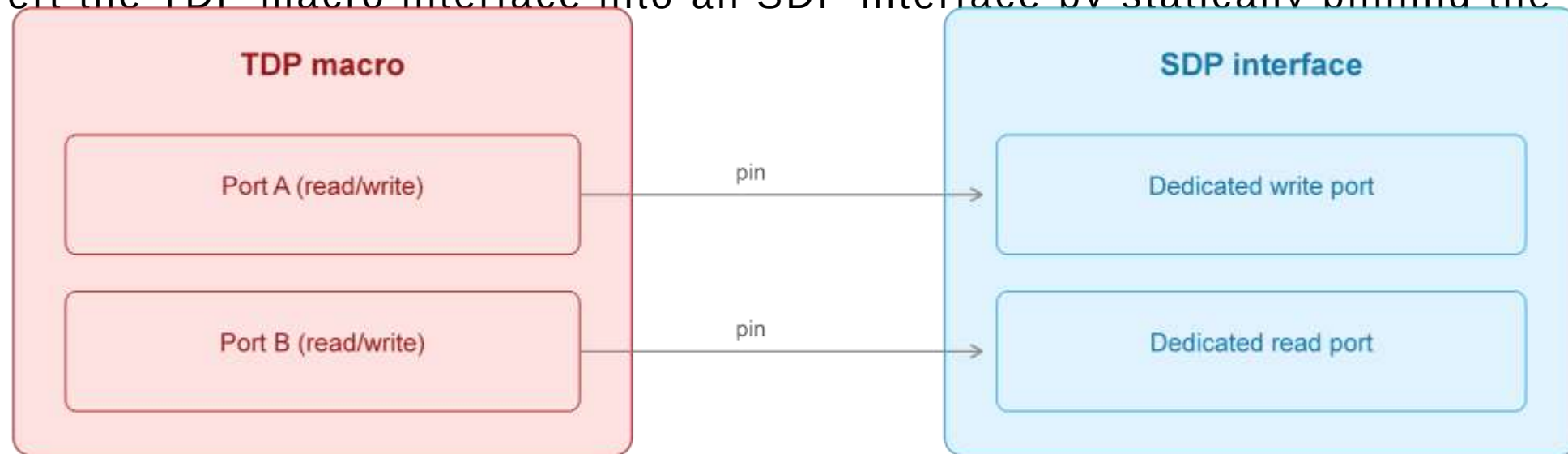
Techniques used:

Technique A: True Dual Port (TDP) → Simple Dual Port (SDP) Adaptation

The physical SAED32 PDK library only provides True Dual Port (SRAM2RW) macros.

However, NVDLA functional memories operate as Simple Dual Port (1 Dedicated Write Port + 1 Dedicated Read Port).

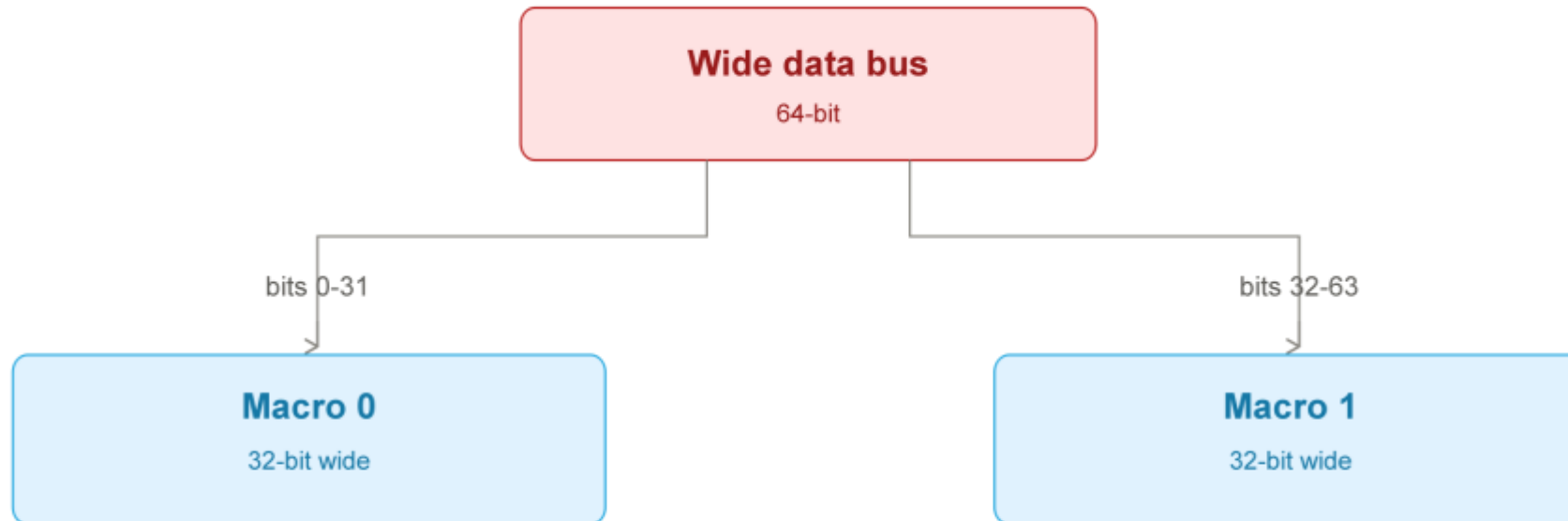
We convert the TDP macro interface into an SDP interface by statically pinning the ports



Techniques used:

Technique B: Width Slicing (Bit-Width Expansion)

When the required memory width exceeds the maximum width of a single SAED32 macro (32), the bus is systematically sliced across multiple parallel macros

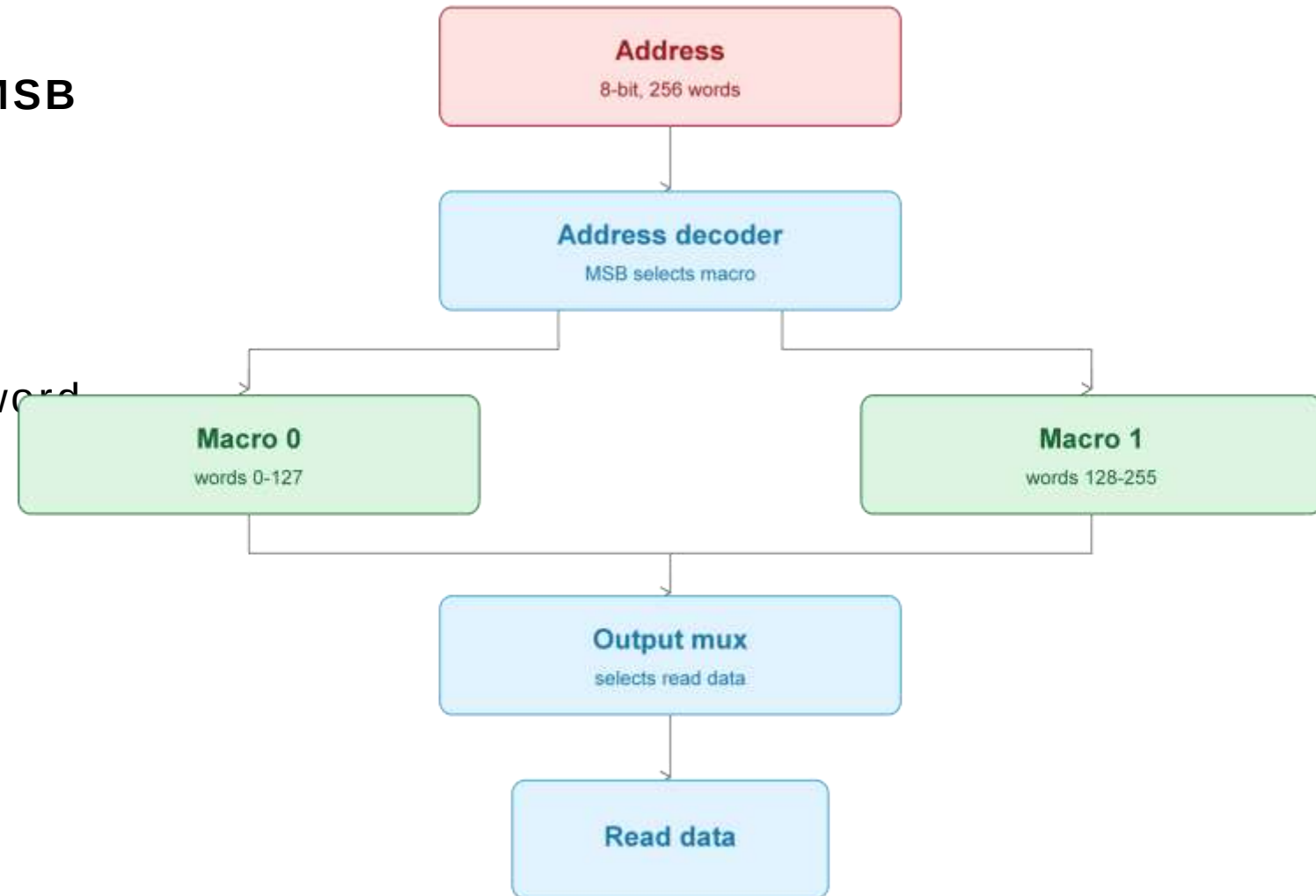


Techniques used:

Technique C: Depth Expansion & MSB

Address Decoding

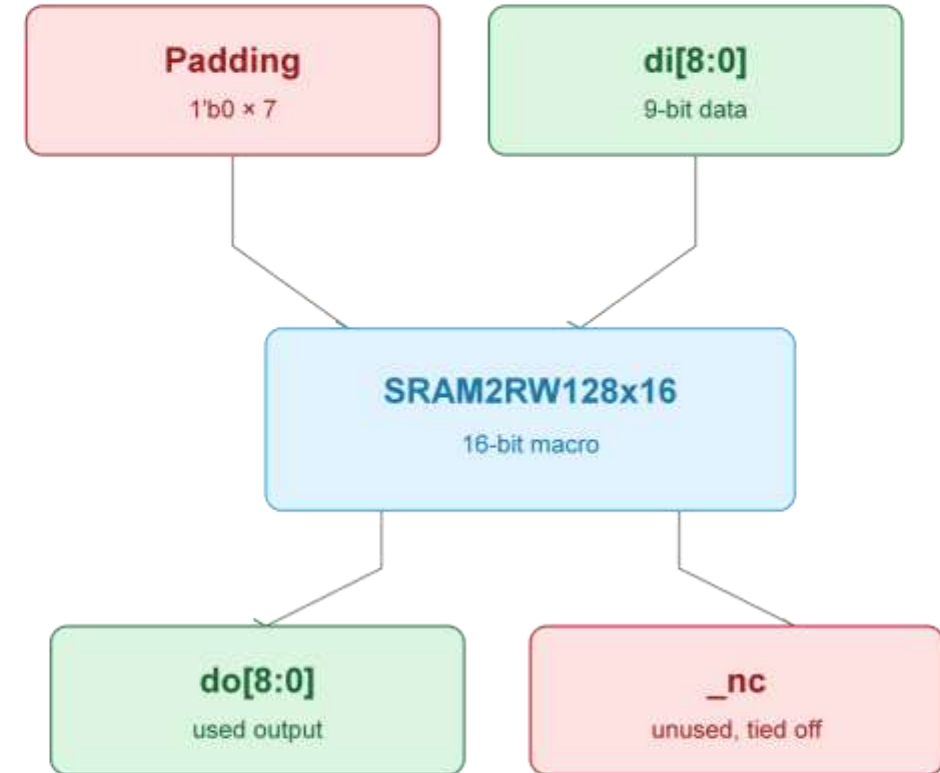
When the required memory depth exceeds standard macro depth (e.g., 256 words deep mapped using 128-word deep macros):



Techniques used:

Technique D: Data Padding & Unused Bit Tie-Offs

- **Input Padding:** If memory width is not a multiple of available macro width (e.g., 9-bit width into a 16-bit macro), upper input bits are padded with 1'b0
- **Output Tie-offs:** Unconnected output bits from the macro are tied to dummy `_nc` (no-connect) wires to suppress synthesis lint warnings.



Zero-pad the extra input bits, tie off the extra output bits

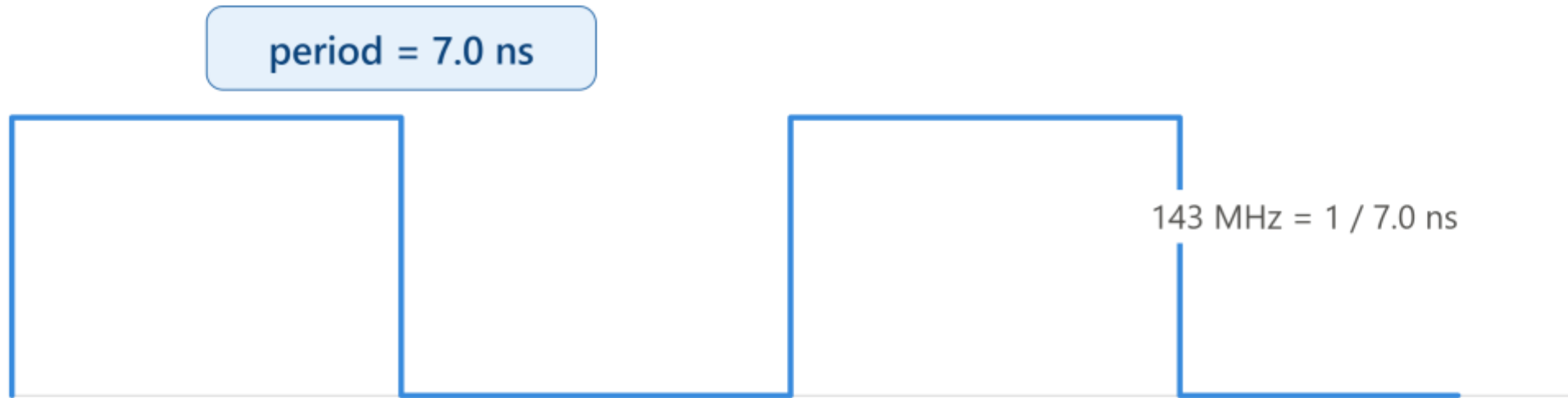
Memory mapping:

#	Behavioral RTL Module	Depth × Width	Physical SAED32 SRAM Macro Instantiations
1	nv_ram_rws_128x18_logic.v	128 × 18	1x SRAM2RW128x32
2	nv_ram_rws_16x256_logic.v	16 × 256	8x SRAM2RW16x32
3	nv_ram_rws_16x272_logic.v	16 × 272	8x SRAM2RW16x32 + 1x SRAM2RW16x16
4	nv_ram_rws_16x64_logic.v	16 × 64	2x SRAM2RW16x32
5	nv_ram_rws_256x3_logic.v	256 × 3	2x SRAM2RW128x8
6	nv_ram_rws_256x64_logic.v	256 × 64	4x SRAM2RW128x32
7	nv_ram_rws_256x7_logic.v	256 × 7	2x SRAM2RW128x8
8	nv_ram_rwsp_128x11_logic.v	128 × 11	1x SRAM2RW128x16
9	nv_ram_rwsp_128x6_logic.v	128 × 6	1x SRAM2RW128x8
10	nv_ram_rwsp_160x16_logic.v	160 × 16	2x SRAM2RW128x16
11	nv_ram_rwsp_160x65_logic.v	160 × 65	4x SRAM2RW128x32 + 2x SRAM2RW128x4
12	nv_ram_rwsp_61x65_logic.v	61 × 65	2x SRAM2RW64x32 + 1x SRAM2RW64x4
13	nv_ram_rwsp_80x14_logic.v	80 × 14	1x SRAM2RW128x16
14	nv_ram_rwsp_80x65_logic.v	80 × 65	2x SRAM2RW128x32 + 1x SRAM2RW128x4
15	nv_ram_rwsp_8x65_logic.v	8 × 65	2x SRAM2RW16x32 + 1x SRAM2RW16x4
16	nv_ram_rwst_256x8_logic.v	256 × 8	2x SRAM2RW128x8
17	nv_ram_rwthp_60x21_logic.v	60 × 21	1x SRAM2RW64x32
18	nv_ram_rwthp_80x15_logic.v	80 × 15	1x SRAM2RW128x16
19	nv_ram_rwthp_80x9_logic.v	80 × 9	1x SRAM2RW128x16

- Libs
- Memory mapping
- **Constraints**
- Formality
- MMMC and library creation

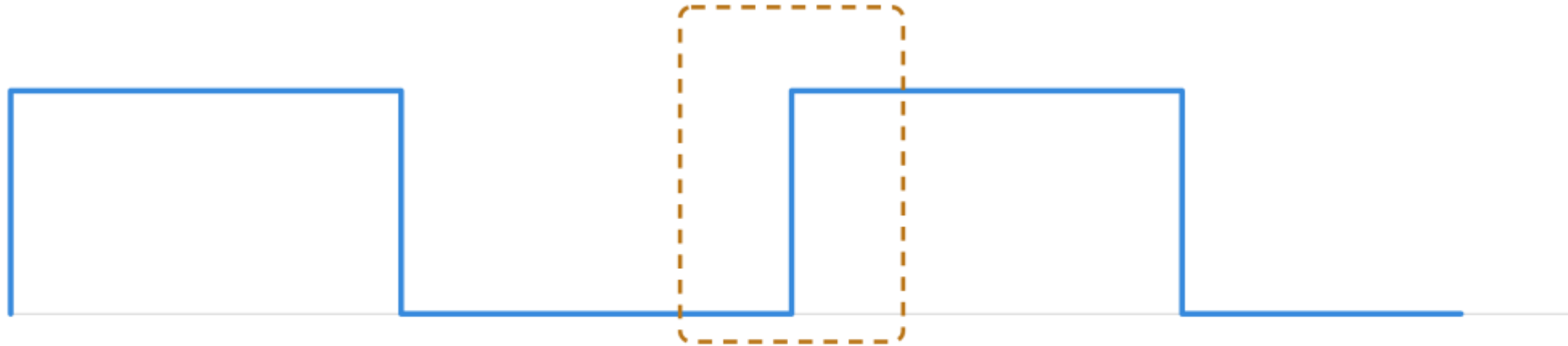
Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained	7.0 ns (143 MHz)



Transitioning from Generic SDC to Realistic SAED32 Constraints

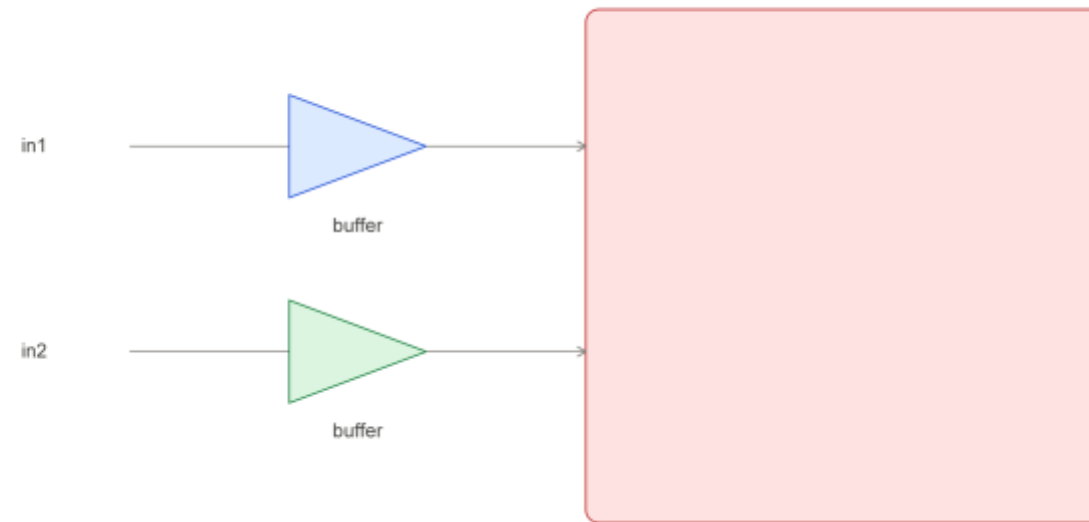
SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns



setup 0.3 ns ← —→ hold 0.1 ns

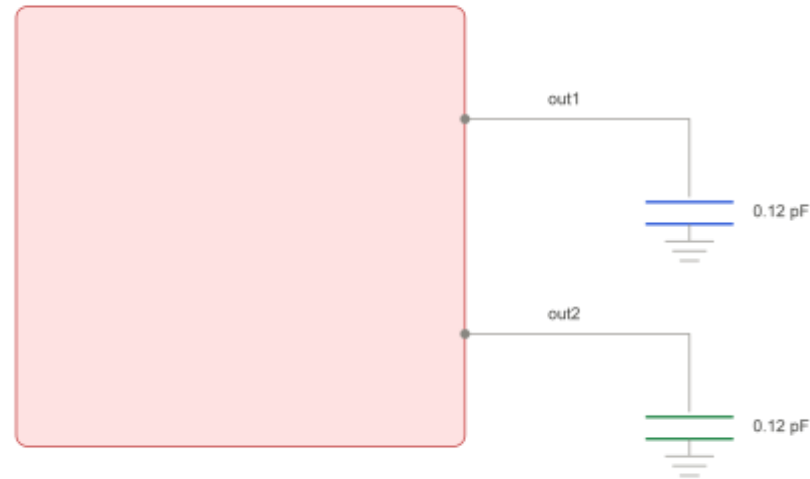
Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFFX4_RVT Buffer



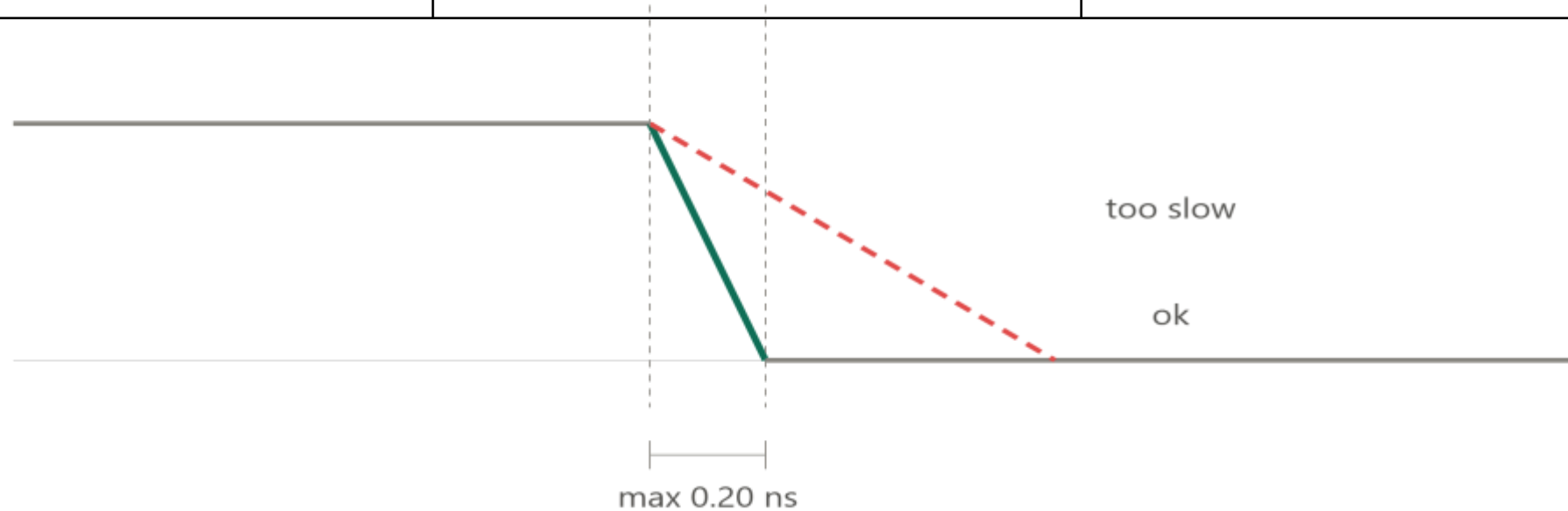
Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)



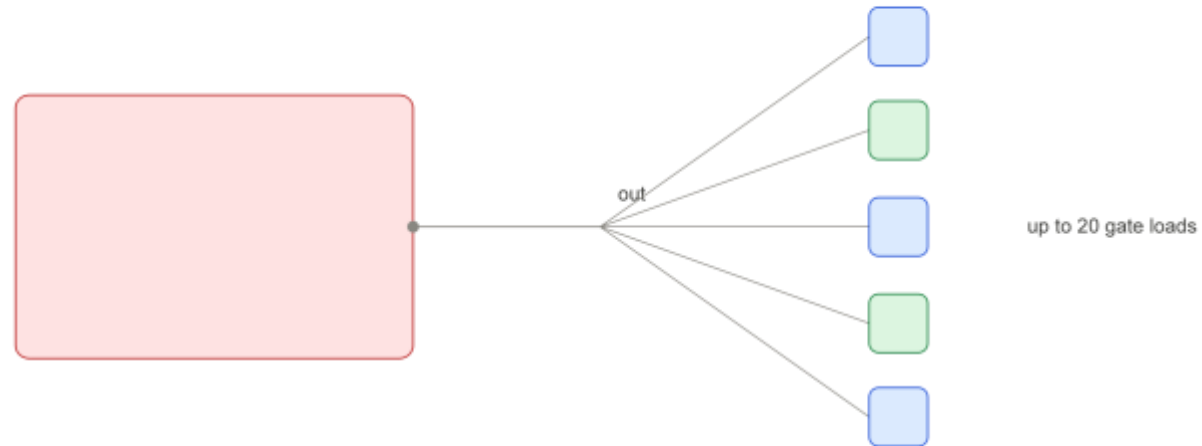
Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)
Max Transition	None	0.20 ns (200ps)



Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)
Max Transition	None	0.20 ns (200ps)
Max Fanout	None	20



Transitioning from Generic SDC to Realistic SAED32 Constraints

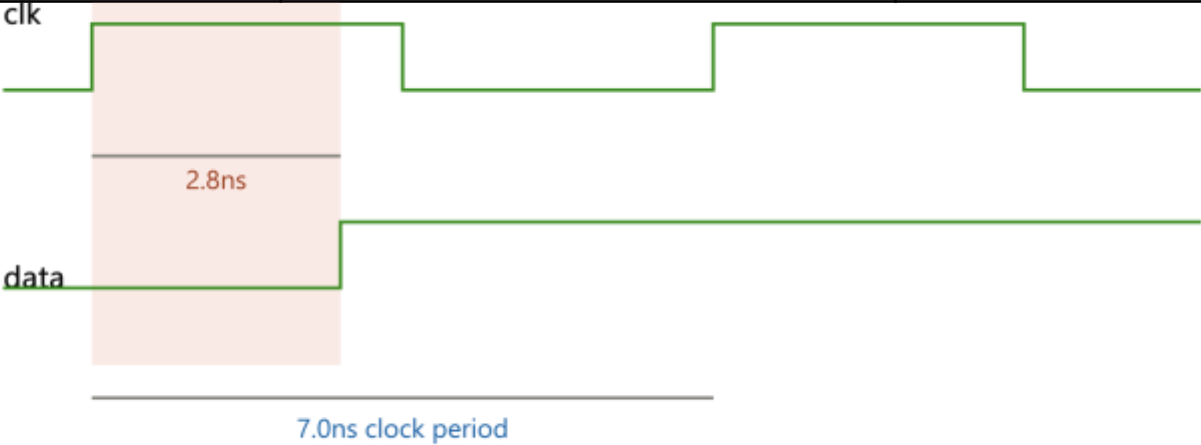
SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)
Max Transition	None	0.20 ns (200ps)
Max Fanout	None	20
RAM Power Paths	Unconstrained	set_false_path



false path — excluded from timing checks

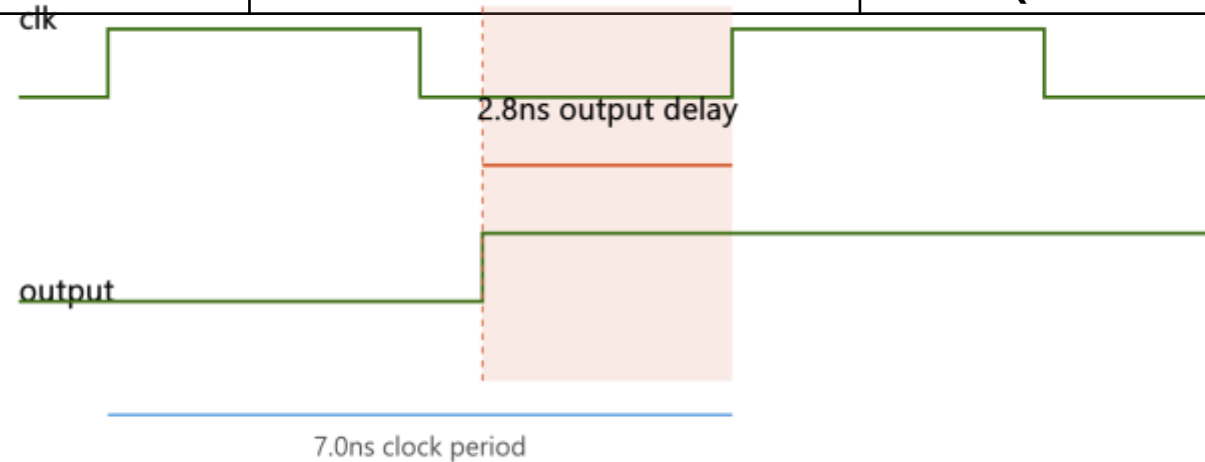
Transitioning from Generic SDC to Realistic SAED32 Constraints

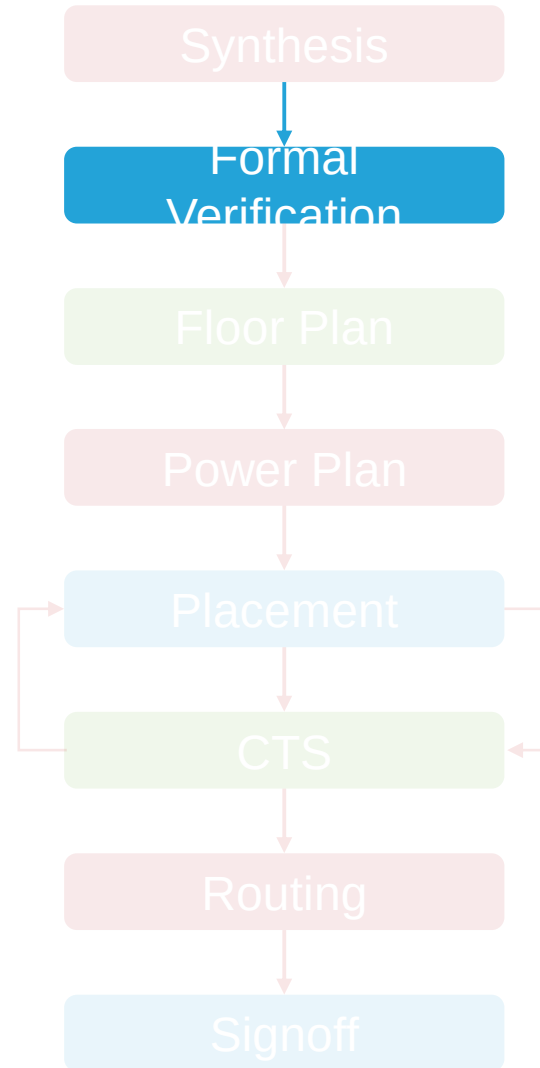
SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)
Max Transition	None	0.20 ns (200ps)
Max Fanout	None	20
RAM Power Paths	Unconstrained	set_false_path
Input Delay	None	2.8 ns (40% of clock period)



Transitioning from Generic SDC to Realistic SAED32 Constraints

SDC Constraint	Generic / Raw NVDLA SDC	Our Enhanced SAED32 SDC
Clock Period	Unconstrained / 16nm target	7.0 ns (143 MHz)
Clock Uncertainty	0.0 ns	Setup: 0.3ns / Hold: 0.1ns
Input Drive	Infinite / Ideal	IBUFX4_RVT Buffer
Output Load	0.0 pF	0.12 pF (120 fF)
Max Transition	None	0.20 ns (200ps)
Max Fanout	None	20
RAM Power Paths	Unconstrained	set_false_path
Input Delay	None	2.8 ns (40% of clock period)
Output Delay	None	2.8 ns (40% of clock period)

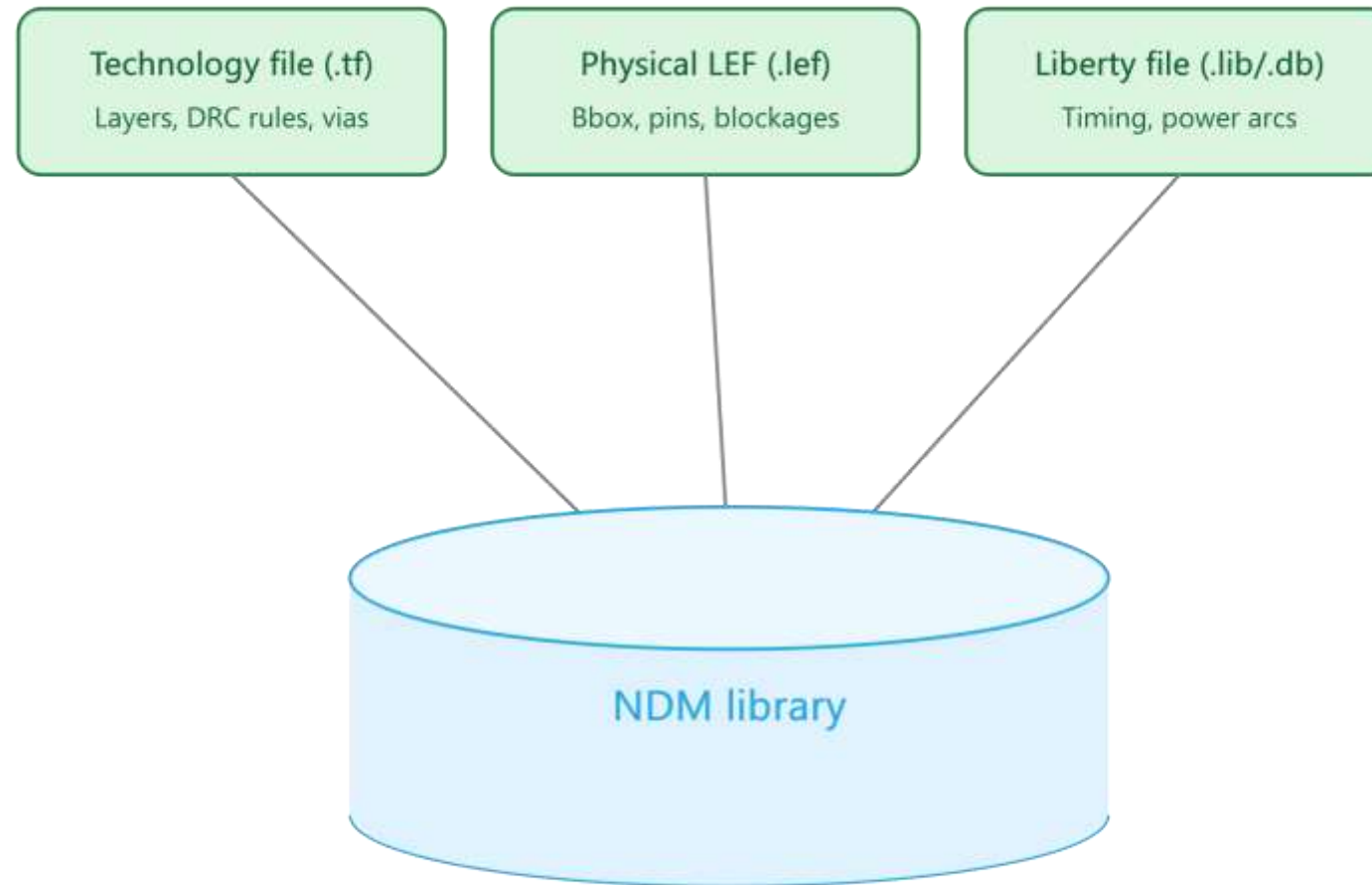


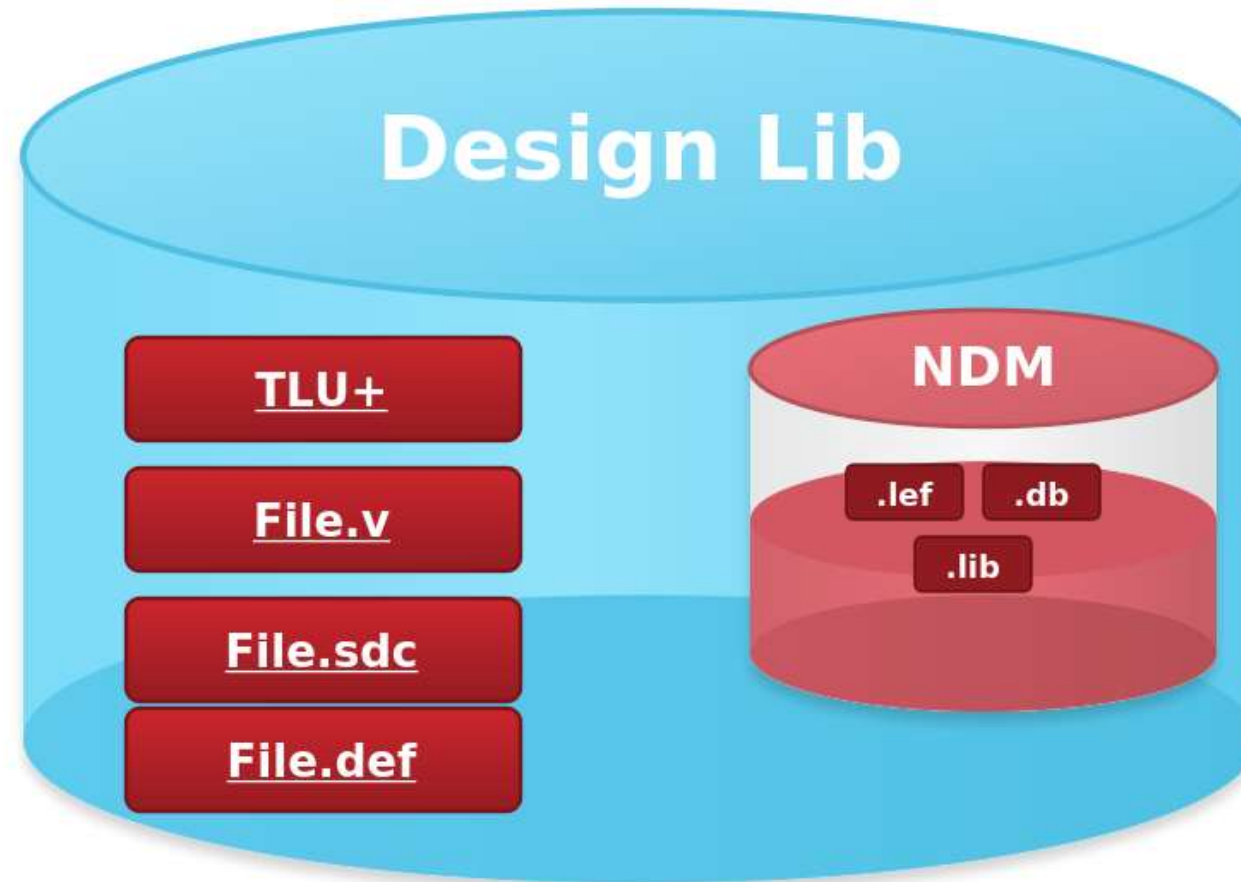


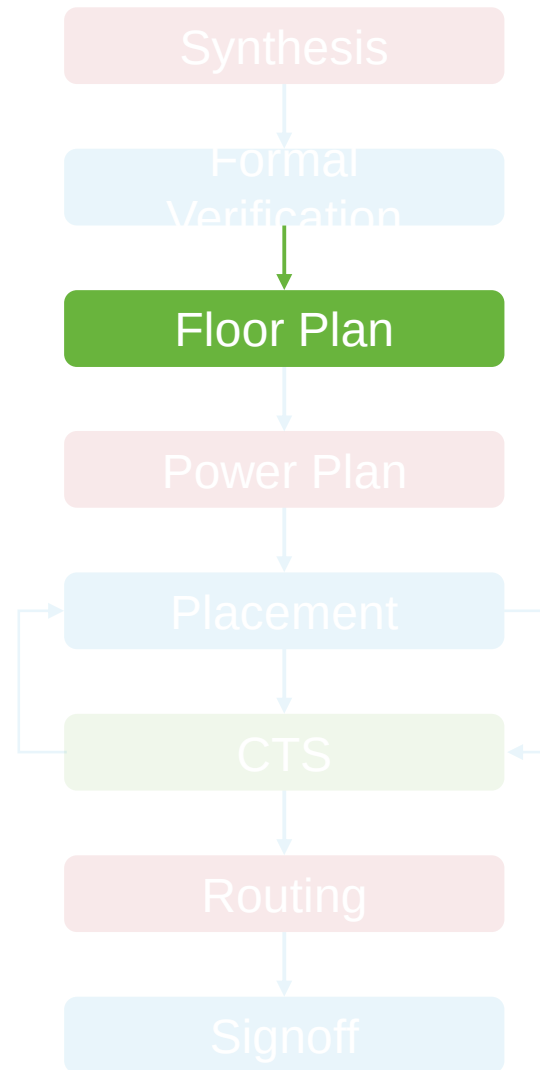


- Libs
- Memory mapping
- Constraints
- Formality
- **MMMC and library creation**









1. Initialize Floor Planning
2. I/O Ports Placed
3. Macro Placed
4. Add Blockage
5. Optimized Floorplan

- Define the `core_utilization` = 0.35

Area

```
-----  
Combinational Area:    277126.2447  
Noncombinational Area: 255764.1743  
Buf/Inv Area:          25268.5217  
Total Buffer Area:     6092.0861  
Total Inverter Area:   19176.4355  
Macro/Black Box Area: 316260.4834  
Net Area:              337046.1221  
-----
```

```
-----  
Cell Area:             849150.9024  
Design Area:           1186197.0245  
-----
```

- Utilization = Cell Area / core_area
- core_area = Cell Area / Utilization
- core_area = 2.426 (mm²)



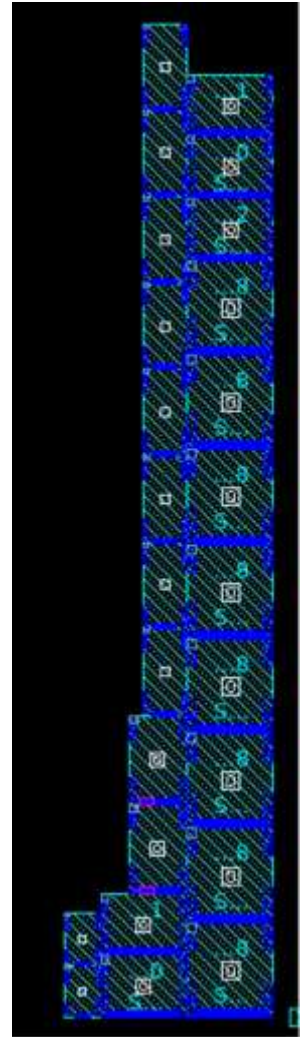
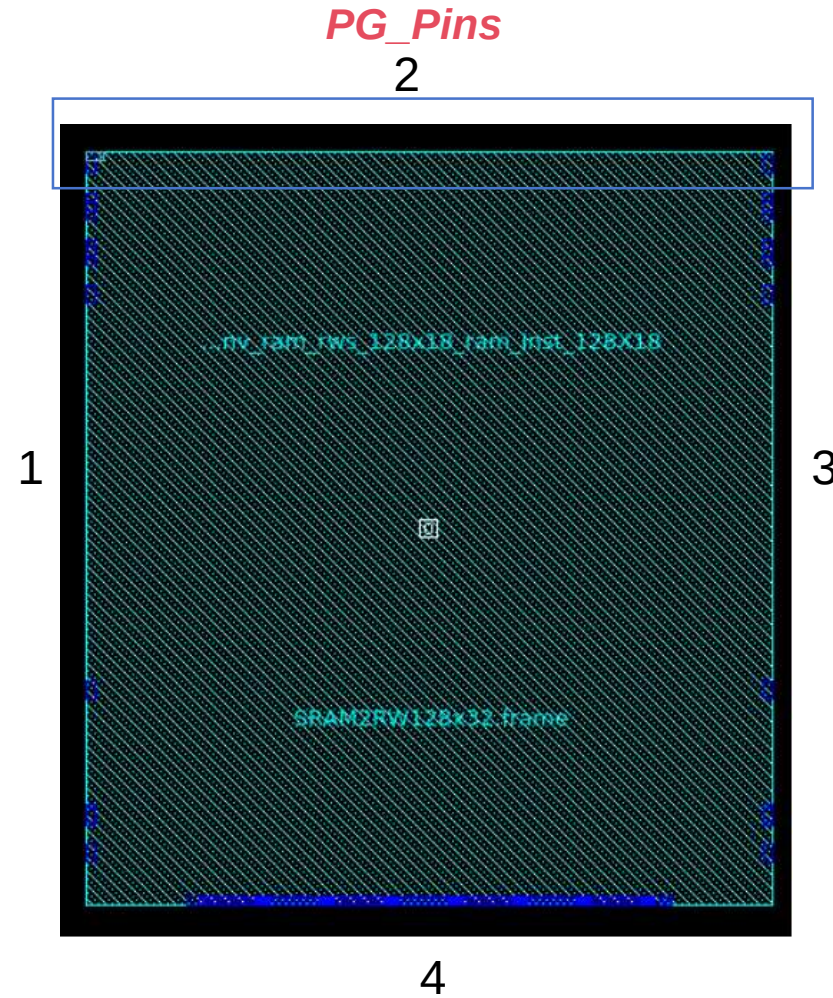
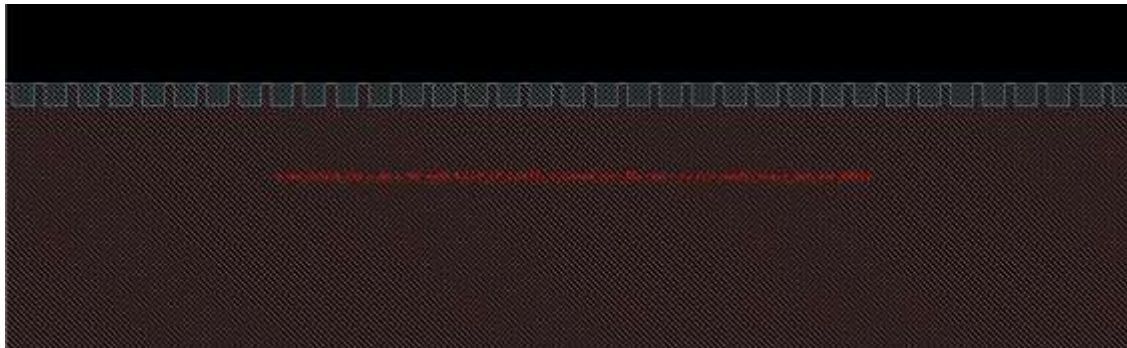
- Auto Placement Ports (M2 | M3)

```
icc2_shell>  
icc2_shell> place_pins -ports [get_ports *]  
  
Min routing layer: M1  
Max routing layer: M9  
  
Number of block ports: 1750  
Number of block pin locations assigned from router: 0  
Number of PG ports on blocks: 0  
Number of pins created: 1750  
CPU Time for Pin Placement: 00:00:05.08u 00:00:00.00s 00:00:05.09e:  
Total Pin Placement CPU Time: 00:00:05.29u 00:00:00.00s 00:00:05.30e:  
1
```



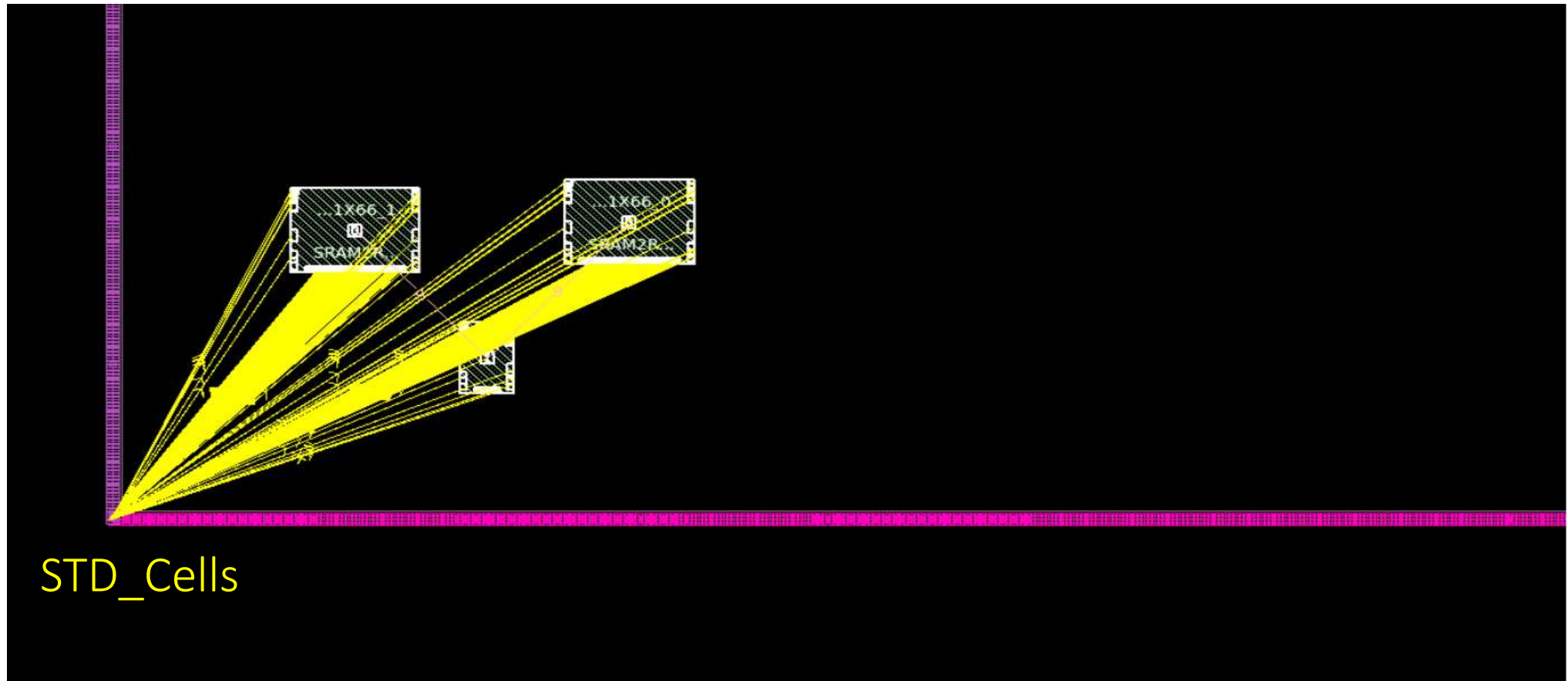
Allowed directions

```
MACRO SRAM2RW128x16  
  CLASS BLOCK ;  
  SOURCE USER ;  
  ORIGIN 0 0 ;  
  SIZE 87.63 BY 135.673 ;  
  SYMMETRY X Y R90 ;
```

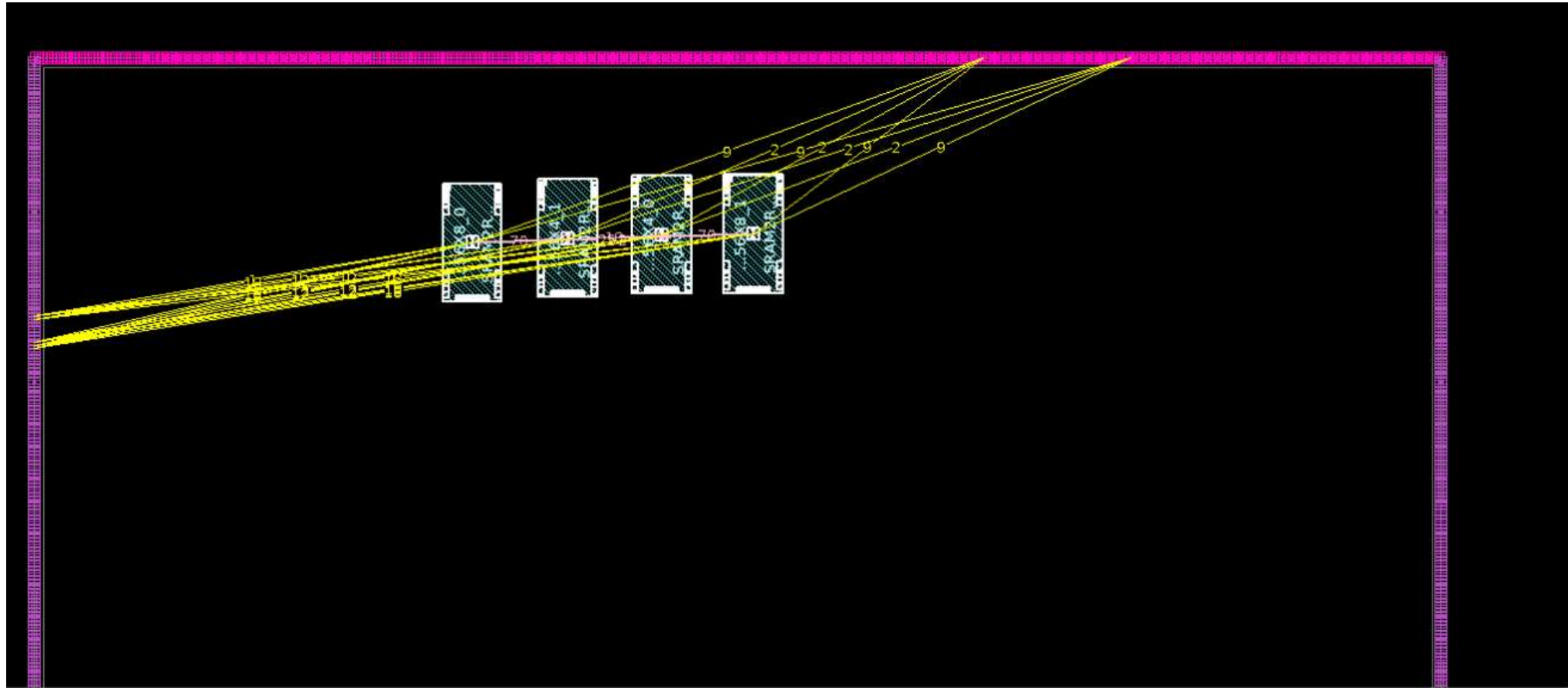


- We started by analyzing the connectivity between the macros and identifying their relationships with the I/O ports using **flylines**.
- Net_Connections.
- Data_Flow.
- Register_Tracing.

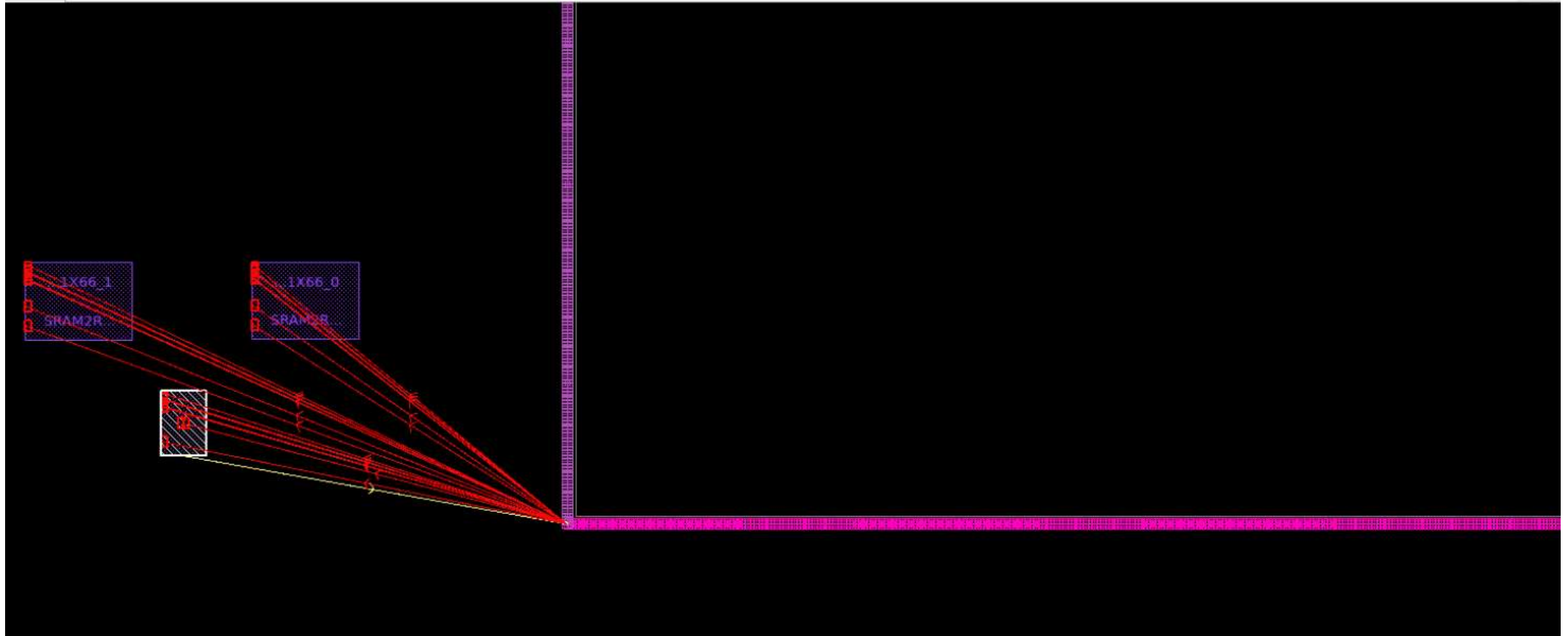
- **Net_Connections:** Displays **direct electrical connections** from macro pins to the first standard cell, other macros, or I/O ports to optimize orientation and channel spacing.



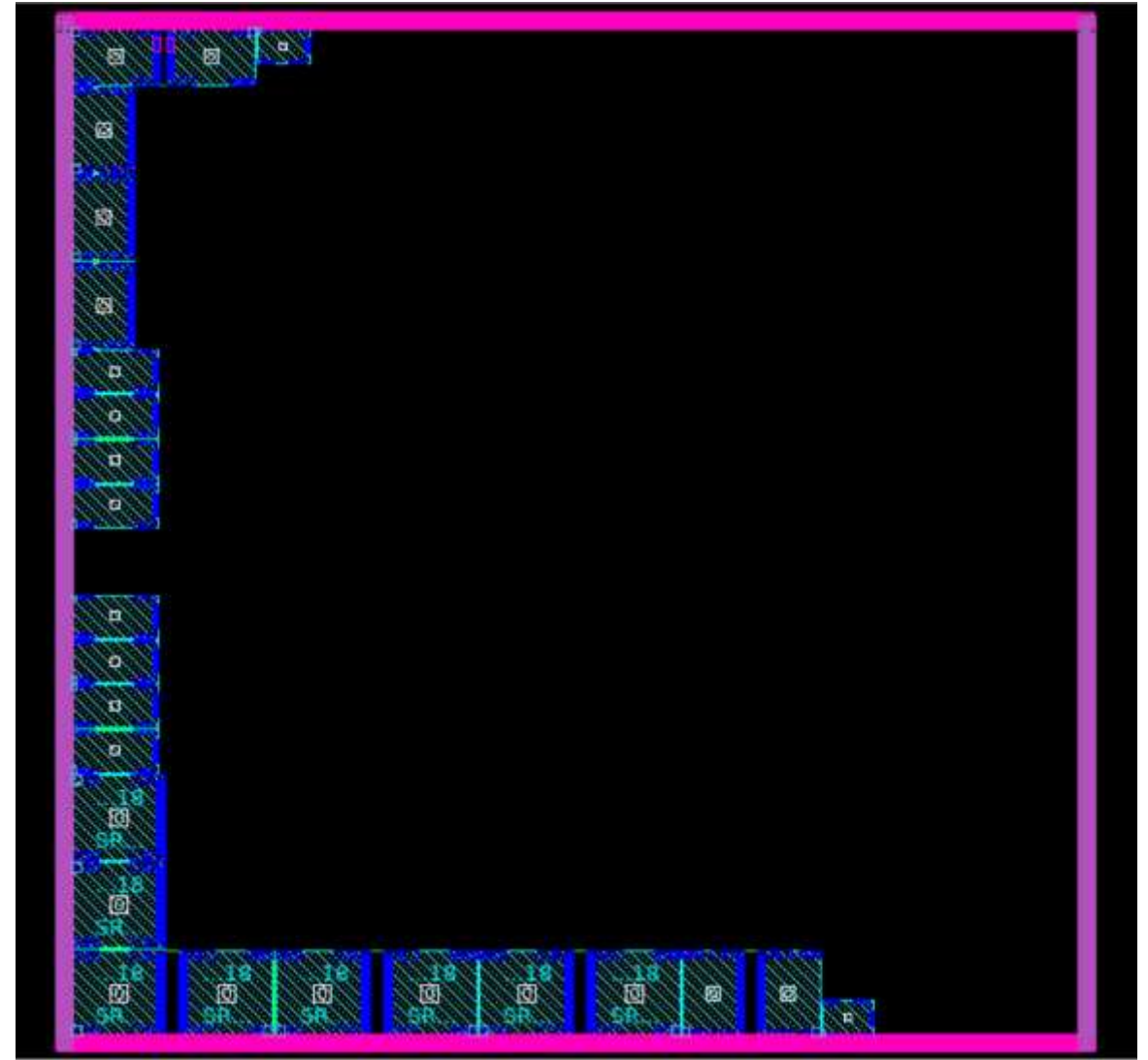
- **Data_Flow:** Visualizes the high-level architectural communication between macro groups and I/O ports to guide the overall floorplan topology and minimize total wirelength.



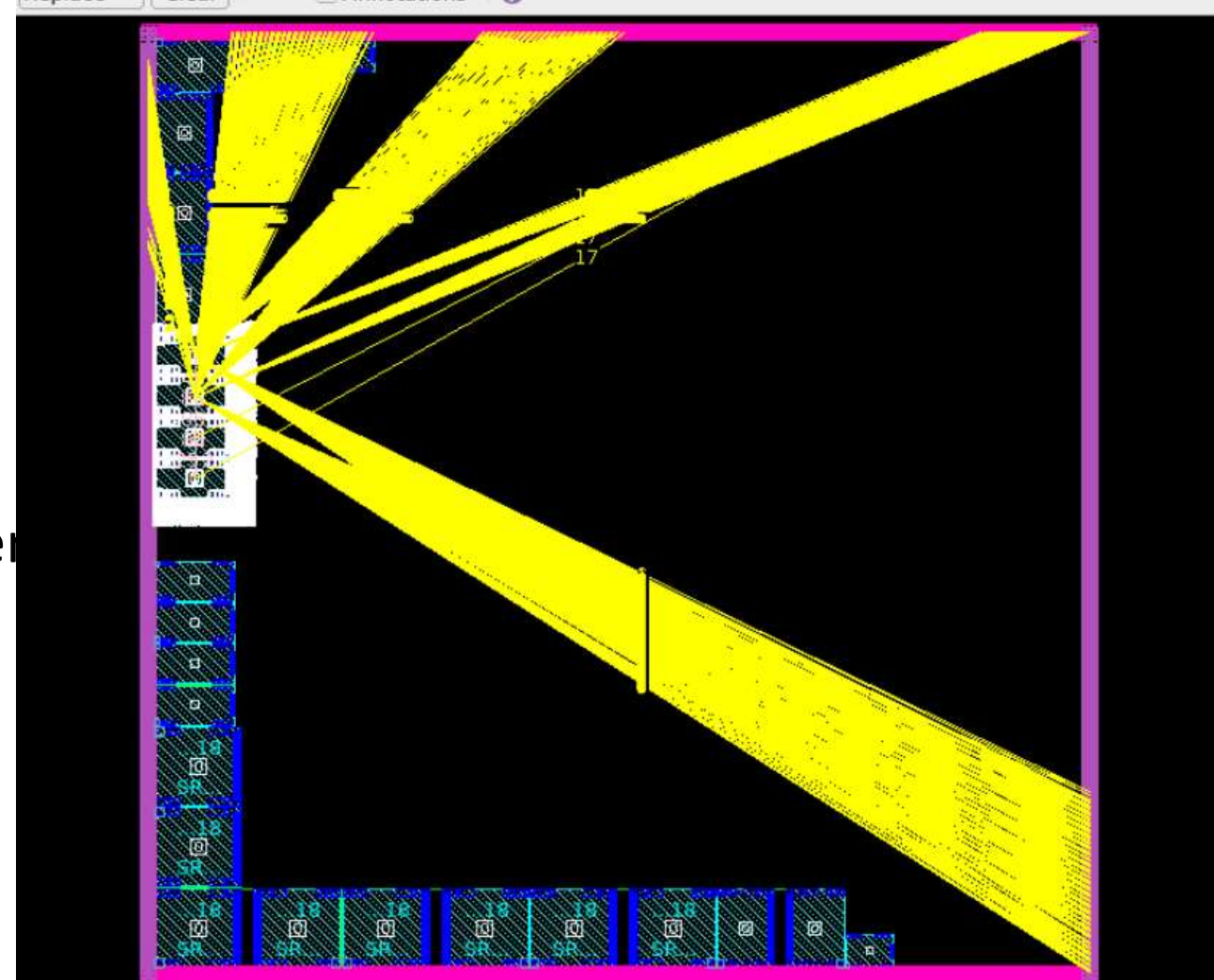
- **Register_Tracing**: Tracks the number of intermediate flip-flops (pipeline stages) between macros to reveal **timing criticality** and dictate their required physical proximity.



- However, the automatically generated macro placement was inefficient and did not follow recommended macro placement techniques, resulting in poor floorplan quality.

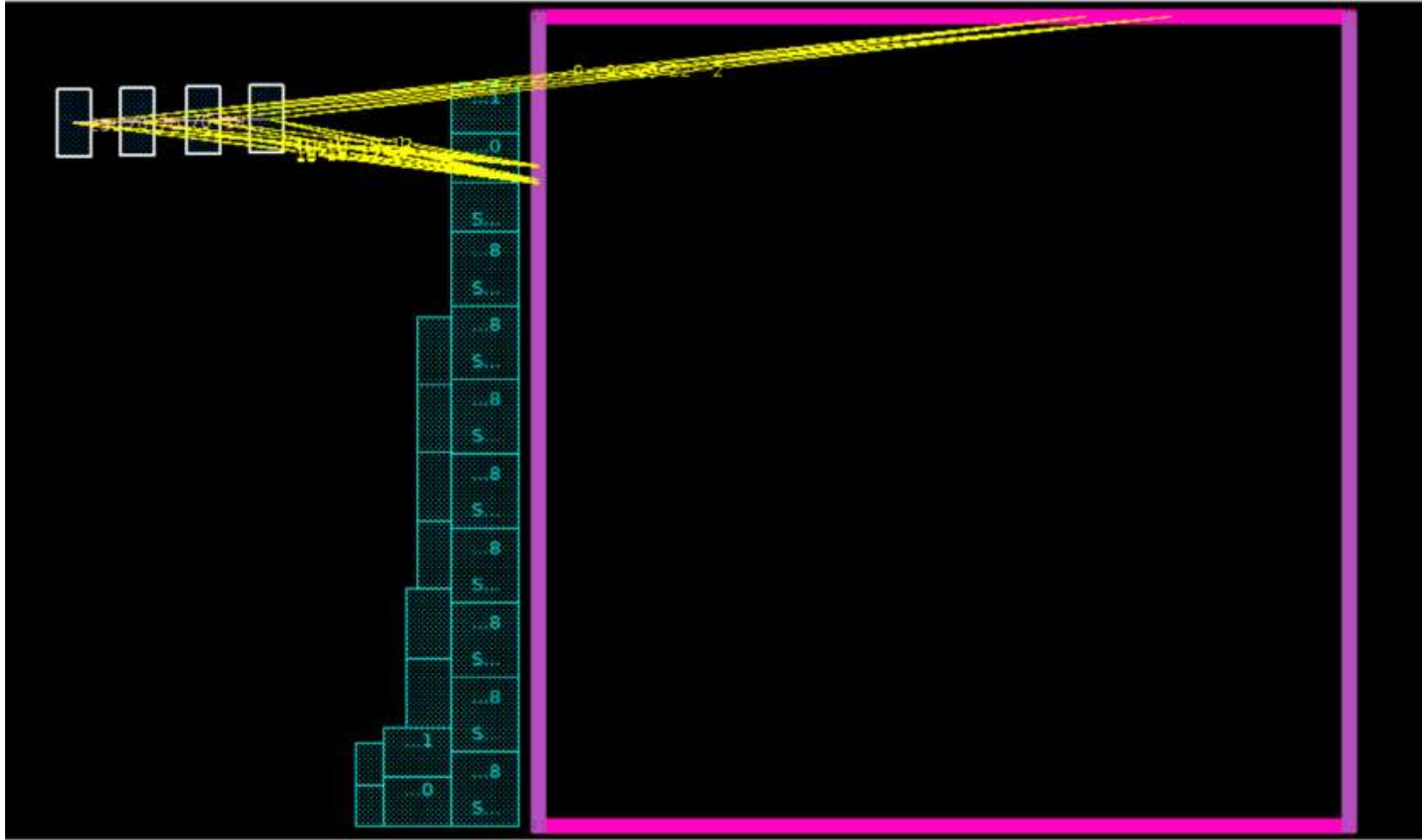


- The I/O ports were not aligned with the macro pins.
- We decided to relocate the macros following the recommended macro placement guidelines. In addition, we optimized the I/O port locations to better align with the macro pins, resulting in shorter interconnects and improved routability.



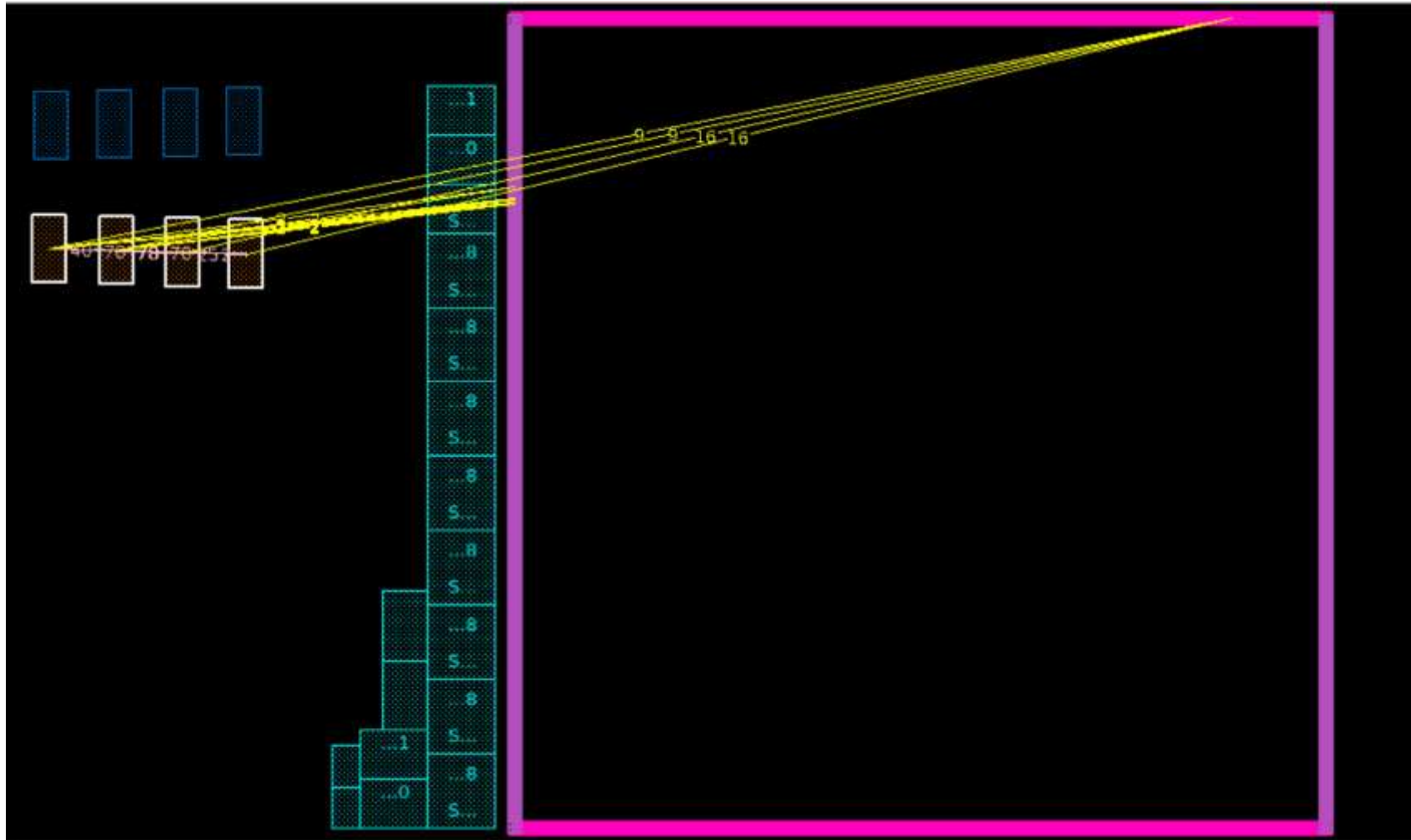
Dividing the macros into different groups

Group_1 communicates with each other and I/O Ports



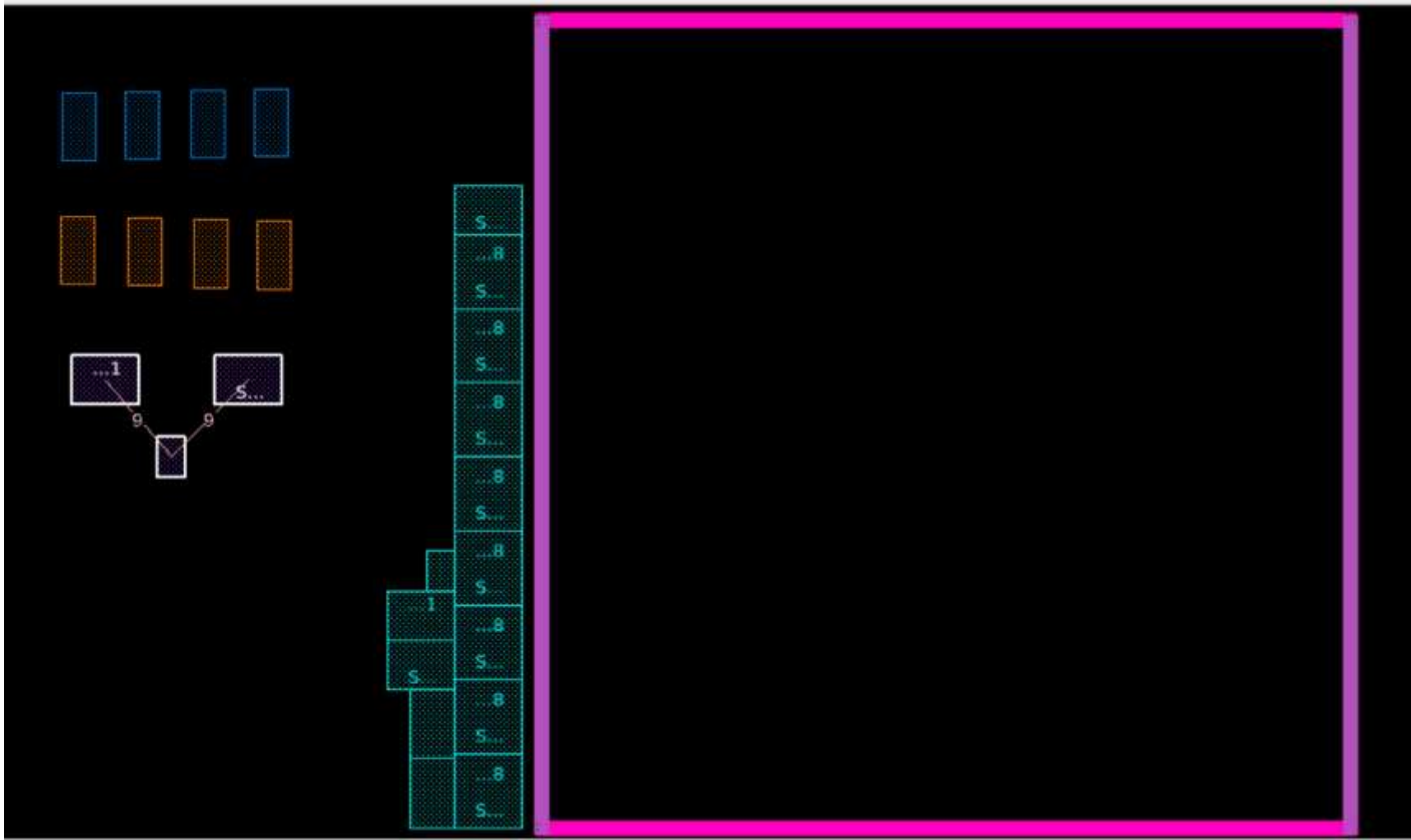
Dividing the macros into different groups

Group_2 communicates with each other and I/O Ports



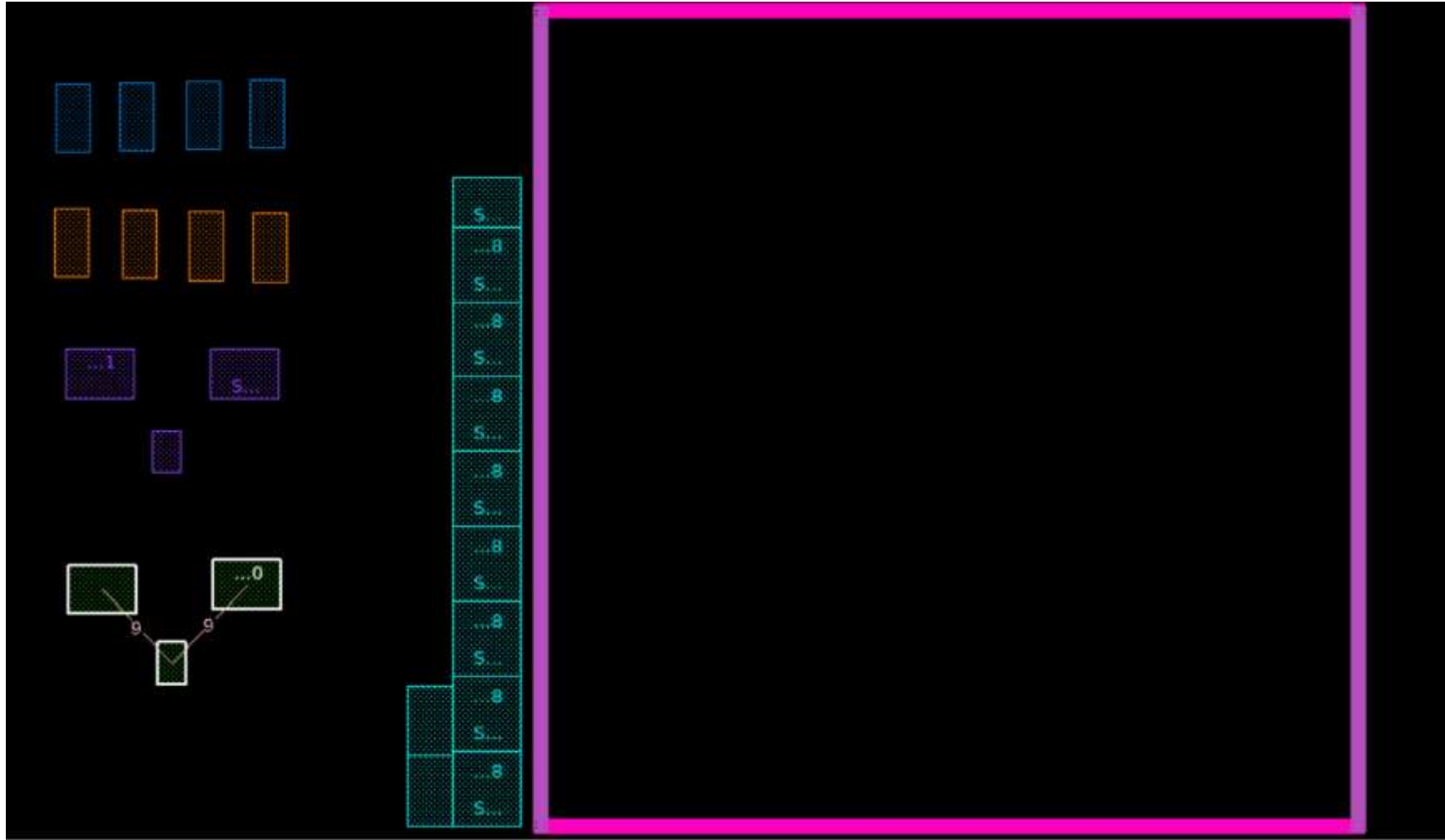
Dividing the macros into different groups

Group_3 communicates with each other and STD cells



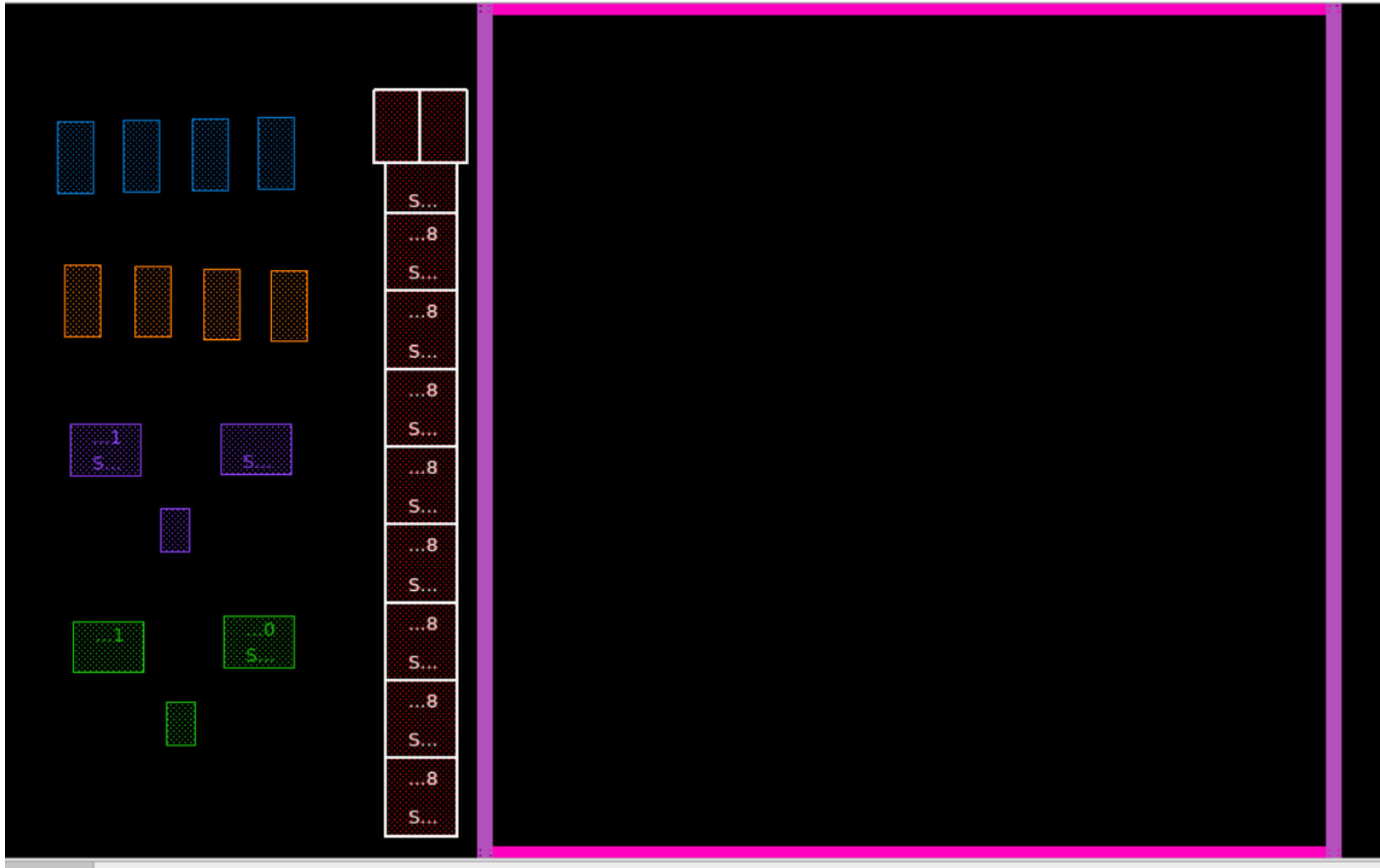
Dividing the macros into different groups

Group_4 communicates with each other and STD cells

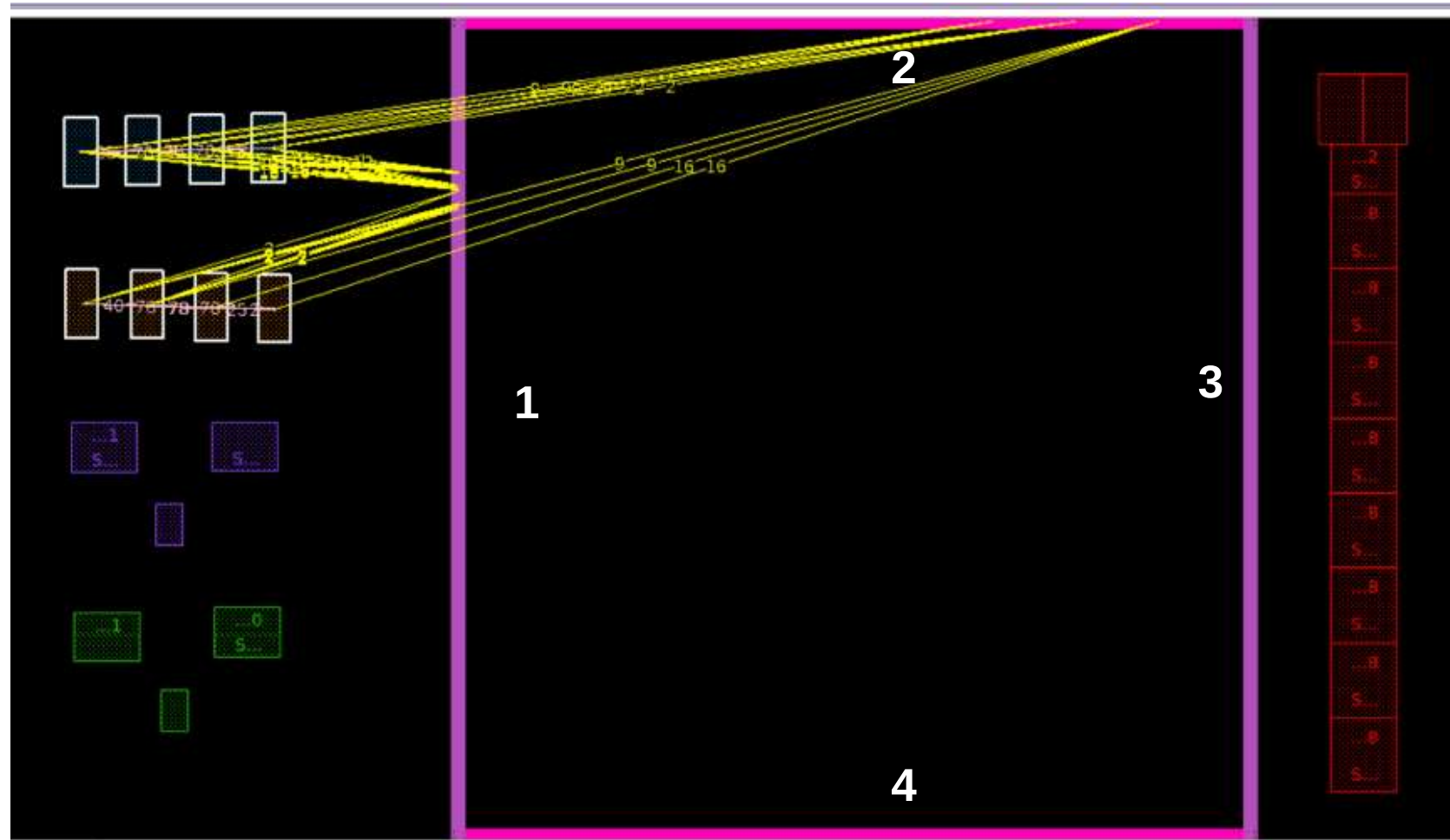


Dividing the macros into different groups

Group_5 communicates with STD cells only.



- Based on the [Flylines](#) , we found that Groups 1 and 2 communicate with the I/O ports on Sides 1 and 2

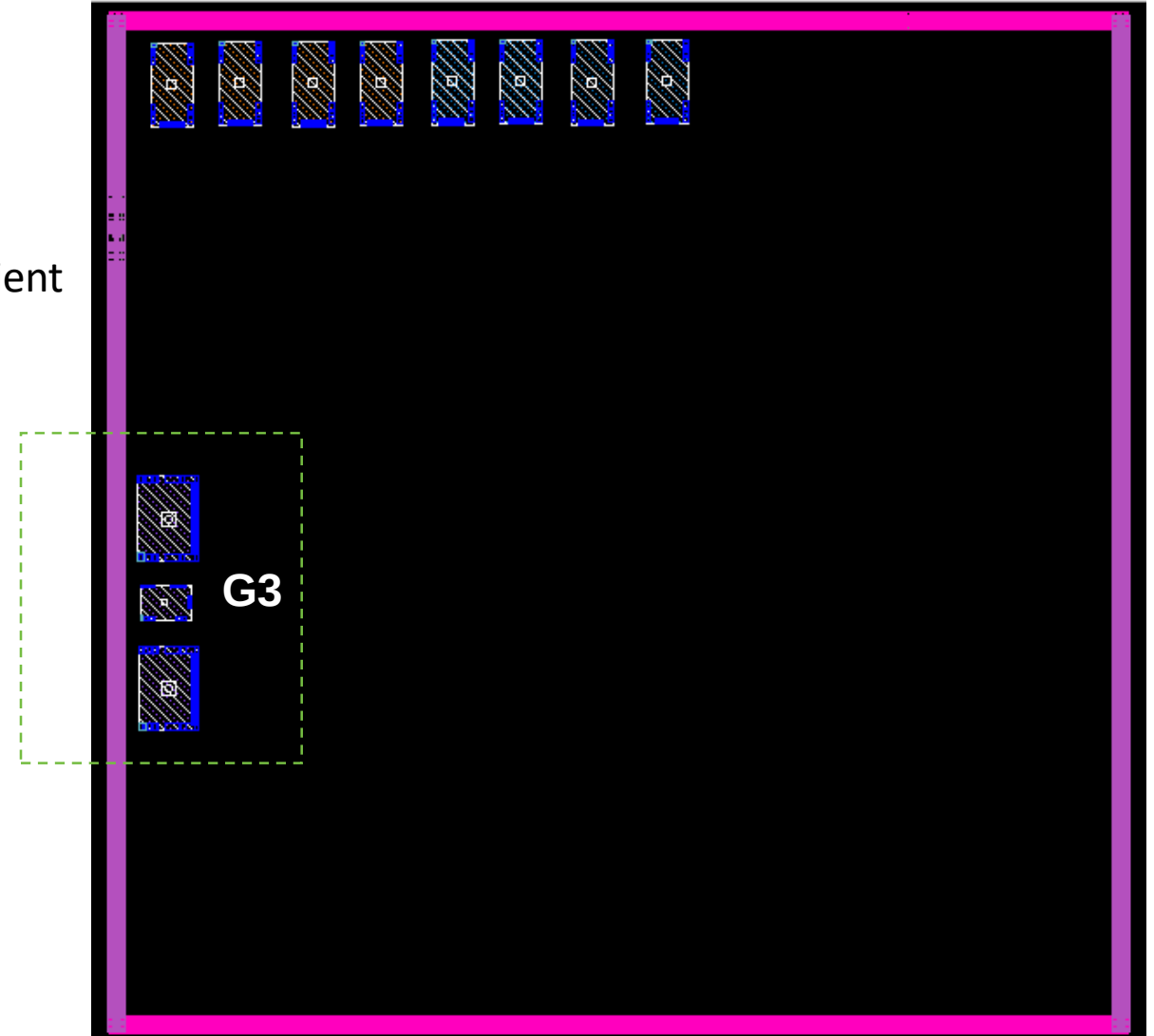


Manual Macro Placement

- We started by placing each macro group in its appropriate location based on the fly lines.
- Placed **Group_1** and **Group_2** in the Core Area



- Placed **Group_3** in the Core Area
- Rotated macros in **Group_3** by 90° **counterclockwise** to orient their top sides toward the core boundary.



- Placed **Group_4** in the Core Area
- Rotated macros in **Group_4** by 90° **counterclockwise** to orient their top sides toward the core boundary.



- Placed **Group_5** in the Core Area
- Rotated some macros in **Group_5** by 90° **clockwise** to orient their top sides toward the core boundary.

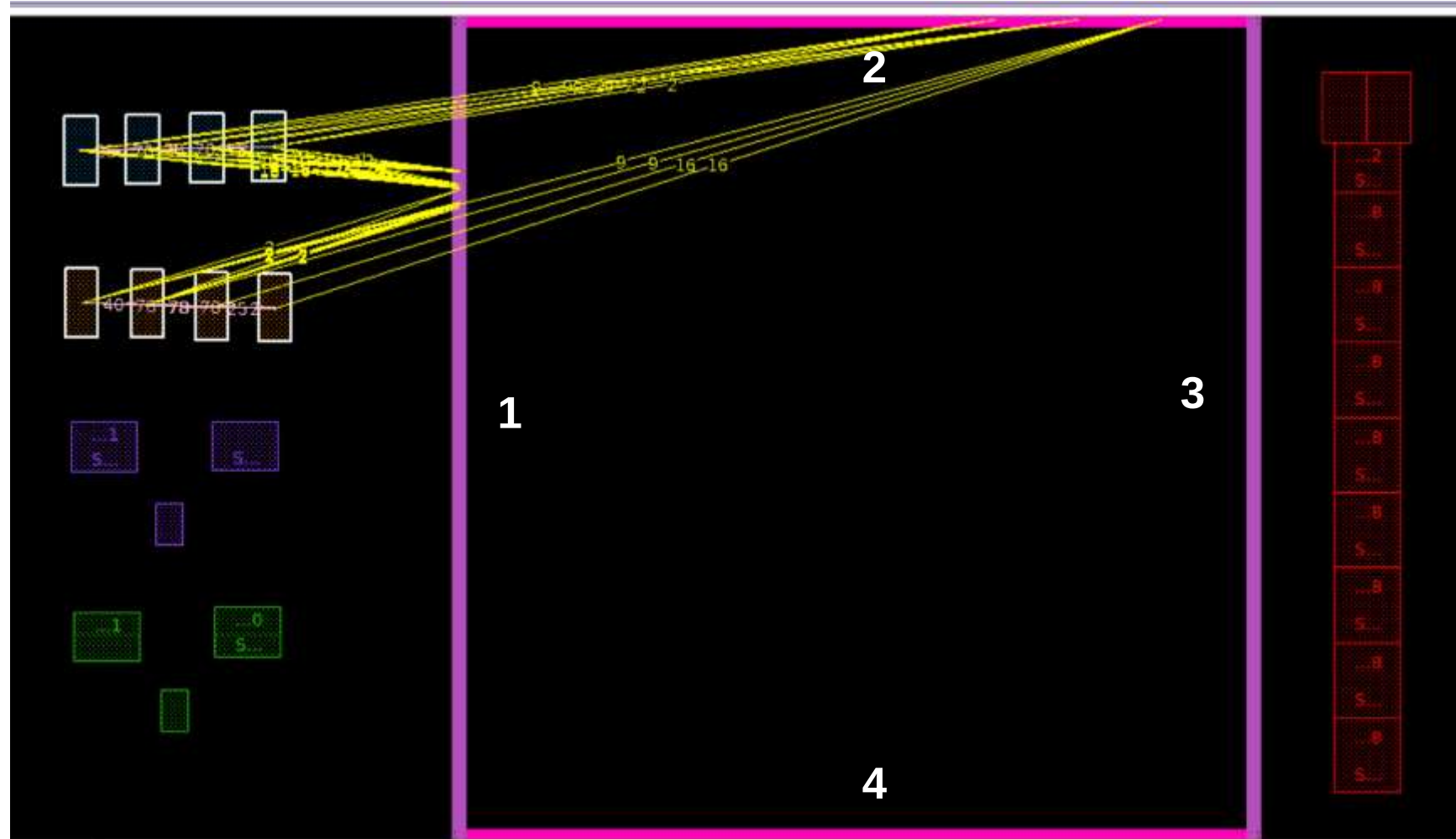


- After placing all the macro groups

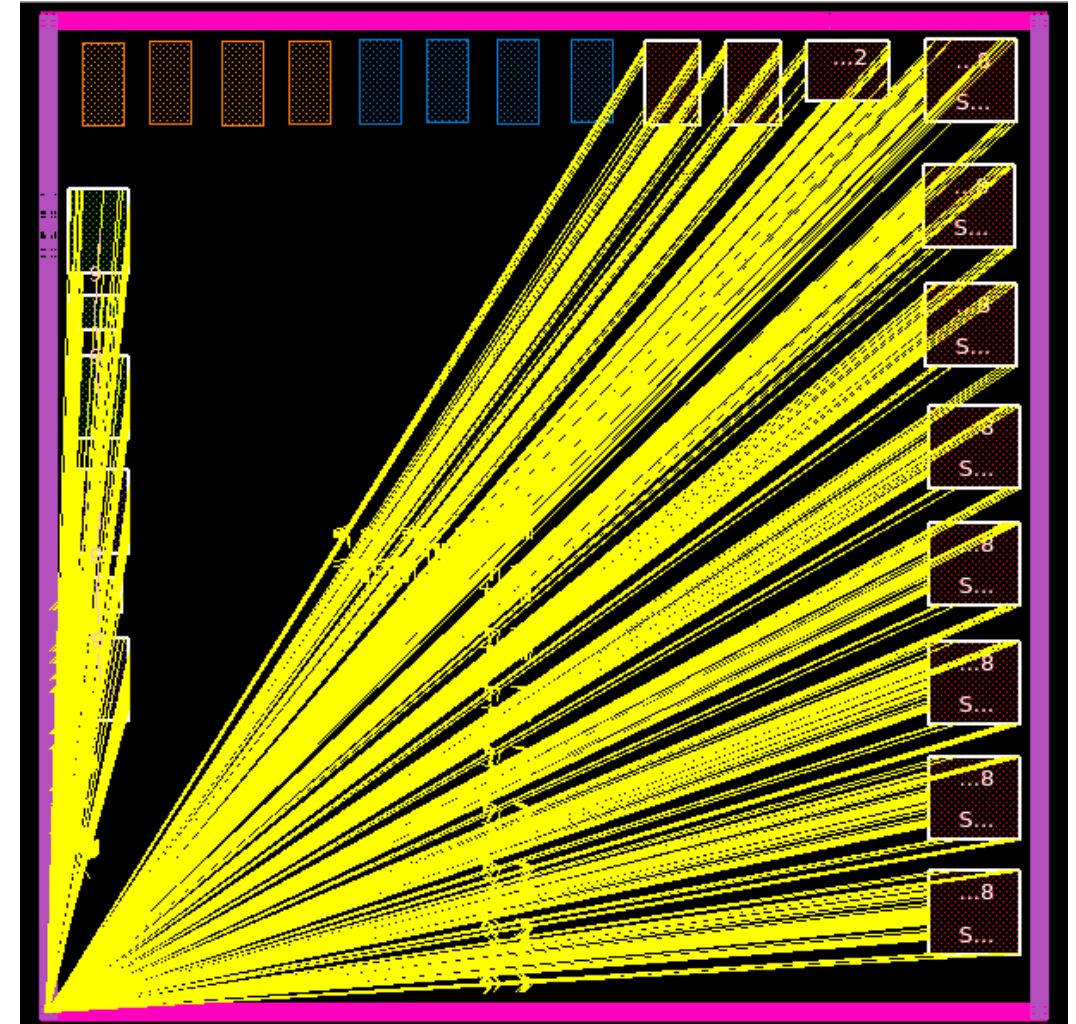
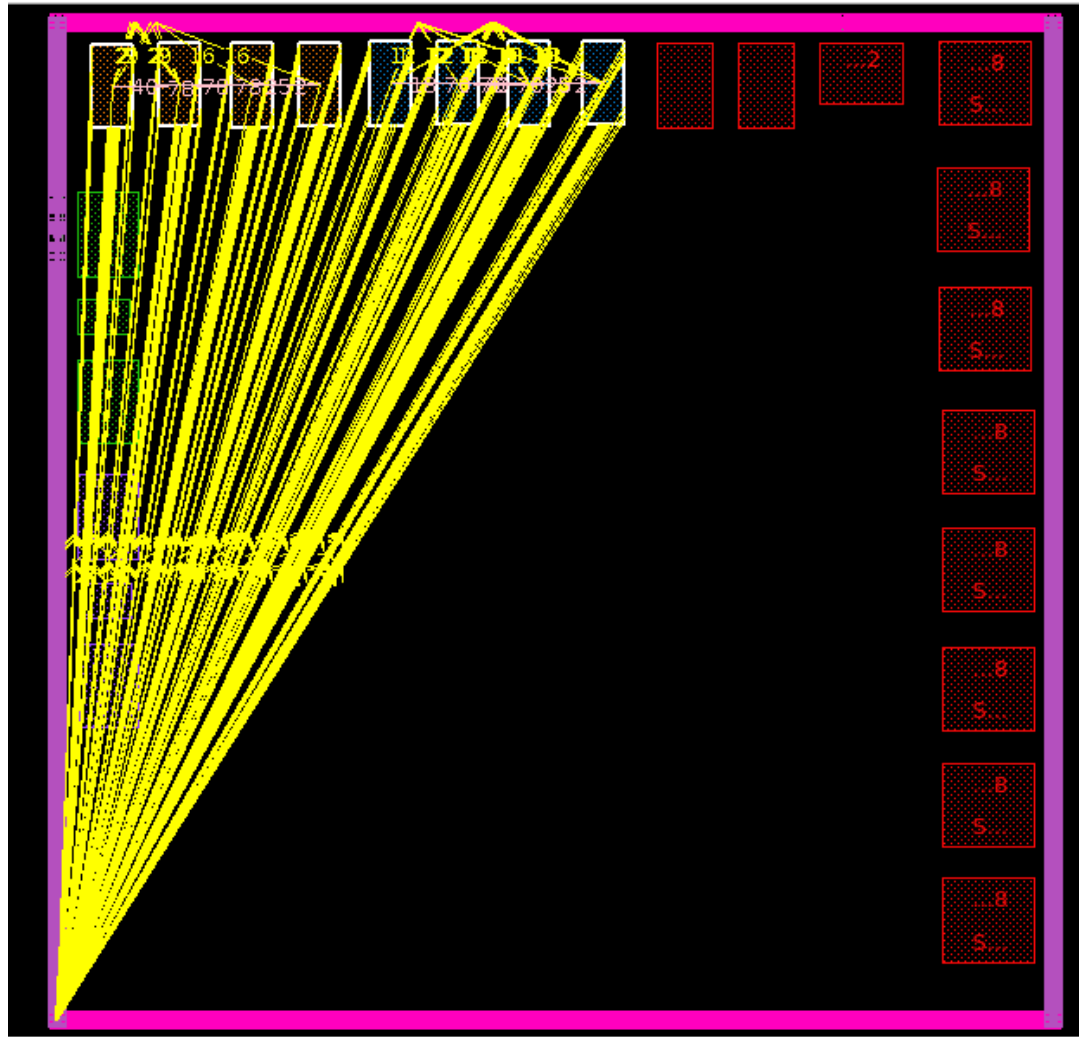


Adjusted the I/O Port Locations

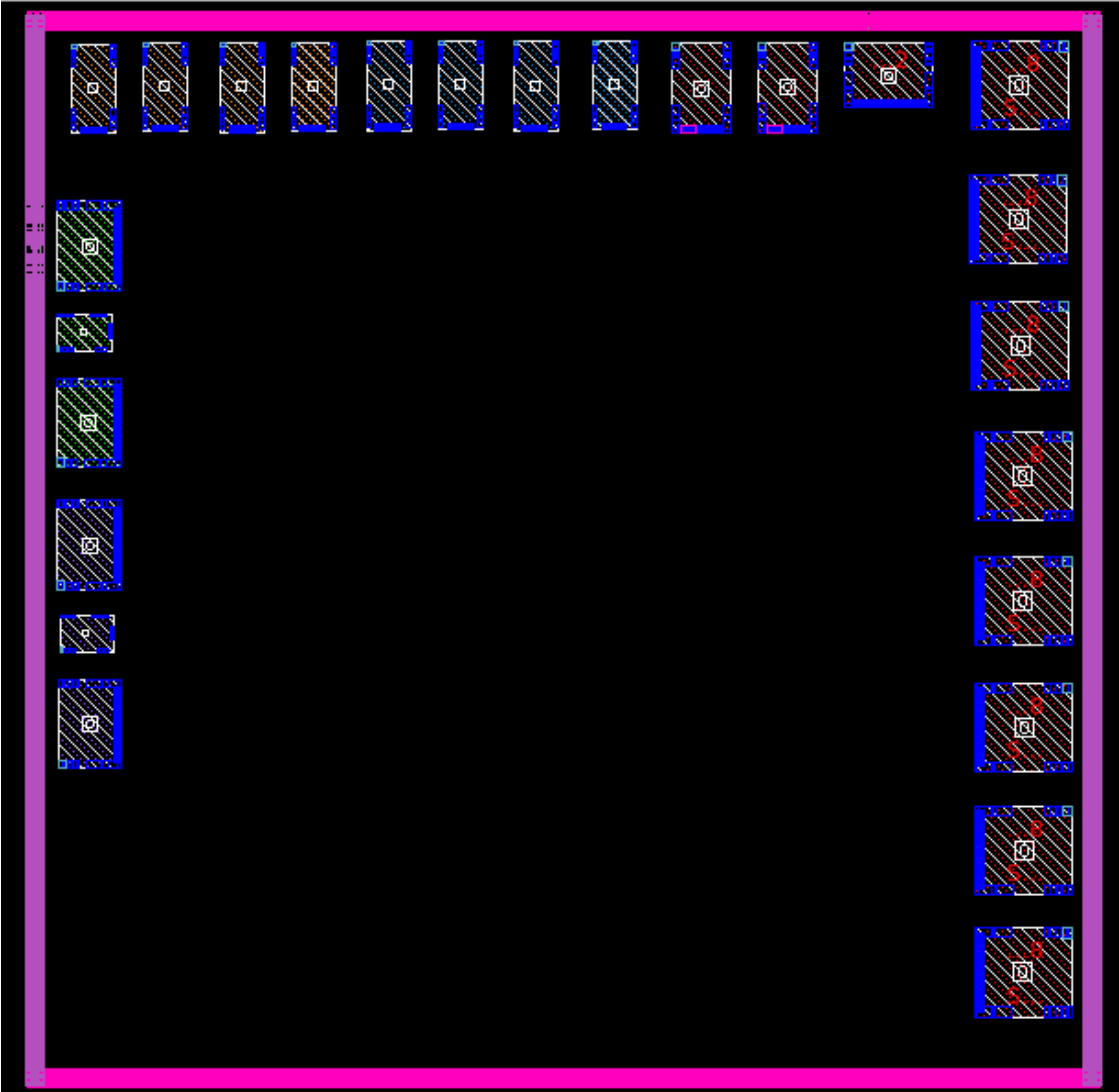
- `set_individual_pin_constraints`
- Adjust the I/O port locations to **align** with each macro group.



-

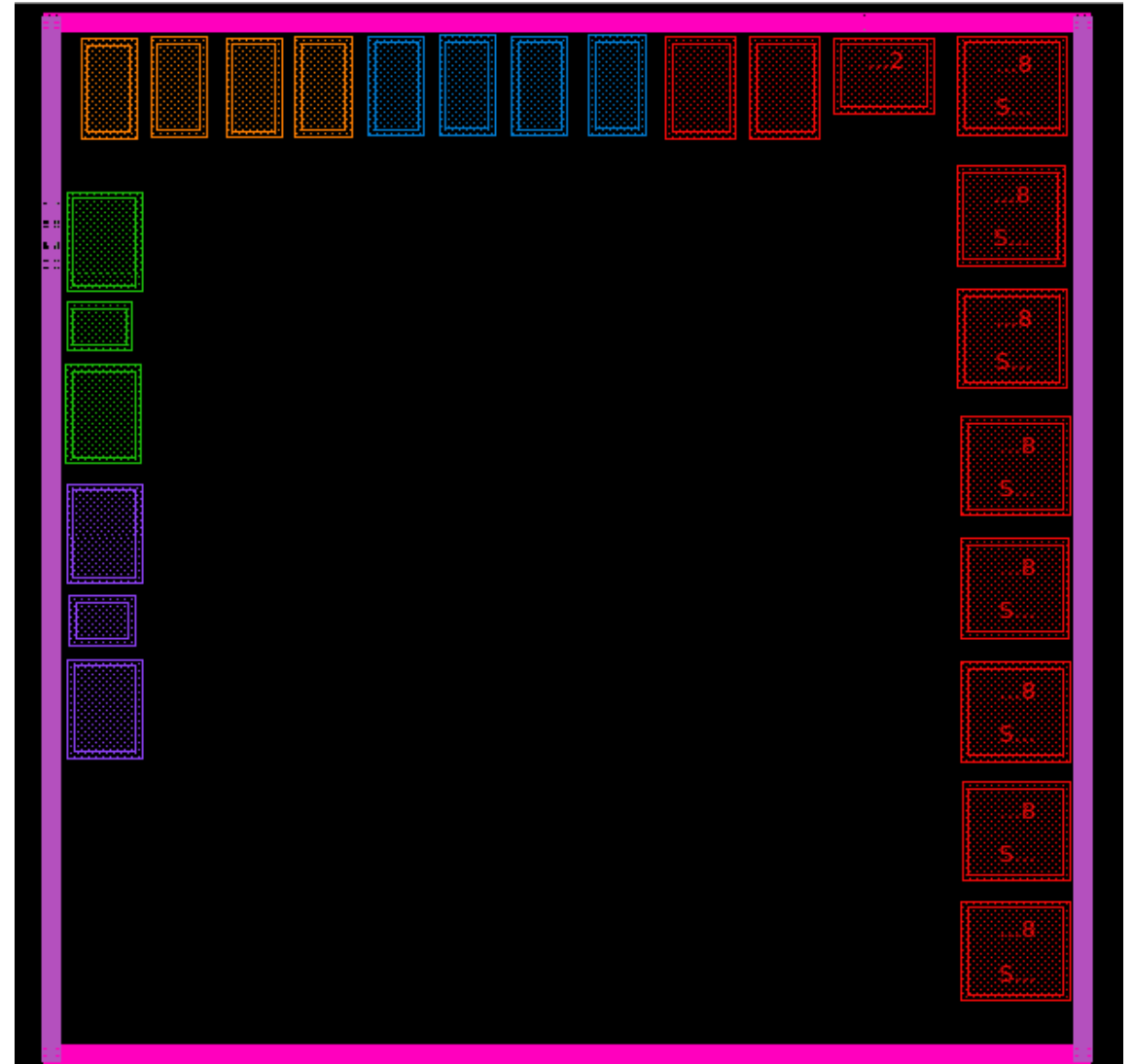
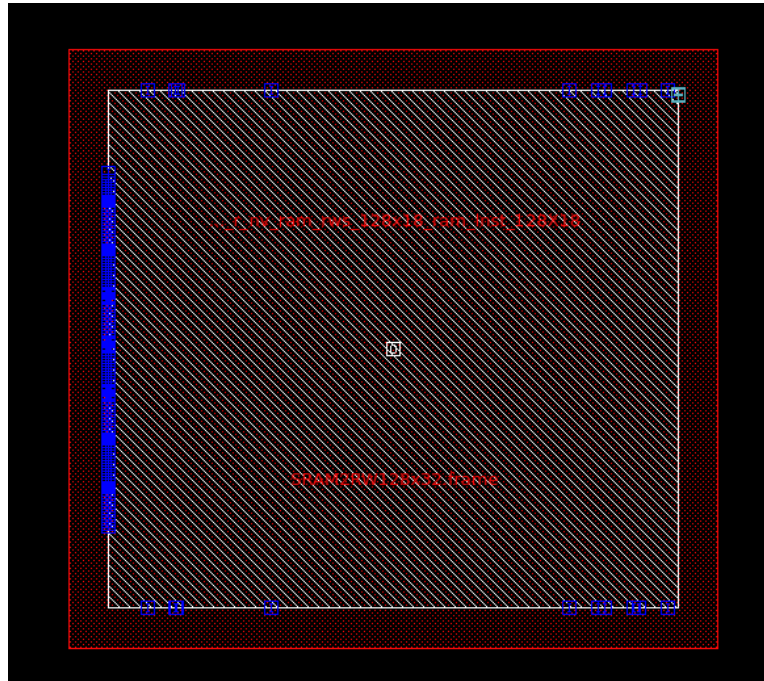


Final Floorplan View



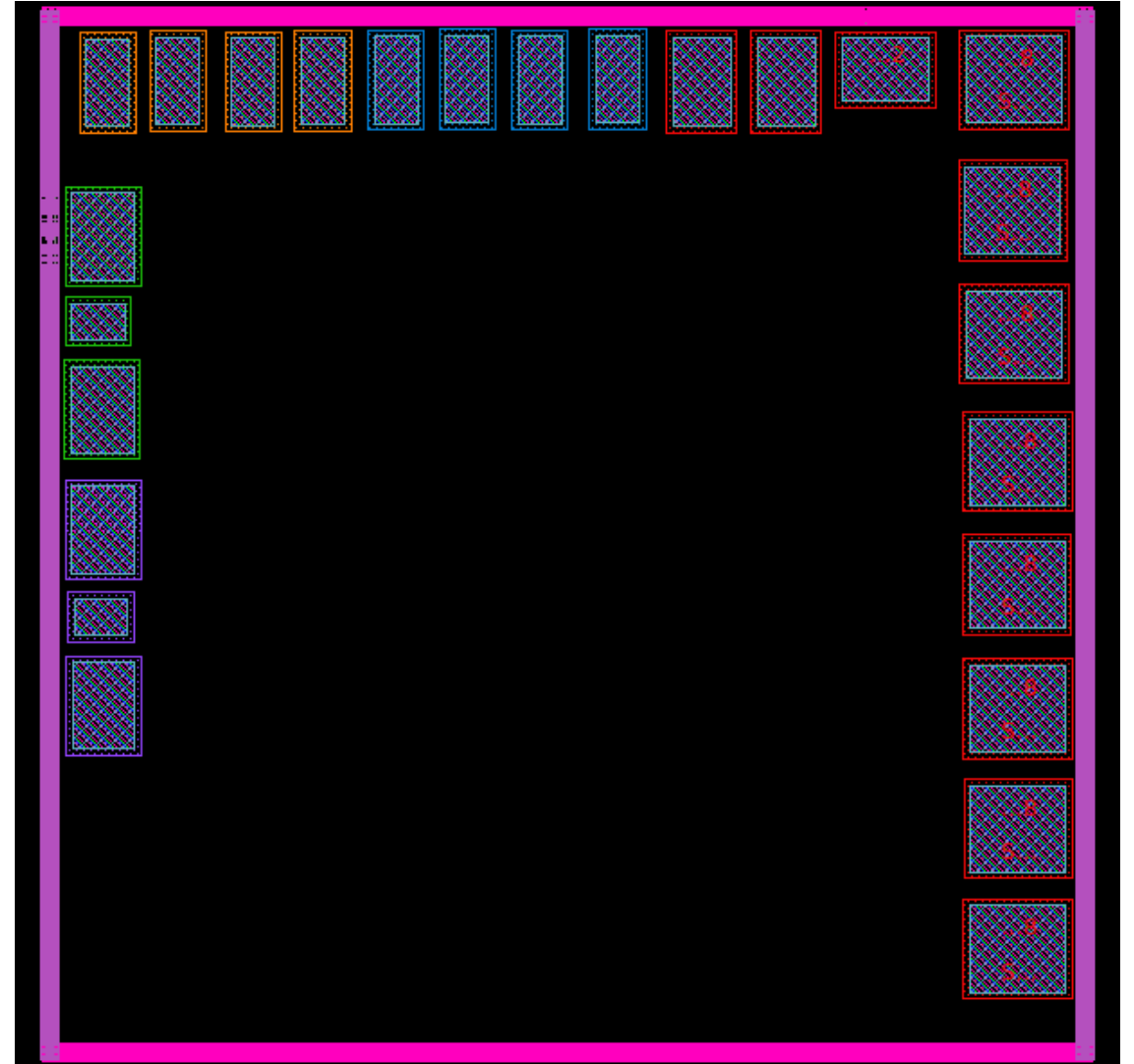
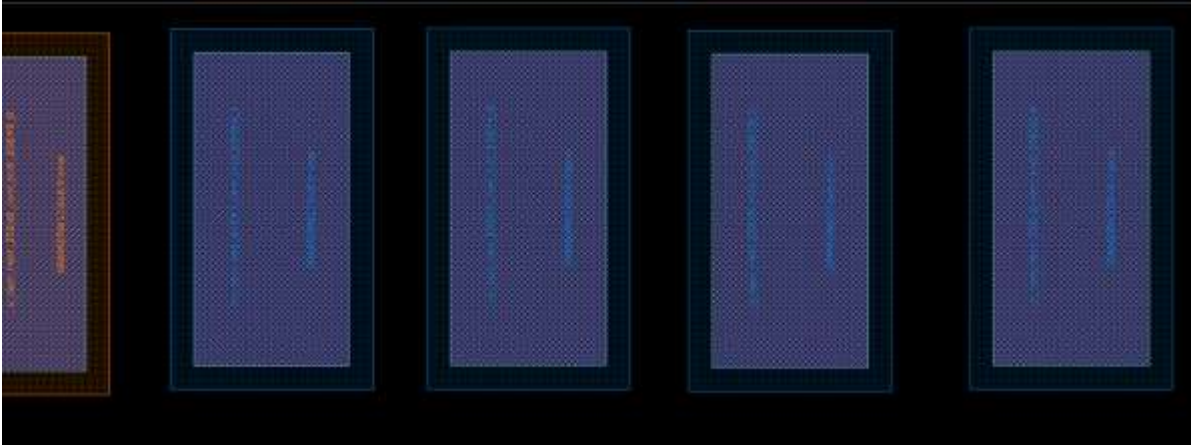
Add Blockages|Keepout Margin

- Keepout margins (Halo): Prevents standard cells from being placed too close to macro boundaries to preserve dedicated channel space for pin access and avoid placement congestion.



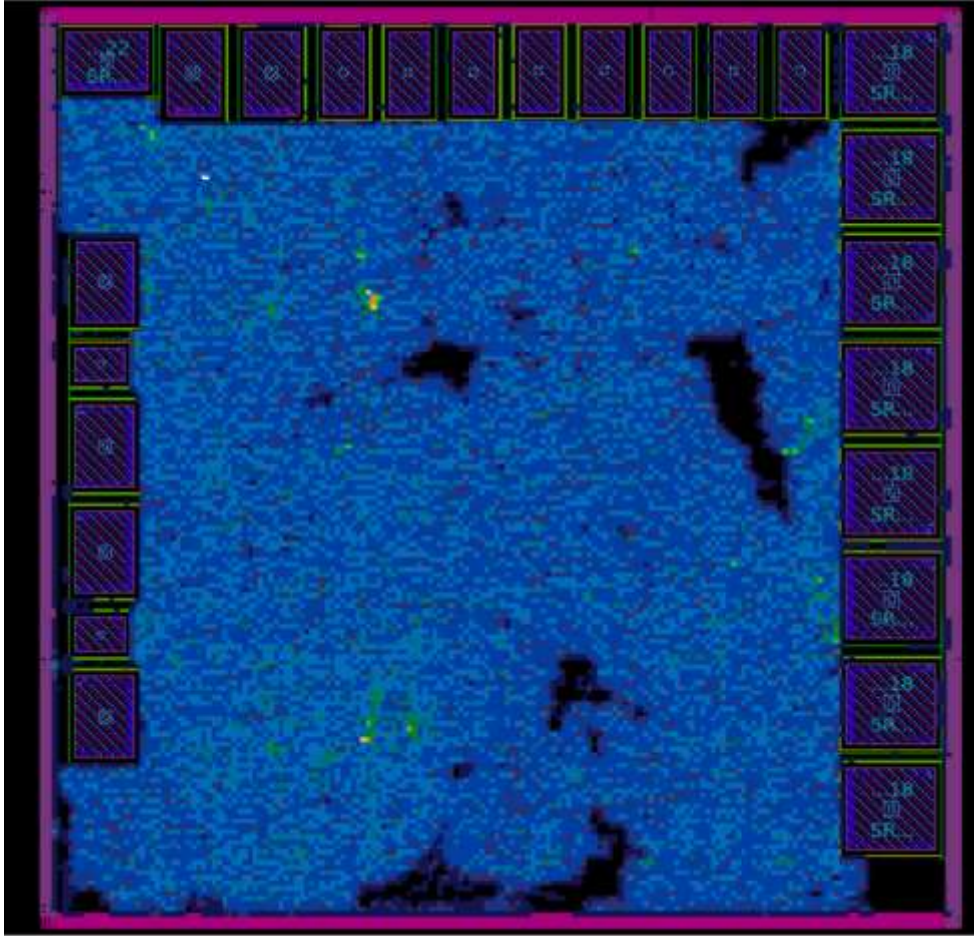
Add Blockages| Routing blockage

- Routing blockages: Restricts routing across specified metal layers over macros to prevent shorts with internal layout circuitry, minimize crosstalk noise.

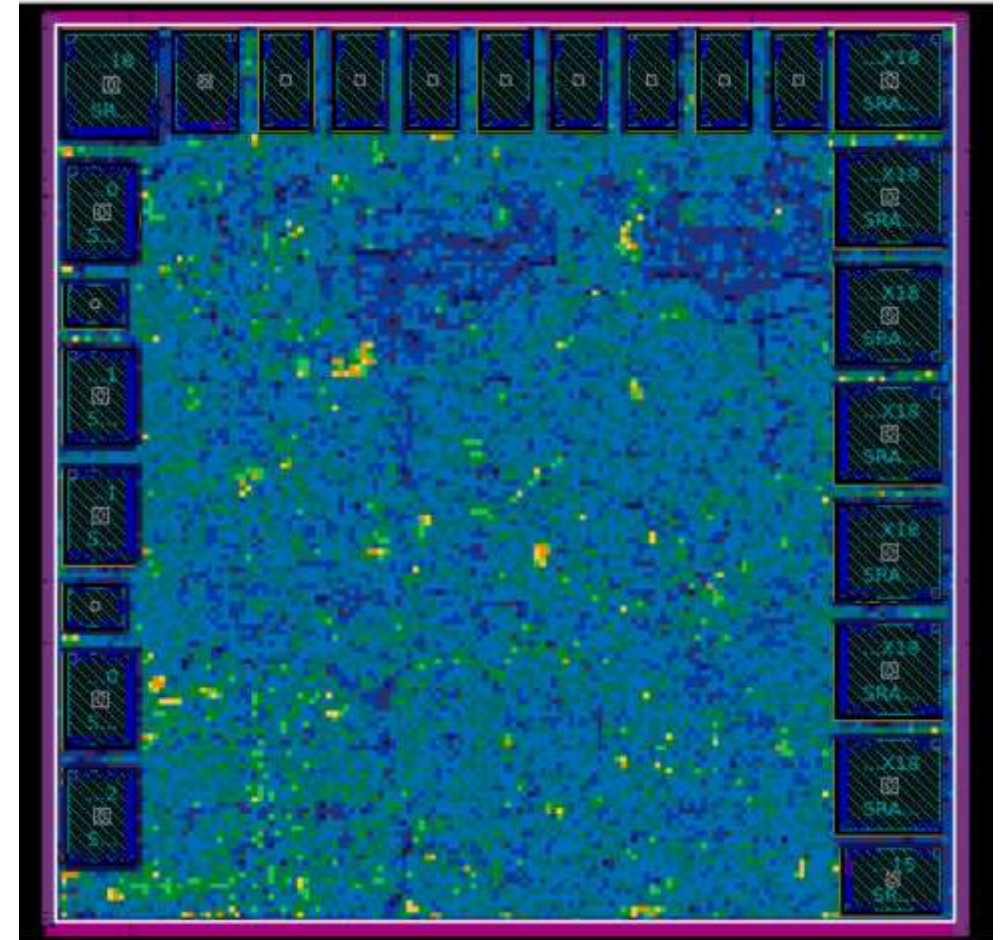


- After many iterations of the power planning and placement stages, we continuously refined the floorplan to resolve various implementation issues.
- Reducing the core area (U from 0.35 to 0.5)
- Added M1 routing blockages around the macros to prevent shorts and minimum-spacing DRC violations between the M1 power rails and the macro pins.

- Reducing the core area (Utilization from 0.35 to 0.5)

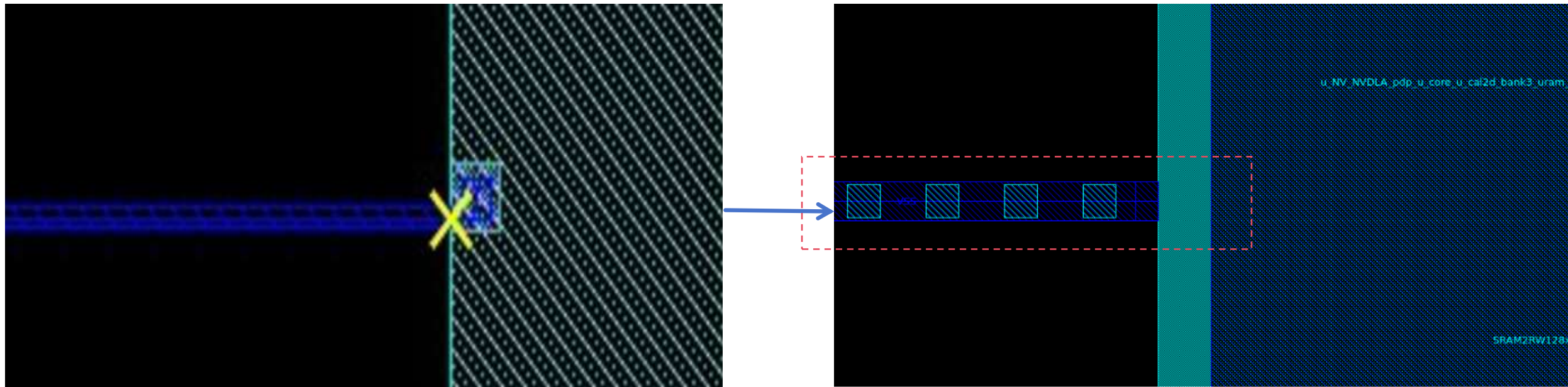


- Utilization = 0.35

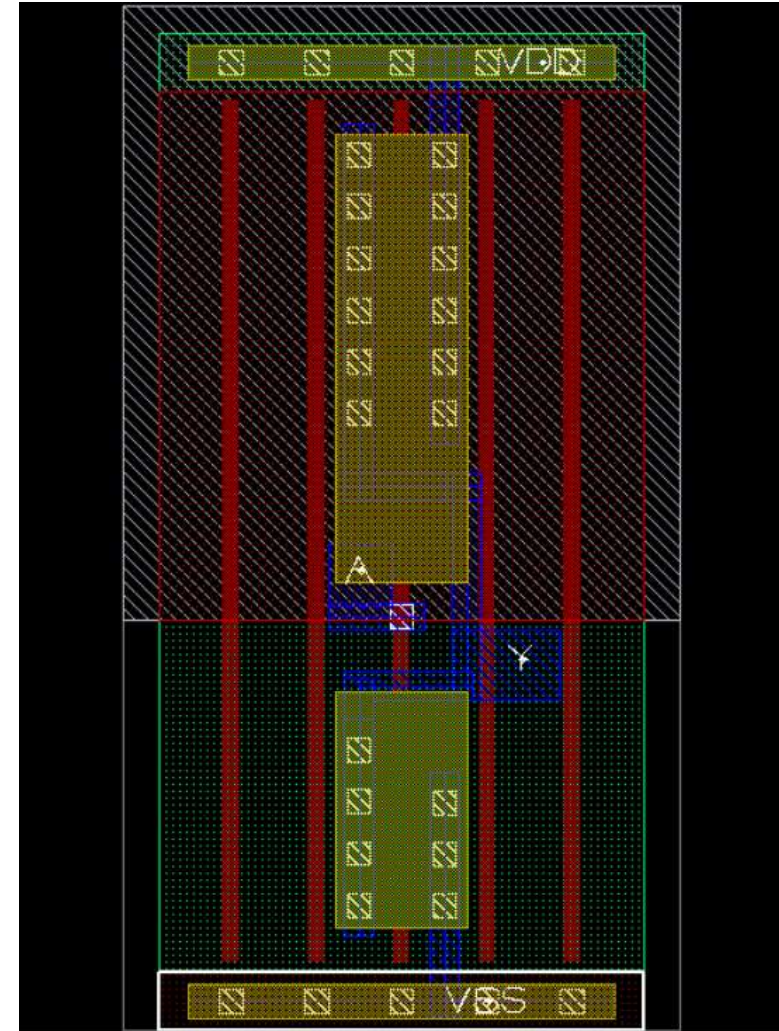


- Utilization = 0.5

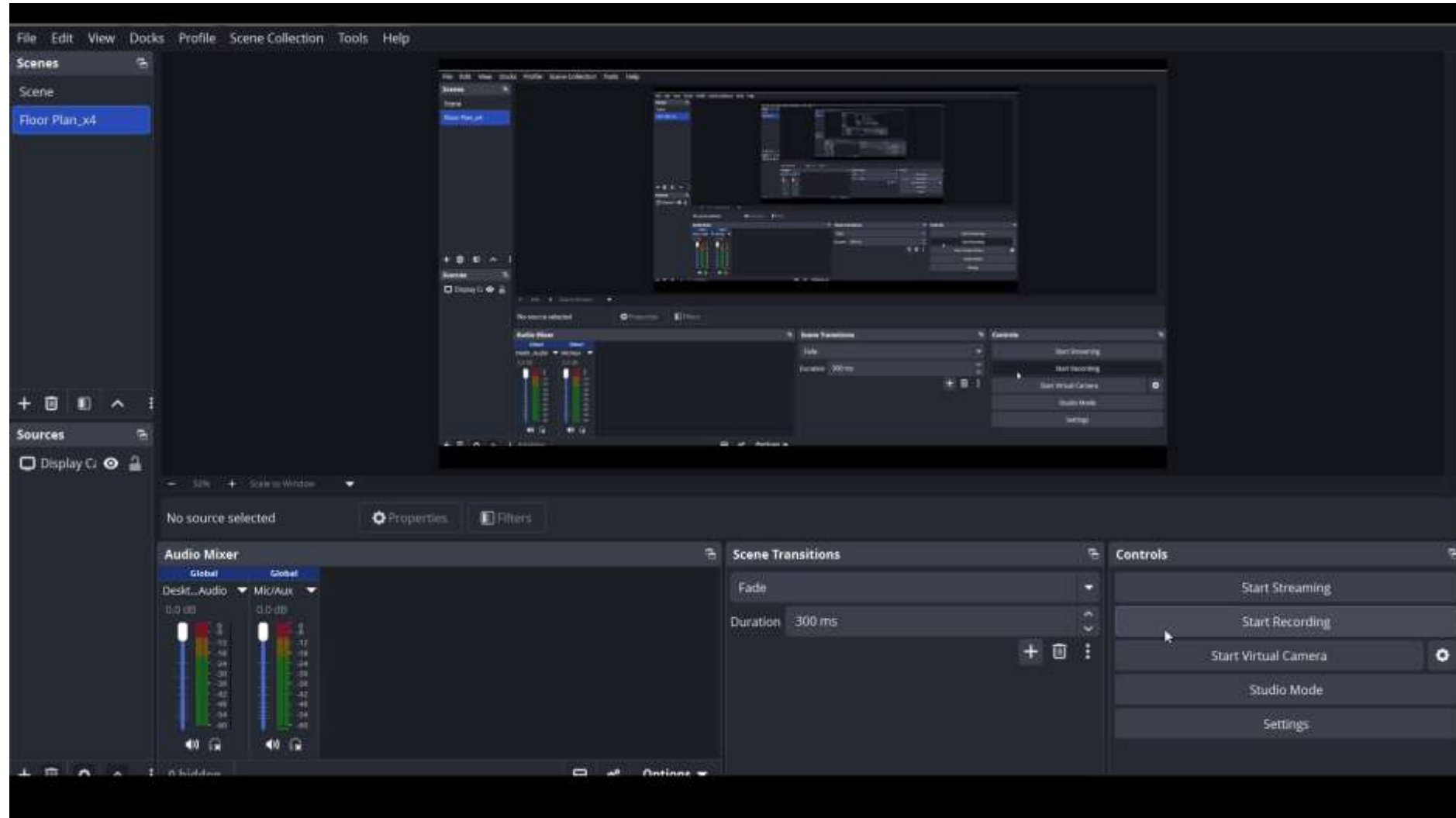
- Added **M1 routing blockages** around the macros to prevent shorts and minimum-spacing DRC violations between the M1 power rails and the macro pins.

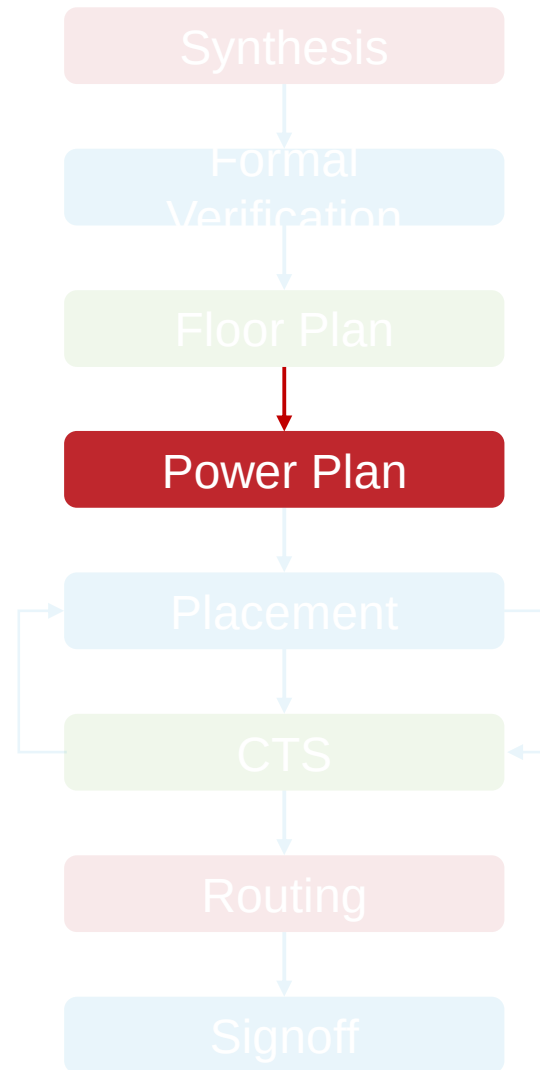


- No well tap cells were added because the PDK contains traditional standard cells, not tapless cells.
- Endcap cells were inserted during the pre-placement stage, both around each macro and at the ends of all sites rows.



Summary of Each Floorplanning Step

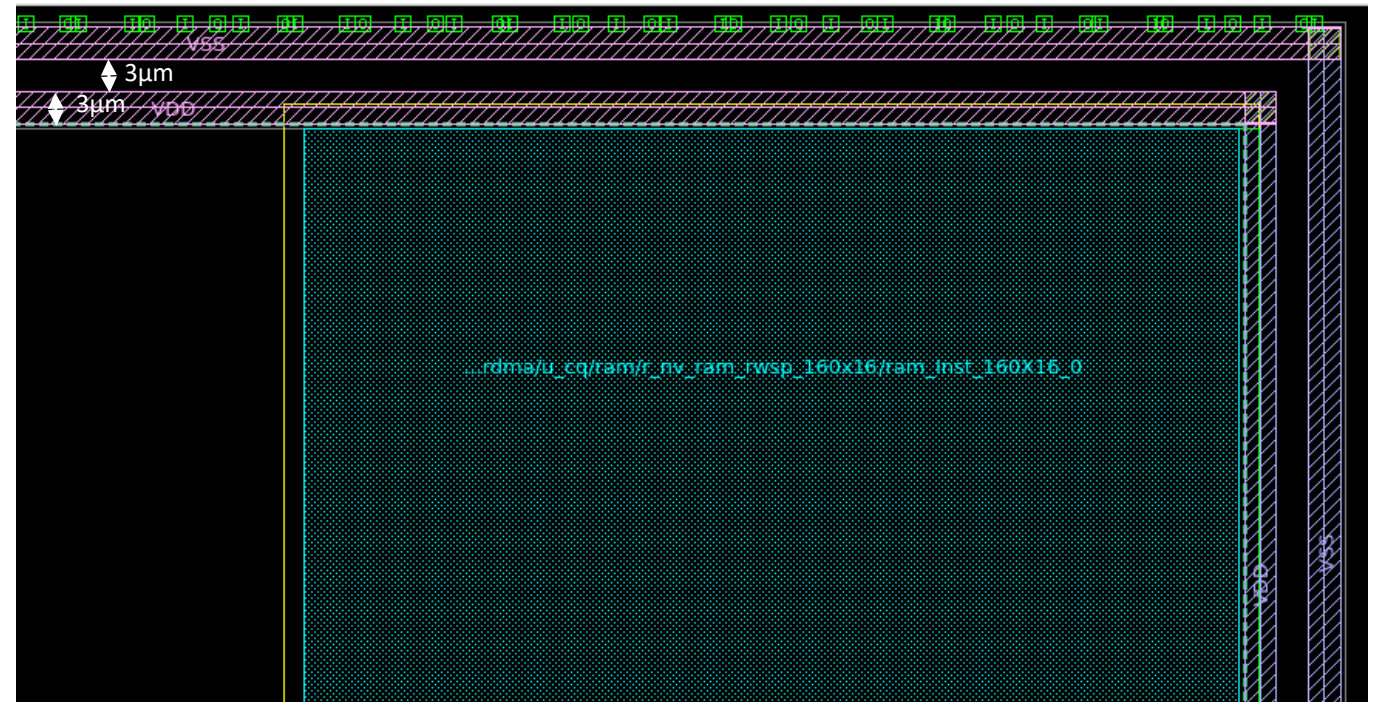




1. Core Power Ring
2. Power Mesh
3. Standard Cell M1 Power Rails
4. DRCs
5. Macro PG Pins Connectivity Strategies

- Constructed VDD and VSS rings around the core boundary using **top thick metal layers (M8 and M9)** to evenly distribute current from rings into the internal floorplan.

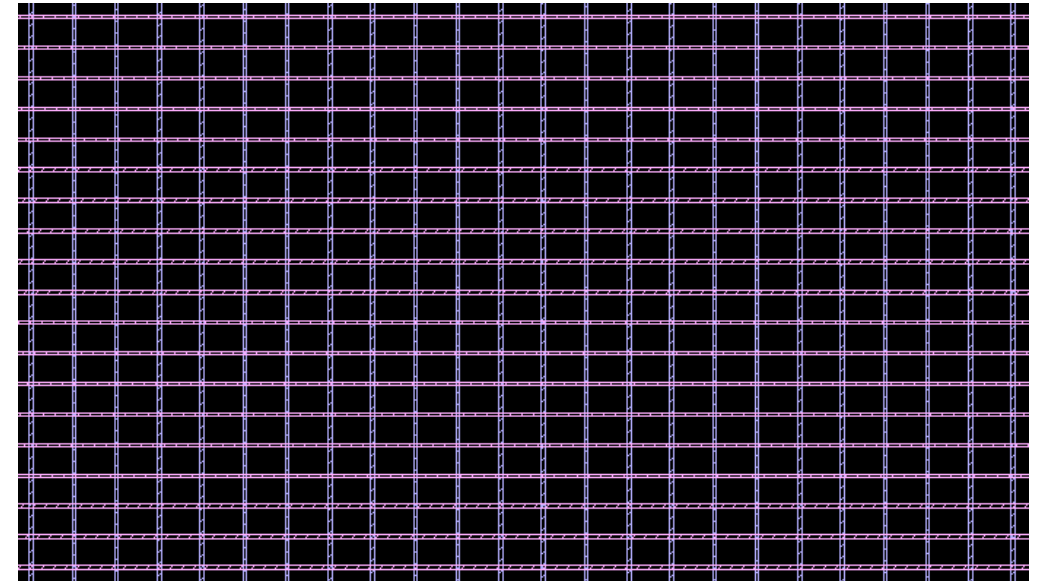
Variable	Value
Ring Offset	0.5 μ m
Ring Width	3 μ m
Ring Spacing	3 μ m
Horizontal Metal Layer	M9
Vertical Metal Layer	M8



- Implementing the internal Power/Ground (PG) mesh inside the core area
- It creates interleaved VDD/VSS straps on **top metal layers (M8 & M9)**, extending them to the design boundary to generate pins automatically.

Vertical Layer	M8
Strap Offset	7μm
Strap Width	3μm
Strap Spacing	Interleaving
Strap Pitch	70μm
Trim	True

Horizontal Layer	M9
Strap Offset	7μm
Strap Width	3μm
Strap Spacing	Interleaving
Strap Pitch	50μm
Trim	True

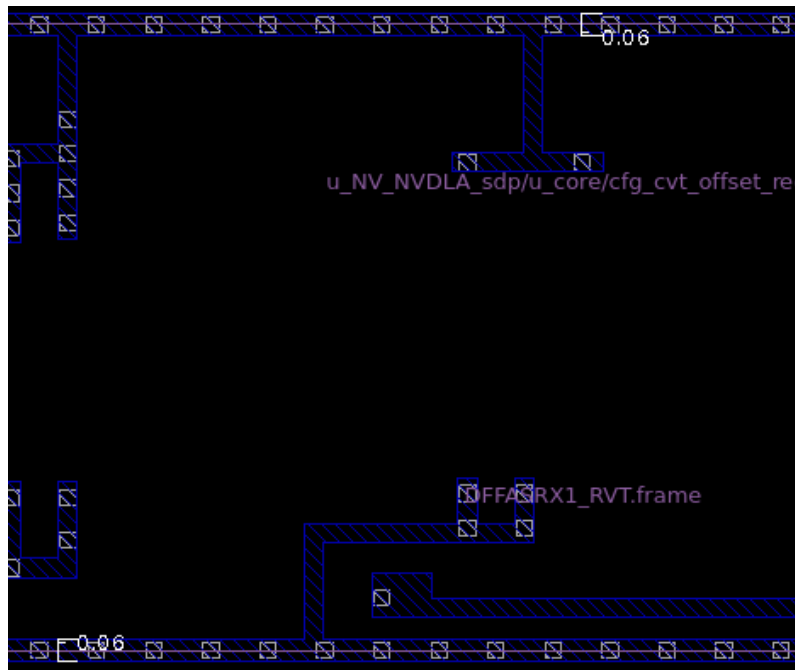


- The **Tech File** specifies an M1 **minWidth** of **0.05 μm**

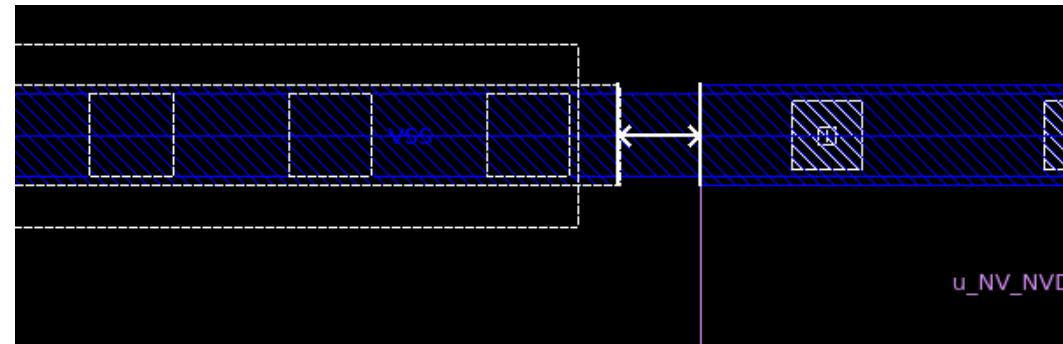
```
Layer      "M1" {  
            /* layout attributes */  
            layerNumber      = 11  
            isDefaultLayer   = 1  
            maskName         = "metall1"  
            pitch            = 0.152  
            /* display attributes */  
            visible          = 1  
            selectable       = 1  
            blink            = 0  
            color            = "blue"  
            lineStyle        = "solid"  
            pattern          = "impdot"  
            /* design rule attributes */  
  
            minWidth         = 0.05  
            maxWidth         = 5  
            defaultWidth     = 0.05  
            minSpacing       = 0.05
```

[1st DRC] Insufficient Spacing On M1

- Root Cause:** When the 0.05 μm PG rail meets the 0.06 μm cell rail at the boundary, it produces a horizontal step/notch. The DRC engine treats these misaligned edges as separate metal geometries on the same net

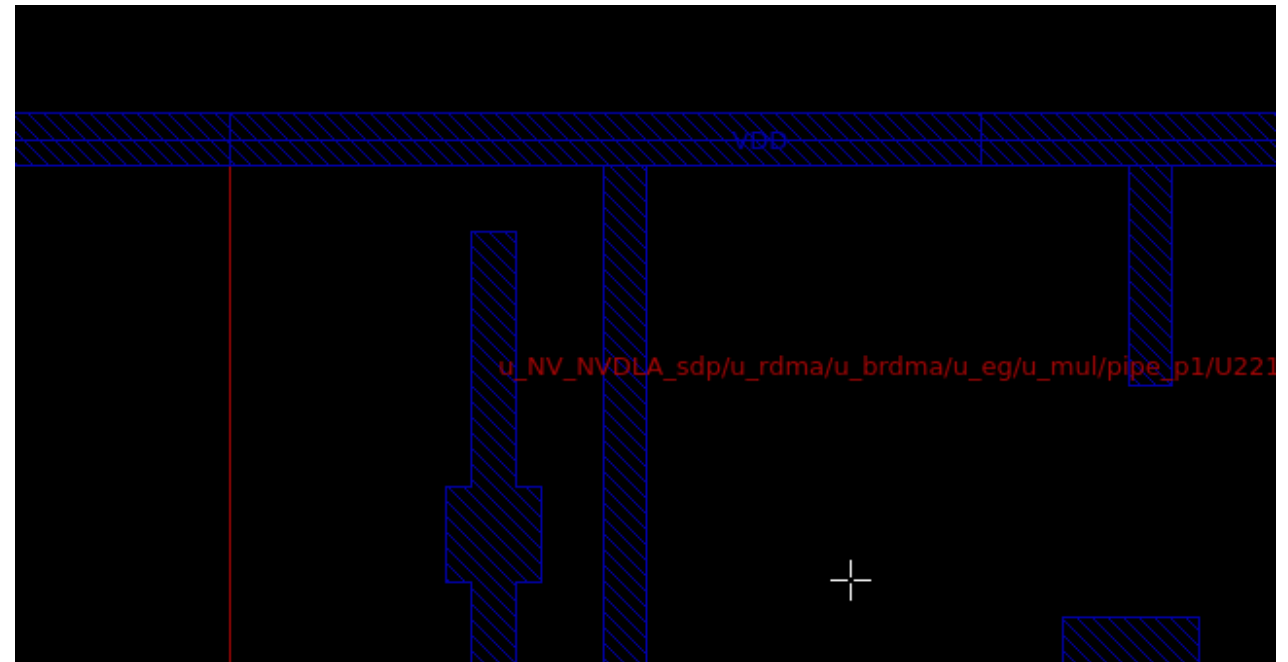


```
icc2_shell> check_pg_drc
Command check_pg_drc started at Thu Aug 6 19:21:35 2026
Command check_pg_drc finished at Thu Aug 6 19:21:53 2026
CPU usage for check_pg_drc: 133.07 seconds ( 0.04 hours)
Elapsed time for check_pg_drc: 17.74 seconds ( 0.00 hours)
Total number of errors found: 117
117 insufficient spacings on M1
```



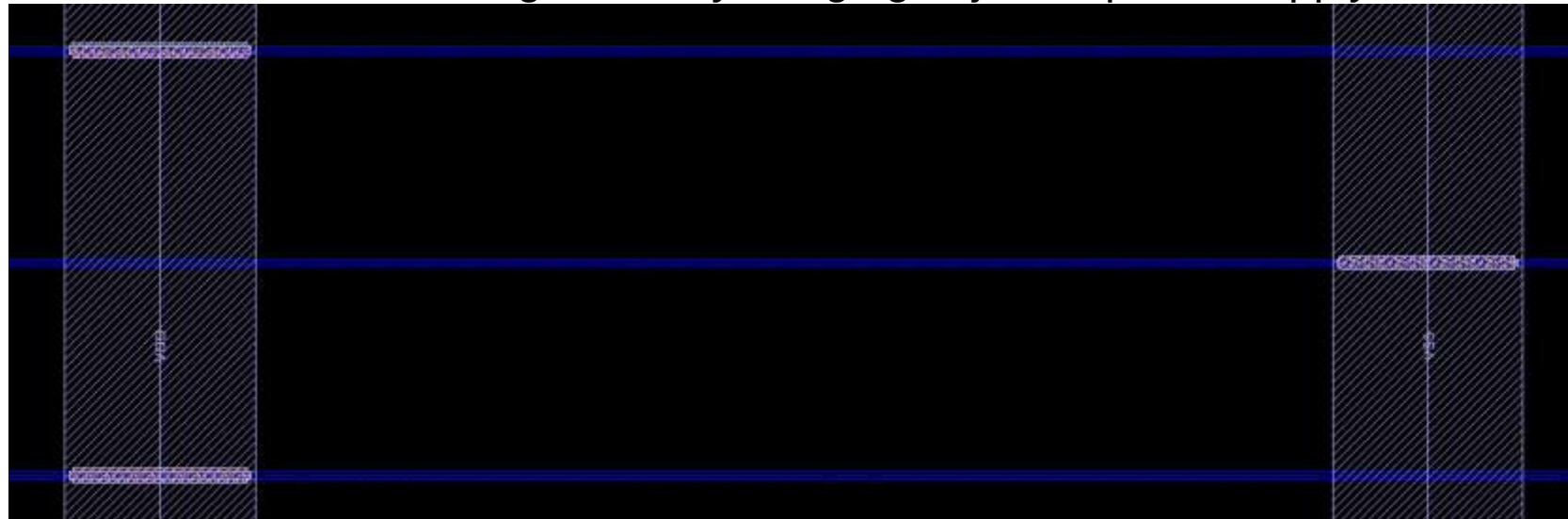
- **Solution:** Change `rail_width` to **0.06 μm** to match the standard cell layout

```
icc2_shell> check_pg_drc
Command check_pg_drc started at Sun Aug 9 03:20:59 2026
Command check_pg_drc finished at Sun Aug 9 03:22:24 2026
CPU usage for check_pg_drc: 85.73 seconds ( 0.02 hours)
Elapsed time for check_pg_drc: 85.74 seconds ( 0.02 hours)
No errors found.
```



- Constructed VDD and VSS rails on layer **M1** and the via transition **from layer M8 to layer**
- Flipped alternating standard cell rows to enable direct VDD and VSS M1 rail sharing through abutment
- Maximized core cell density and saved horizontal routing tracks by merging adjacent power supply rails

Variable	Value
Rails Metal Layer	M1
Rails Width	0.06μm

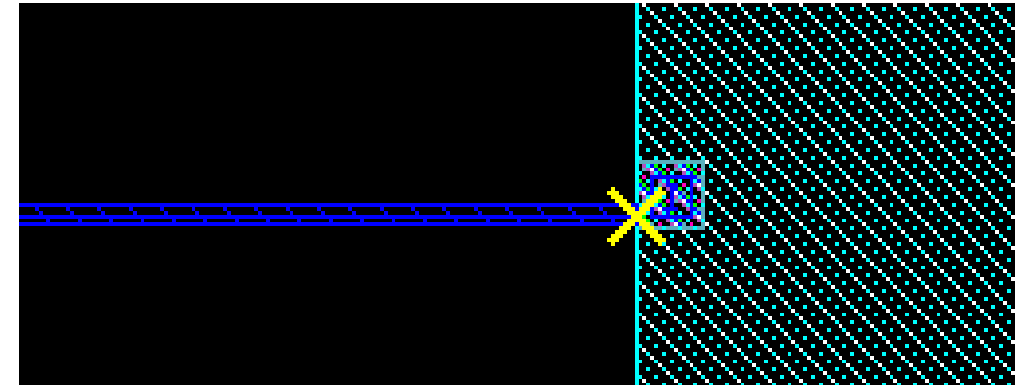


[2nd DRC] Short On M1

- `compile_pg -strategies $rail_strategie -ignore_drc`

```
Committed 1541 wires.  
Committing wires takes 0.00 seconds.  
Committed 39697 vias.
```

```
icc2_shell> check_pg_drc  
Command check_pg_drc started at S  
Command check_pg_drc finished at S  
CPU usage for check_pg_drc: 6.97 s  
Elapsed time for check_pg_drc: 6.9  
Total number of errors found: 504  
58 shorts on M1
```

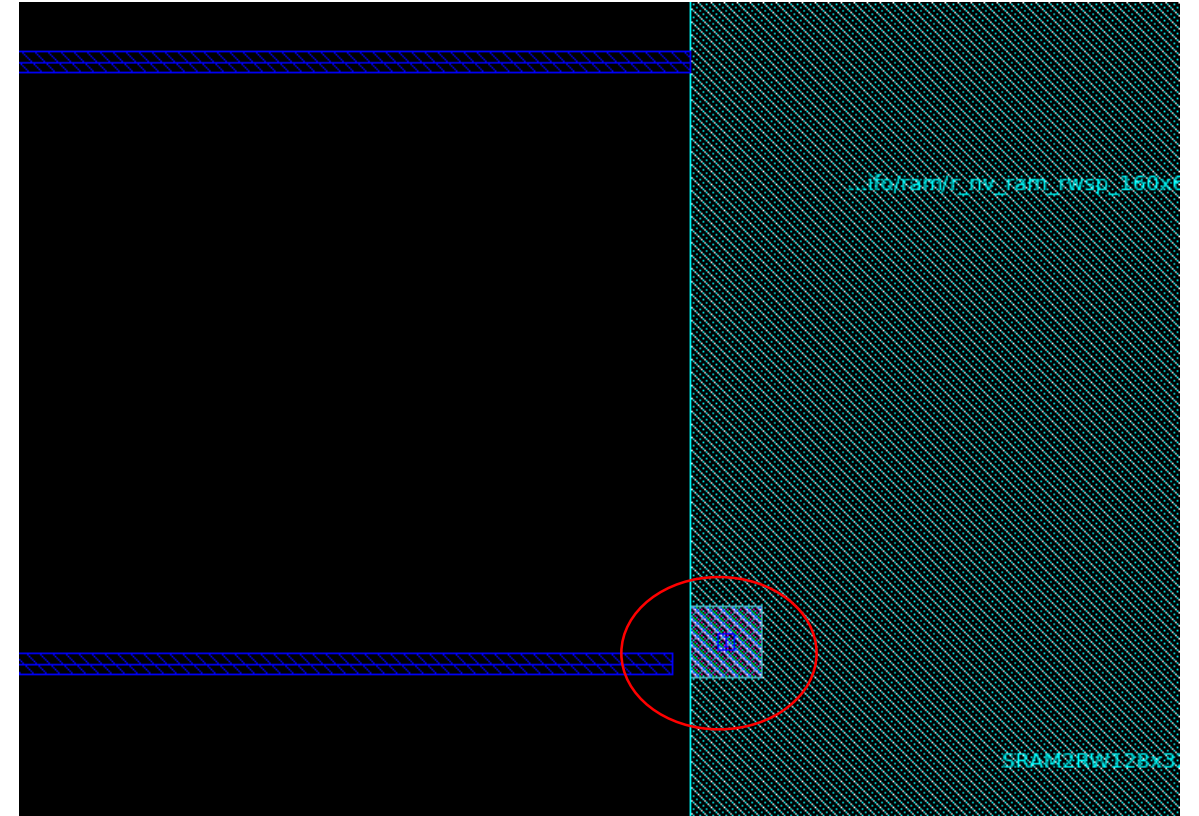


First Solution

- `compile_pg -strategies $rail_startegie`

```
Committed 1542 wires.  
Committing wires takes 0.00 seconds.  
Committed 0 vias.
```

- `create_pg_vias -from_layers M8 -to_layers M1 \
-nets {VDD VSS} -drc no_check`

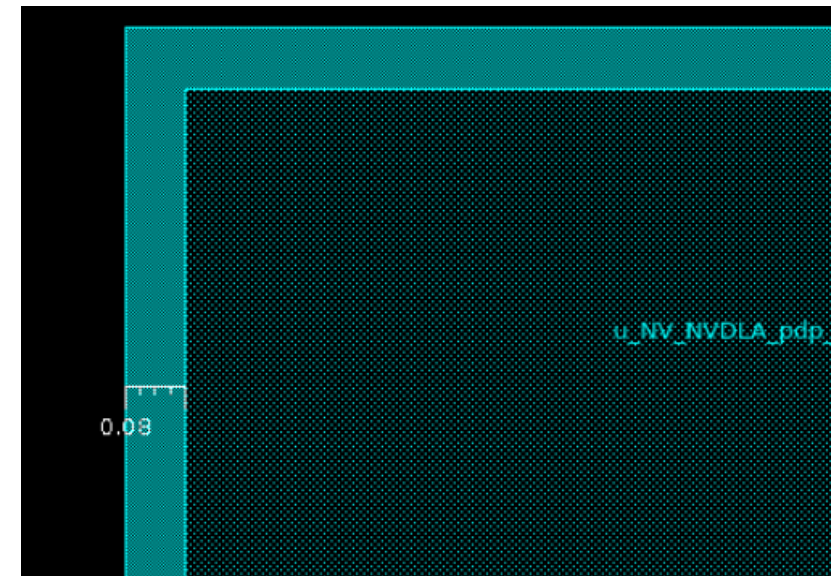
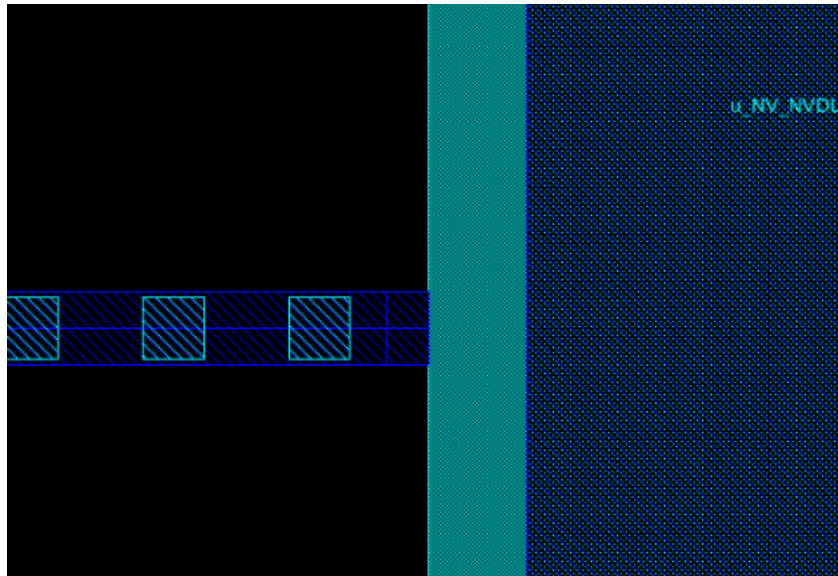


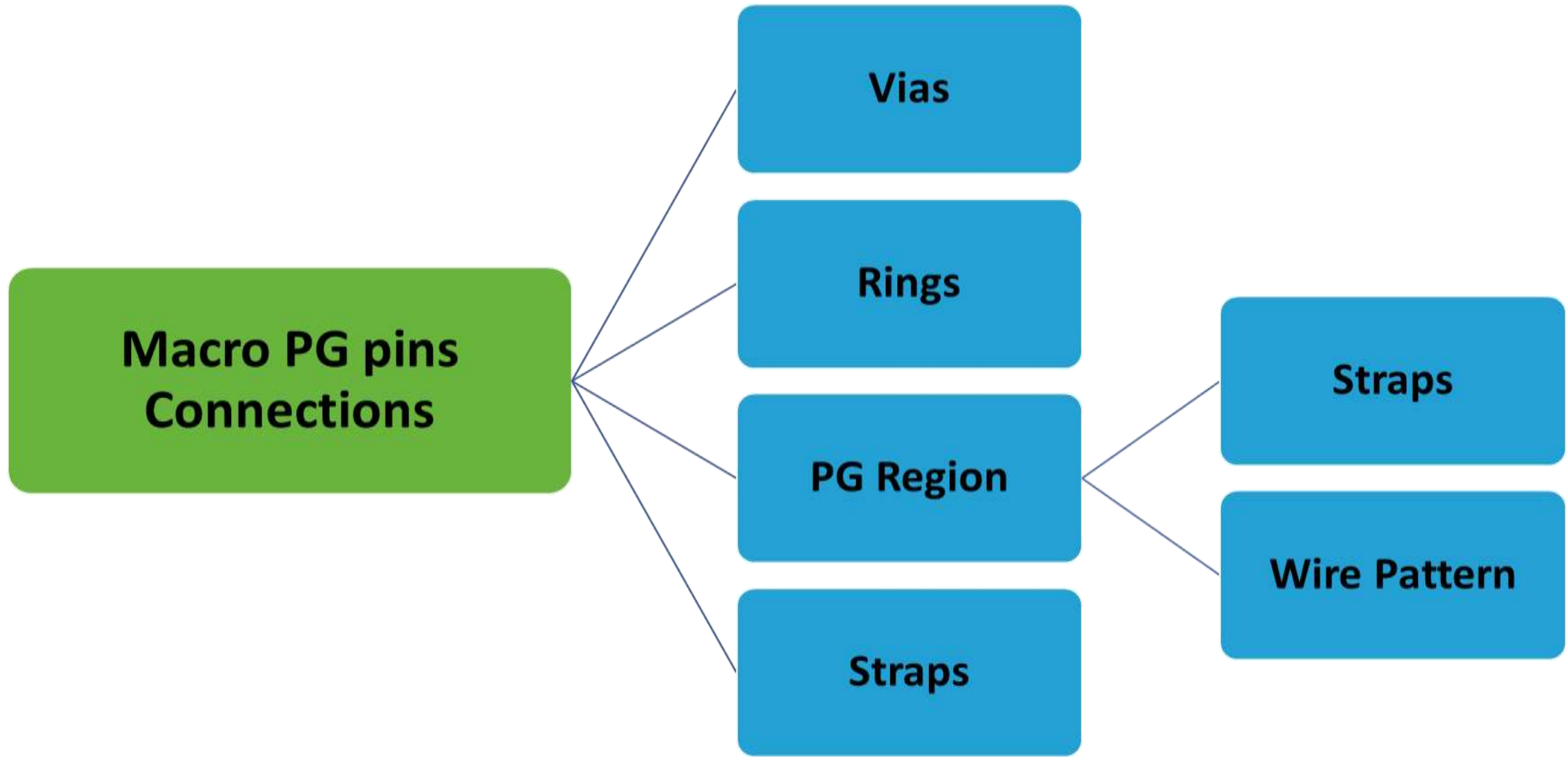
Second Solution

- Adding **M1 routing blockages** around the macros to prevent shorts and minimum-spacing DRC violations between the **M1 power rails** and the **macro pins**

```
set my_macros [get_cells -hierarchical -filter "is_hard_macro==true || design_type==macro"]
```

```
create_keepout_margin -type routing_blockage -outer {0.08 0.08 0.08 0.08} -layers {M1} $my_macros
```

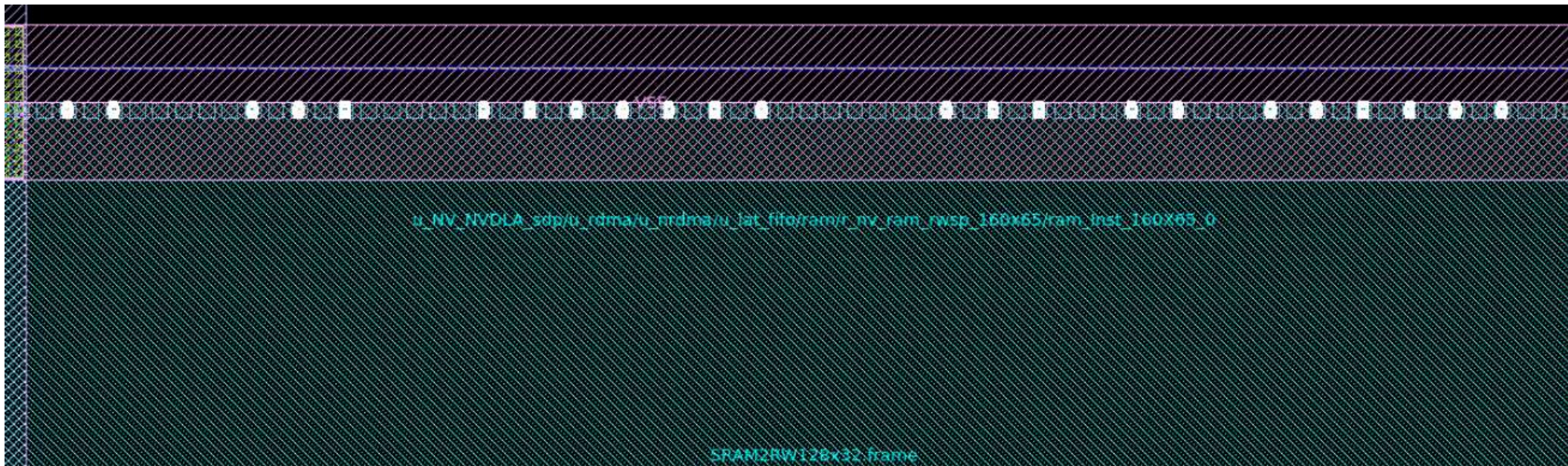




Strategies	Connection Rate For Macro Pins	Routing Resources
Vias	7%	Reduced
PG Region Straps	15%	19.8%
PG Region Wire Pattern	15%	16.6%
Rings	75%	3.2%
Straps	95%	1%

Connecting Macro PG Pins via Vias

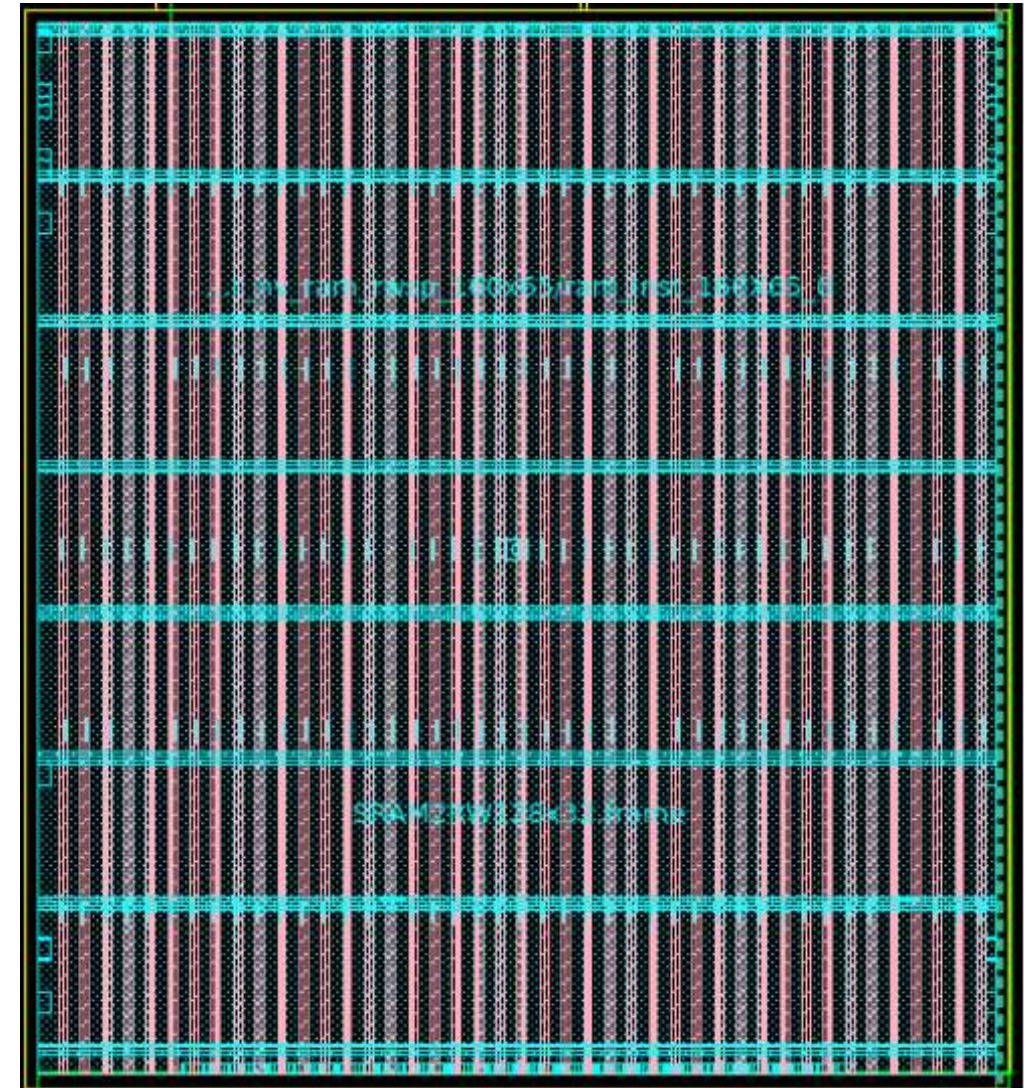
- Only **7%** of the macro PG pins are connected to the power mesh
- **Saves routing resources** and mitigates congestion on upper layers



Connecting Macro PG Pins via PG Region Straps

- Dropping M6 & M7 straps onto the macro pins achieves **15%** connection rate for macro PG
- Power grid allocation consumes **~19.9** routing area, consuming high routing resources.

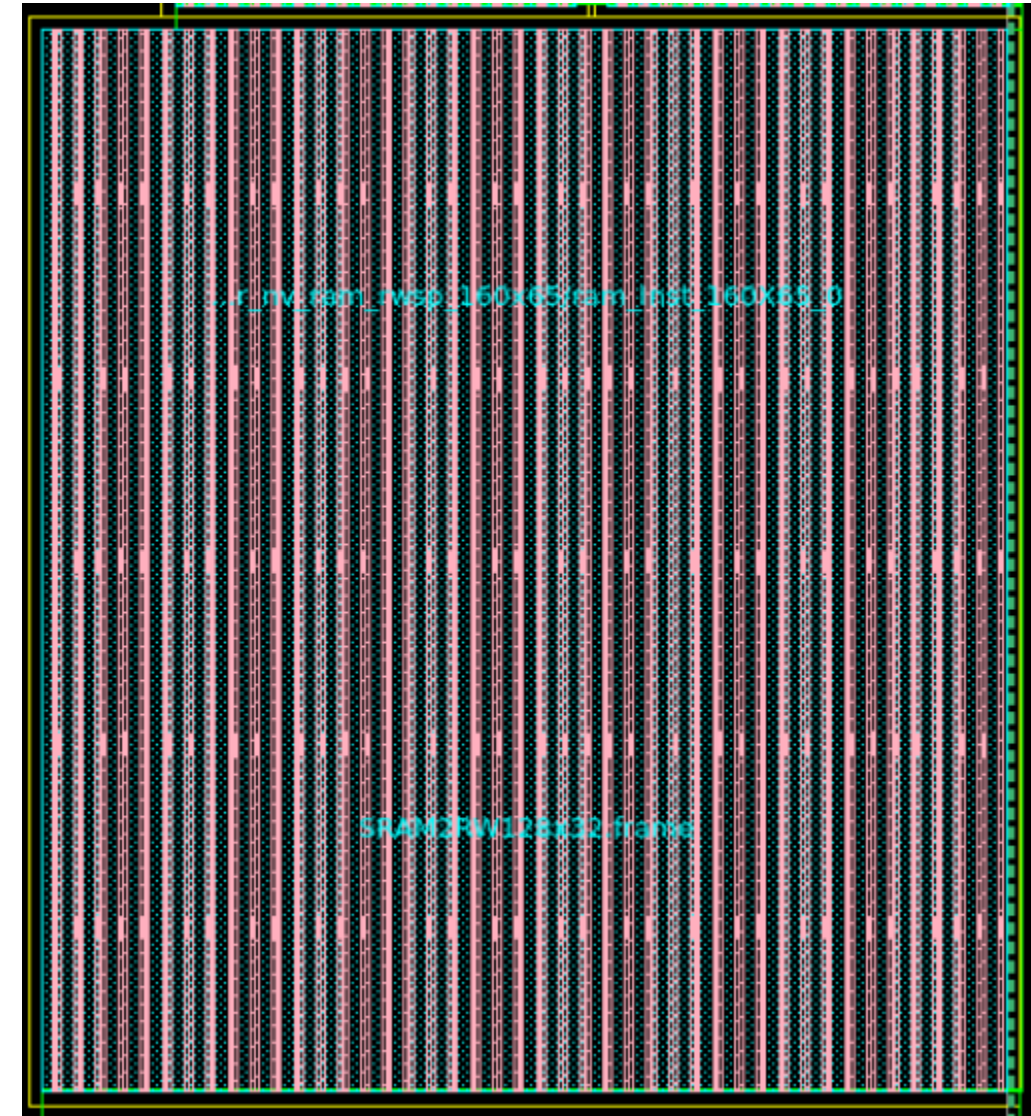
layer	Shape	Spacing	Total
P0	0.000%	0.000%	0.000%
M1	0.000%	0.000%	0.000%
M2	0.000%	0.000%	0.000%
M3	0.000%	0.000%	0.000%
M4	0.000%	0.000%	0.000%
M5	0.000%	0.000%	0.000%
M6	10.294%	6.331%	16.625%
M7	1.965%	1.315%	3.279%



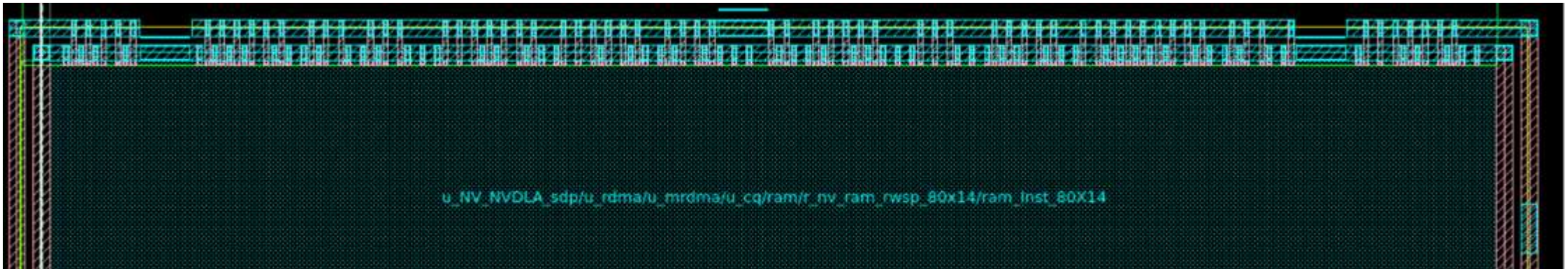
Connecting Macro PG Pins via PG Region Wire Pattern

- Dropping M6 & M7 straps onto the macro pins achieves **15%** connection rate for macro PG
- Power grid allocation consumes **~16.6** routing area, consuming high routing resources.

layer	Shape	Spacing	Total
P0	0.000%	0.000%	0.000%
M1	0.000%	0.000%	0.000%
M2	0.000%	0.000%	0.000%
M3	0.000%	0.000%	0.000%
M4	0.000%	0.000%	0.000%
M5	0.000%	0.000%	0.000%
M6	10.330%	6.269%	16.598%



- Power Rings on M6 & M7 onto the macro pins achieves **75%** connection rate for macro power delivery
- Power grid allocation consumes only **~3.2%** of total M6 & M7 routing area, keeping optimal routing resources available for signal nets

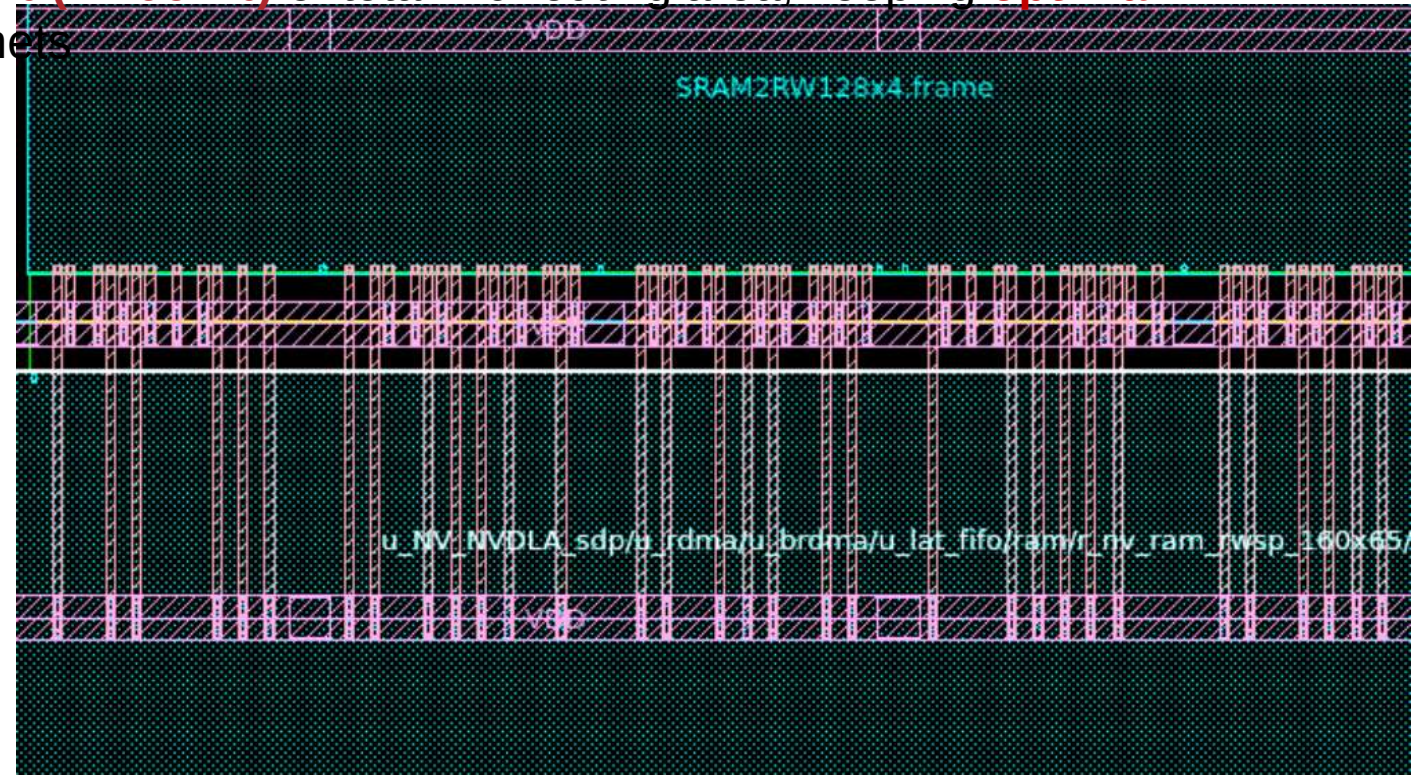


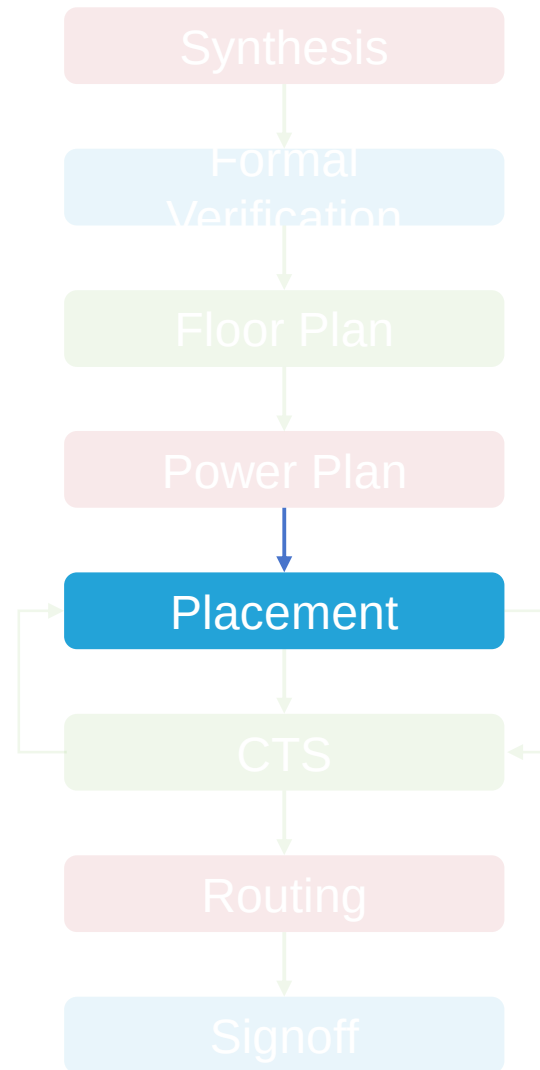
M6		1.242%		0.716%		1.958%
M7		1.008%		0.317%		1.325%

Connecting Macro PG Pins via straps

- Dropping straps on M6 onto the macro pins achieves **95%** connection rate for macro power delivery
- Power grid allocation consumes only **~1% (~1.061%)** of total M6 routing area, keeping **optimal routing resources** available for signal nets

layer	Shape	Spacing	Total
P0	0.000%	0.000%	0.000%
M1	6.758%	0.000%	6.758%
M2	0.181%	0.505%	0.686%
M3	0.456%	0.000%	0.456%
M4	0.182%	0.508%	0.690%
M5	0.465%	0.010%	0.476%
M6	0.401%	0.660%	1.061%





1. Pre-placement Checks
2. Scenario Filtering
3. Boundary Cells (Decaps)
4. Application Options
5. High-Fanout Synthesis (HFS)
6. Tie Cells and Spare Cells

Placement : is the process of placing the standard cells inside the core in an **optimal location**

- Ensure the PG network connectivity is good (**check_pg_connectivity**)
- Catch PG shorts/spacing issues early before placement (**check_pg_drc**)
- Ensure all required stack-ups are present for PG (**check_pg_missing_vias**)
- Legality check before placement for Macros (**check_legality**)
- check for issues that could block placement (**check_design**)

*****Verify net VDD connectivity***

Number of floating wires: 0
Number of floating vias: 0
Number of floating std cells: 0
Number of floating hard macros: 0
Number of floating I/O pads: 0
Number of floating terminals: 0
Number of floating hierarchical blocks: 0

*****Verify net VSS connectivity***

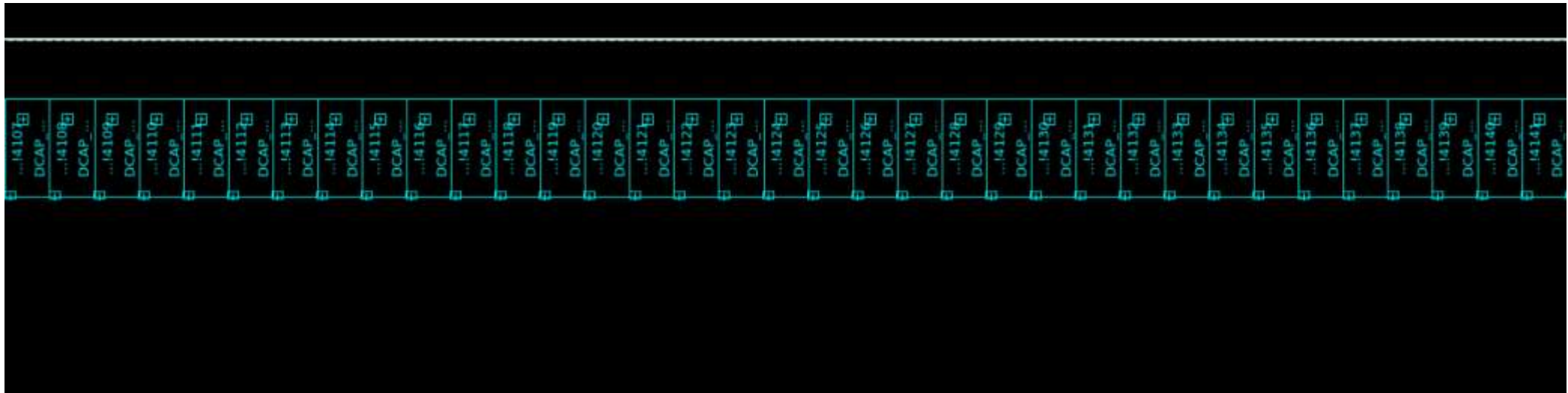
Number of floating wires: 0
Number of floating vias: 0
Number of floating std cells: 0
Number of floating hard macros: 0
Number of floating I/O pads: 0
Number of floating terminals: 0
Number of floating hierarchical blocks: 0

```
icc2_shell> check_pg_drc -ignore_std_cells
Command check_pg_drc started at Sat Aug 8 17:07:17 2026
Command check_pg_drc finished at Sat Aug 8 17:07:31 2026
CPU usage for check_pg_drc: 14.08 seconds ( 0.00 hours)
Elapsed time for check_pg_drc: 14.08 seconds ( 0.00 hours)
No errors found.
```

```
icc2_shell> check_pg_missing_vias
Check net VDD vias...
Number of missing vias: 0
Checking net VDD vias took 1 seconds.
Check net VSS vias...
Number of missing vias: 0
Checking net VSS vias took 0 seconds.
Overall runtime: 1 seconds.
```

Scenario	Active
func_slow_cmax (Late analysis)	true
func_fast_cmin (Early analysis)	false

- Using **Decaps on boundaries** compensates for **missing endcaps** while boosting power reliability
- **Boundary Protection:** shields gates from distortions
- **Dynamic IR Drop Reduction:** Functions as local charge reservoirs to supply instant current during simultaneous switching events



App Option	Value	Purpose
place.legalize.enable_advanced_legalizer	True	Enables advanced legalizer engine to prevent cell overlaps
place.coarse.legalizer_driven_placement	True	legalization challenges during coarse placement to prevent large cell shifts later and keep timing predictable.
place.coarse.auto_timing_control	True	Dynamically balances timing improvement against congestion mitigation
opt.tie_cell.max_fanout, opt.common.max_fanout	5	Limits the max fanout load connected to any Tie cell Restricts the maximum fanout on general logic nets to 20 to control parasitic capacitance and delay.
opt.timing.effort	high	focus on reducing setup timing violations
place_opt.congestion.effort	high	Enables congestion-reduction algorithms during place_opt to resolve routing bottlenecks.

Found 5904 buffer-tree drivers

Roi-HfsDrc SN: 3403505 (1174.419067)

Processing Buffer Trees (ROI) ...

[591] 10% ...

[1182] 20% ...

[1773] 30% ...

[2364] 40% ...

[2955] 50% ...

[3546] 60% ...

[4137] 70% ...

[4728] 80% ...

[5319] 90% ...

[5904] 100% Done

	Deleted	Added
-----	-----	-----
Buffers:	318	2646
Inverters:	1402	1498
-----	-----	-----
Total:	1720	4144
-----	-----	-----

Number of Drivers Sized: 1588 [26.90%]

Before HFNS

NetName	Fanout Driver
-----	-----
csb2sdp_rdma_req_pvld	33 csb2sdp_rdma_req_pvld
csb2sdp_req_pvld	33 csb2sdp_req_pvld
nvdla_core_clk	4926 nvdla_core_clk
nvdla_core_rstn	4816 u_partition_p_reset/sync_reset_synced_rstn/UI_test_mode_outmux/U1/Y
u_NV_NVDLA_sdp/reg2dp_interrupt_ptr	64 u_NV_NVDLA_sdp/u_reg/U53/Y
u_NV_NVDLA_sdp/reg2dp_output_dst	20 u_NV_NVDLA_sdp/u_reg/U95/Y
u_NV_NVDLA_sdp/u_rdma/u_mrdma/nvdla_gated_clk	1833 u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_gate/nvdla_core_clk_slcg_0/p_clkgate/U3/Y

After HFNS

NetName	Fanout Driver
-----	-----
nvdla_core_clk	4926 nvdla_core_clk
u_NV_NVDLA_sdp/u_rdma/u_mrdma/nvdla_gated_clk	1833 u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_gate/nvdla_core_clk_slcg_0/p_clkgate/U3/Y
u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_cq/nvdla_core_clk_mgated	33 u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_cq/nvdla_core_clk_mgate/p_clkgate/U3/Y
u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_eg/u_cmd/u_sfifo/nvdla_core_clk_mgated	63 u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_eg/u_cmd/u_sfifo/nvdla_core_clk_mgate/p_clkgate/U3/Y
u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_eg/u_cmd/u_dfifo/nvdla_core_clk_mgated	67 u_NV_NVDLA_sdp/u_rdma/u_mrdma/u_eg/u_cmd/u_dfifo/nvdla_core_clk_mgate/p_clkgate/U3/Y

Before HFNS

NET FANOUT AND PIN COUNT INFORMATION			
Fanout	Netcount	netPinCount	NetCount
<2	88026	<2	8746
2	30141	2	79298
3	14919	3	30123
4	4131	4	14924
5	1900	5	4127
6-10	3385	6-10	4890
11-20	4812	11-20	5085
21-30	838	21-30	884
31-50	1168	31-50	1243
51-100	54	51-100	54
101-500	39	101-500	39
501-1000	2	501-1000	2
>1000	6	>1000	6

After HFNS

NET FANOUT AND PIN COUNT INFORMATION			
Fanout	Netcount	netPinCount	NetCount
<2	109208	<2	631
2	37985	2	108577
3	26095	3	37985
4	23691	4	26095
5	21895	5	23691
6-10	7	6-10	21902
11-20	3	11-20	3
21-30	7	21-30	5
31-50	1	31-50	3
51-100	39	51-100	39
101-500	38	101-500	38
501-1000	2	501-1000	2
>1000	5	>1000	5

```
saed32rvt_c/TIEL_RVT saed32rvt_c/TIEL_RVT boolean dont_use true
```

- Unlocks Tie-High (TIEH) and Tie-Low (TIEL) cells (dont_use=false) so the tool can place them
- Runs add_tie_cells to connect static gate inputs to dedicated tie cells instead of direct Vdd/Vss rails, protecting gates noise spikes
- Runs legalize_placement to align newly added tie cells onto the placement grid.

Before Tie Cell Insertion

Tie Low	0
Tie High	0
Others	0

After Tie Cell Insertion

Tie Low	461
Tie High	24
Others	0

- Instantiated 50 base gates (NAND2, AND2, INV) distributed randomly across the core while tying inputs to Tie Low to prevent floating gate noise and leakage power
- Legalized their locations without disturbing active functional logic
- Locked down physical locations and attributes of spare cells to ensure subsequent placement and routing passes preserve them for post-silicon ECOs

Before Spare Cell Insertion

Spare cells	0
ICG cells	0
Flip-flop cells	14218
Latch cells	0
Antenna cells	0

After Spare Cell Insertion

Spare cells	50
ICG cells	0
Flip-flop cells	14218
Latch cells	0
Antenna cells	0

Problems and Solutions

1st Problem: No Valid scan def found

- The placer defaults to scan-chain aware to group ffs in the same scan chain close together
- Setting **continue_on_missing_scandef true** allows place_opt to complete without crashing, but scan chain ffs will be placed based on functional logic connectivity rather than scan ordering

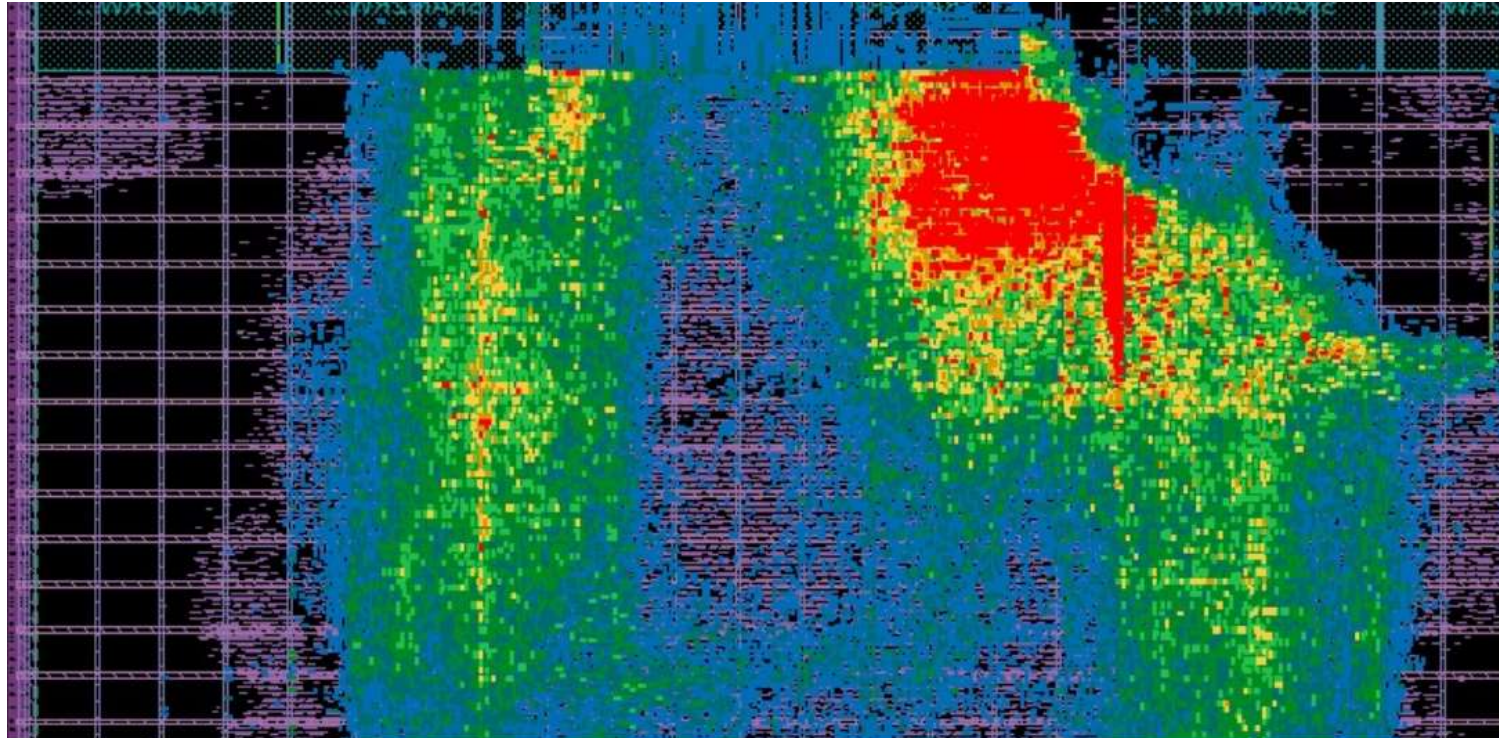
If you need continue without def file

```
set_app_options -list {place.coarse.continue_on_missing_scandef {true}}
```

```
Error: No valid scan def found. (PLACE-042)
Error: Optimization failed at create_placement
Error: place_opt failed during initial placement.
Error: place_opt failed during create_placement
```

2nd Problem: Congestion

- Insufficient cell spreading resulted in **high-density** regions, creating multiple placement **hotspots** in specific areas




```
set_app_options -name place.coarse.congestion_driven_max_util -value 0.4
```



[3rd Problem] Max Capacitance and Max Transition Violations

- We observed **maximum capacitance and max transition violations on the macro pins** due to excessive net lengths and the high input capacitance of the first-stage standard cells
- Consequently, the macro pins could not drive these loads effectively, resulting in max capacitance and max transition violations

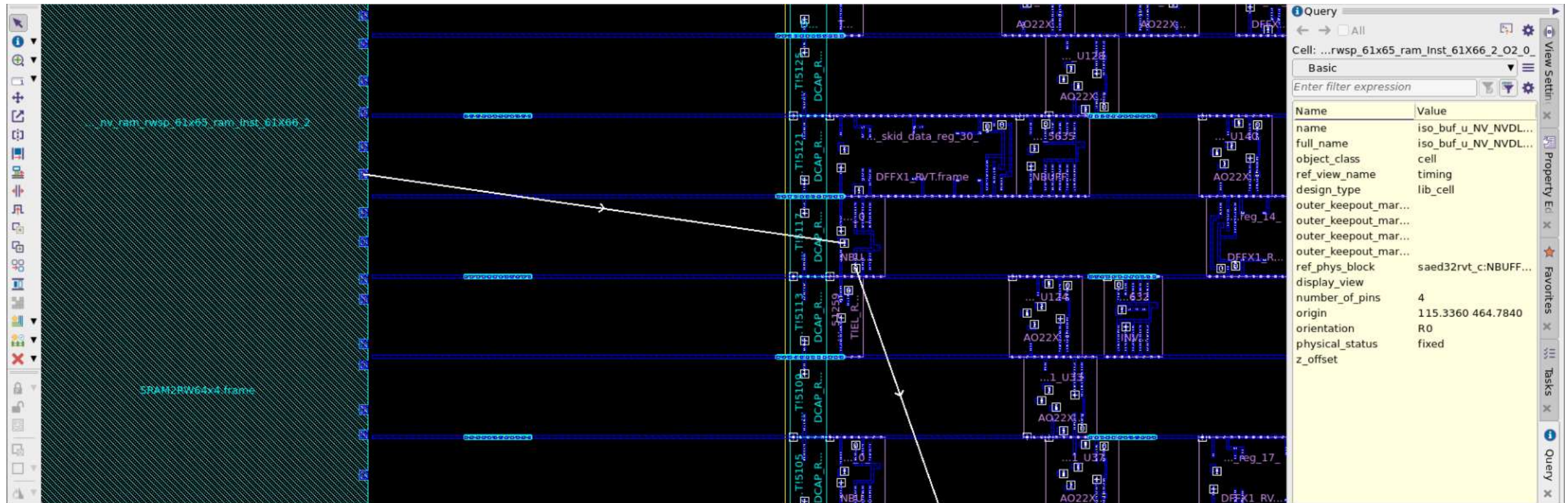
max_capacitance	Required Capacitance	Actual Capacitance	Slack	Violation
Net				
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[13]	25.00	147.16	-122.16	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[13]	25.00	147.16	-122.16	(VIOLATED)
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[11]	25.00	147.00	-122.00	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[11]	25.00	147.00	-122.00	(VIOLATED)
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[6]	25.00	146.28	-121.28	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[6]	25.00	146.28	-121.28	(VIOLATED)
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[5]	25.00	145.51	-120.51	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[5]	25.00	145.51	-120.51	(VIOLATED)
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[7]	25.00	145.34	-120.34	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[7]	25.00	145.34	-120.34	(VIOLATED)
u_NV_NVDLA_pdp_u_core_u_cal2d_mem_rdata_1[8]	25.00	144.37	-119.37	(VIOLATED)
PIN : u_NV_NVDLA_pdp_u_core_u_cal2d_bank1_uram_0_r_nv_ram_rws_128x18_ram_Inst_128X18/O2[8]	25.00	144.37	-119.37	(VIOLATED)

- Constrained maximum wire lengths to reduce interconnect capacitance and optimize path delay using:

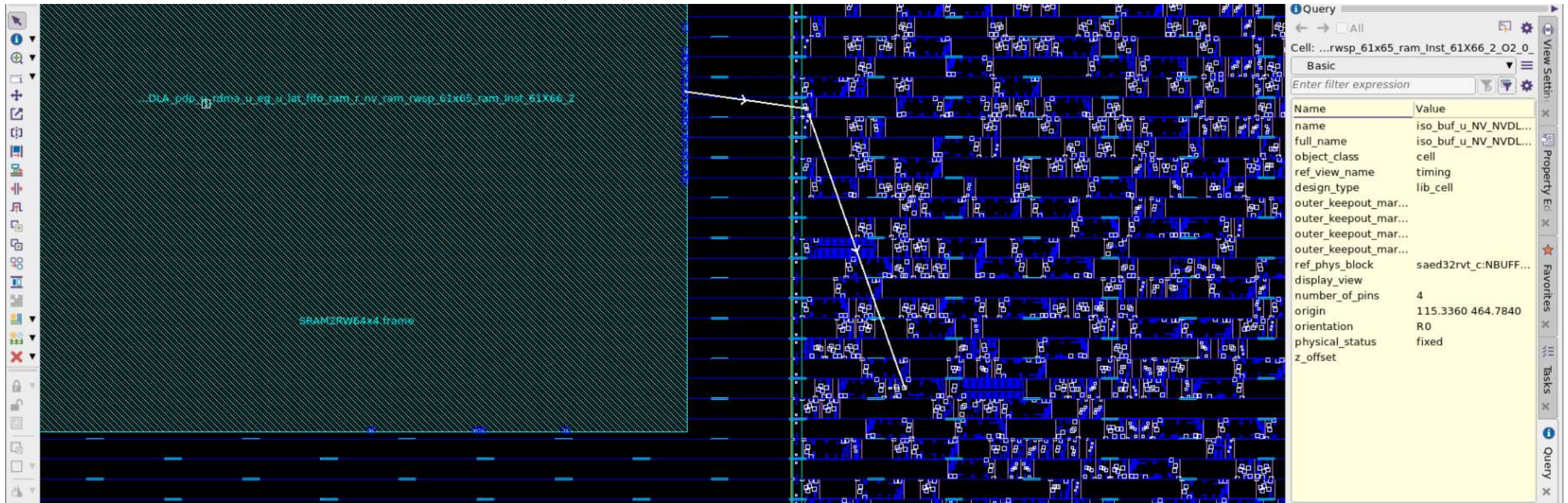
```
set_app_options -list { opt.common.max.net.length  
{150} }
```

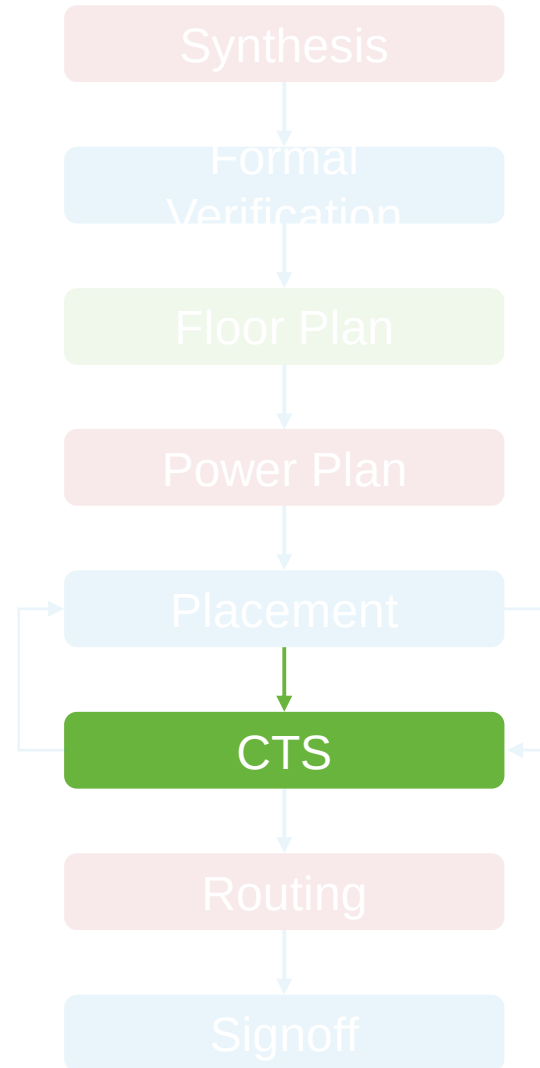

Inserting Isolation Buffers

- Added isolation buffers at macro pins to boost drive strength and isolate heavy interconnect loads



- Grouped interacting standard cells closer to macro boundaries using magnet placement





- 1. PreCTS Checks**
- 2. Preparation Steps For CTS**
- 3. Application Options**
- 4. Results**

1. Design is placed
2. Power and Ground nets are routed
3. Estimated congestion (acceptable)
4. Estimated time for setup (acceptable)
5. High Fanout Synthesis is done
6. Clocks has been defined

1- Activation for all scenarios

- Before CTS

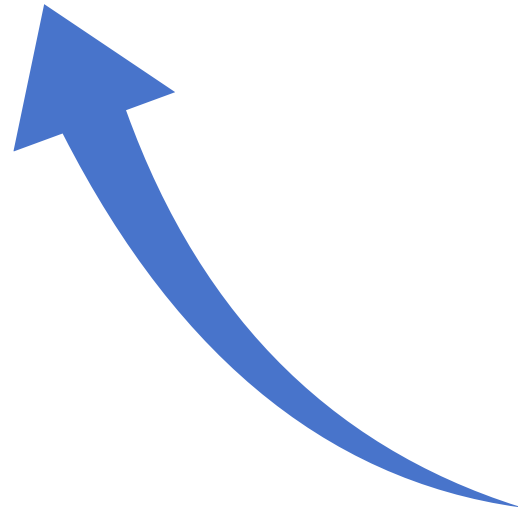
Scenario	Active	Setup	Hold
func_slow_cmax	true	true	false
func_fast_cmin	false	false	true

- During CTS

Scenario	Active	Setup	Hold
func_slow_cmax	true	true	true
func_fast_cmin	true	true	true

2- Specifying metal layers used in CTS

Max_Layer :M7

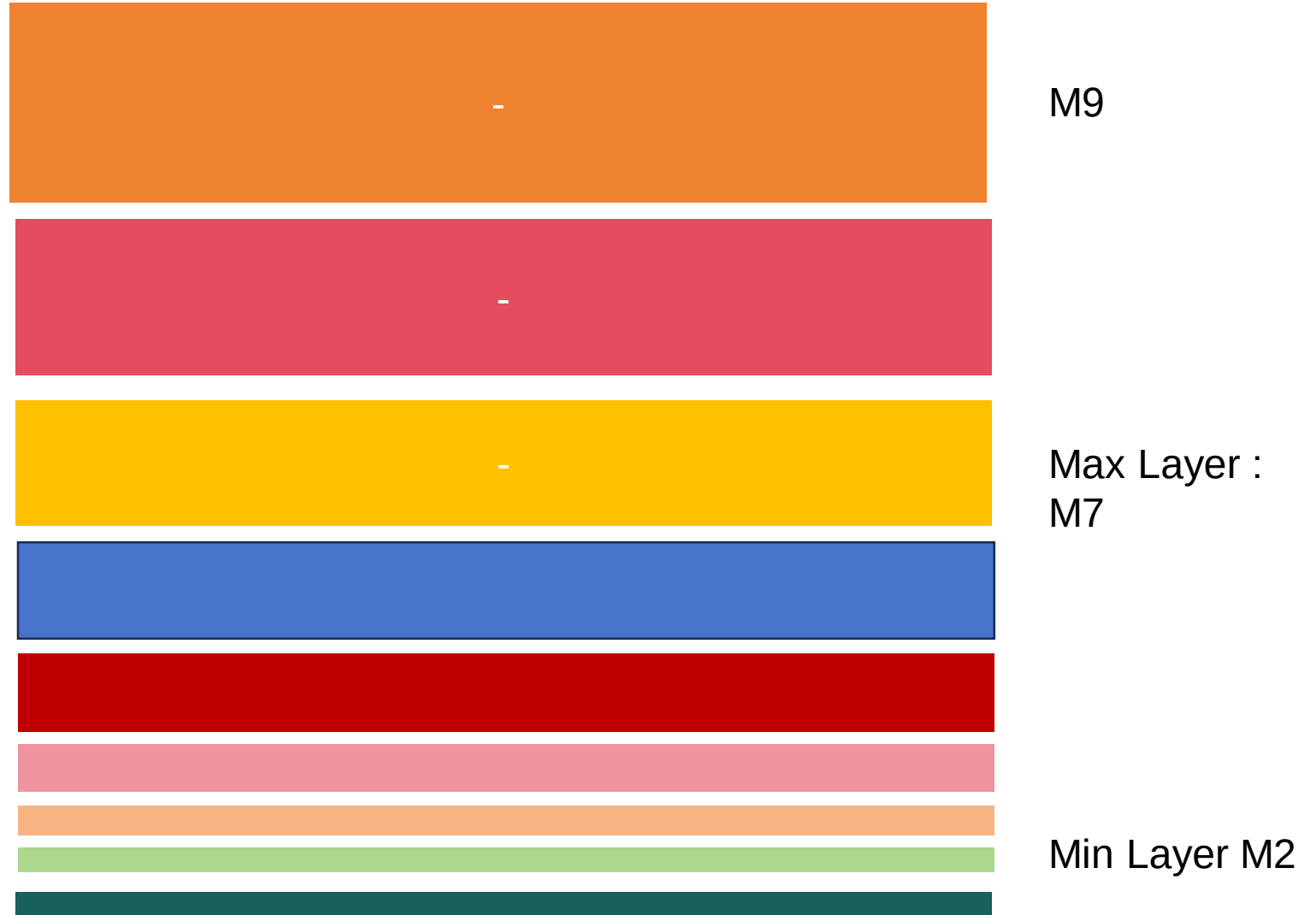


Min_Layer :M2



Preparation Steps For CTS

2- Specifying metal layers used in CTS



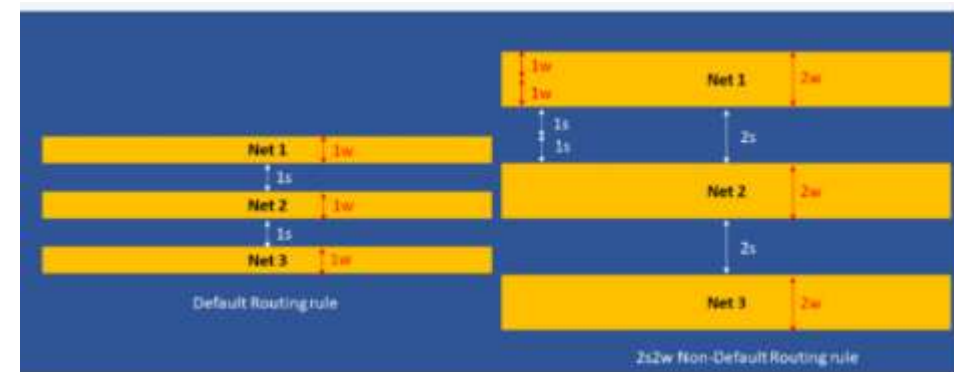
3- Specify Target skew ,Target latency ,Max Cap ,Max transition

- Specify them for all the scenarios

4- Non Default Routing (NDR)

- Specify double width and double spacing for the clock net **From M7 till M3 only**

- Default width and spacing from M3 till M1



5- Choose cells used for building clock tree , fix hold and optimization

- Our PDK doesn't have **Clock buffers**
- We had two options either to use normal inverters or normal buffers
- We made a comparison between them and we took **Partition C** as a sample for that to decide which cells would we choose to build clock tree
- During that comparison we put the same constraints for both :
 - Target_skew = 0.2 ns
 - Target_Latency = 0.2ns

Preparation Steps For CTS

	Built with inverters	Built with buffers
Acheived Global_skew (slow corner) (ns)	0.48	0.64
Acheived Global_skew (fast corner) (ns)	0.17	0.21
Acheived Global_latency (slow corner) (ns)	2.06	5.42
Acheived Global_latency (fast corner) (ns)	0.81	1.94
no. of cells	35,043	41,607
area after cts (um^2)	5822908.39	5826890.82

Were the values expected ?

Yes , according to the values of the transition time for inverter and buffers from lib

Cells	INV2X_RVT	NBUFF2X_RVT
Max_tran (ns)	0.438	0.1948
Max_cap (ff)	16	16

So, accordingly we chose **Inverters** to build our clock tree

Preparation Steps For CTS

Before we include those cells we exclude all cells from cts ,hold and optimization

For building our clock tree :

- Inverters (x2 till x16)

For fixing hold :

- Delay cells

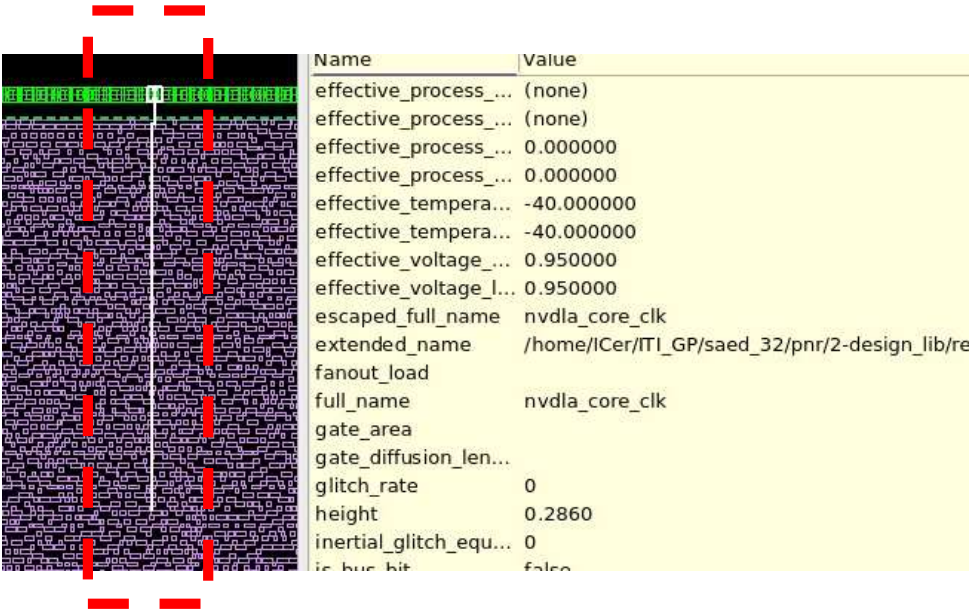
For optimization (fixation) :

- ~~Inverters & buffers (x2 till x16)~~

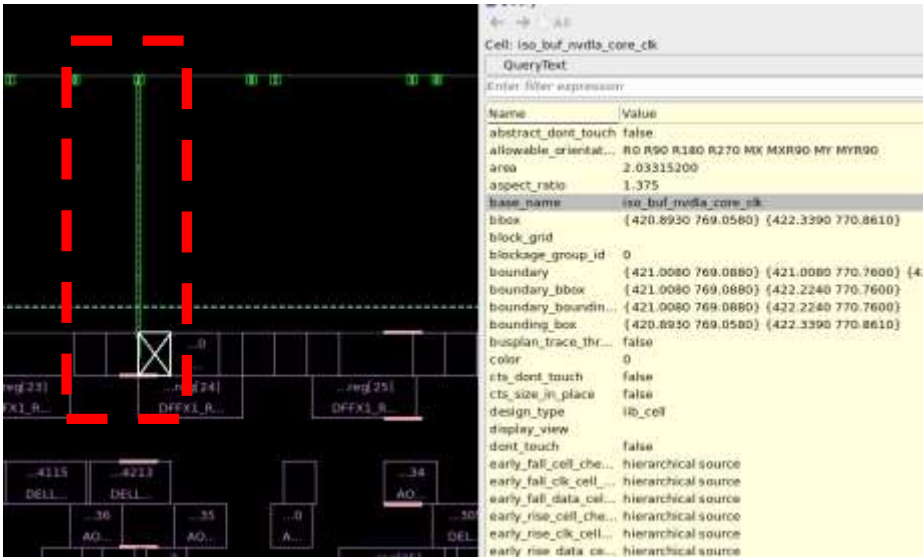


Made a **problem**

App option	Value	Usage
cts.common.max_fanout	15-20	Determine allowable max fanout during cts
clock_opt.flow.enable_ccd	true	Enables CCD that uses useful skew to improve timing
time.enable_si_timing_windows	true	Enables signal integrity that protects electrical quality of clock signals
time.si_enable_analysis	true	Enables signal integrity that protects electrical quality of clock signals
clock_opt.flow.hold.effort	high	Determine the effort for solving hold



Before app option

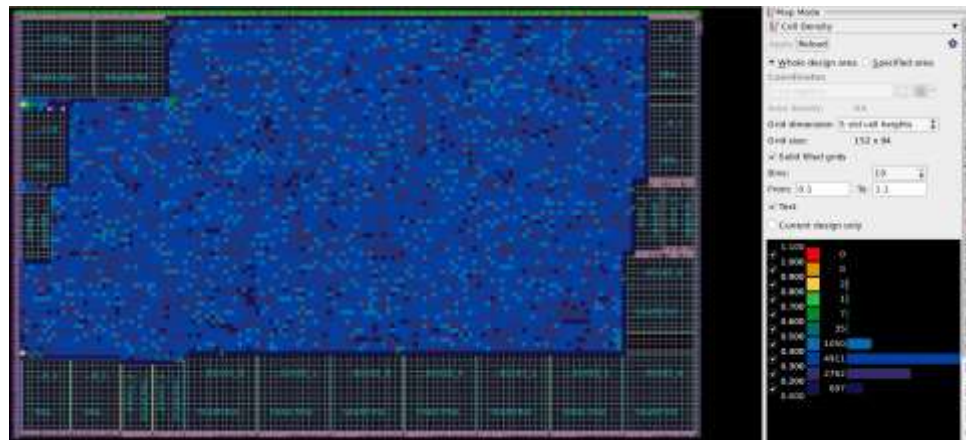


After app option

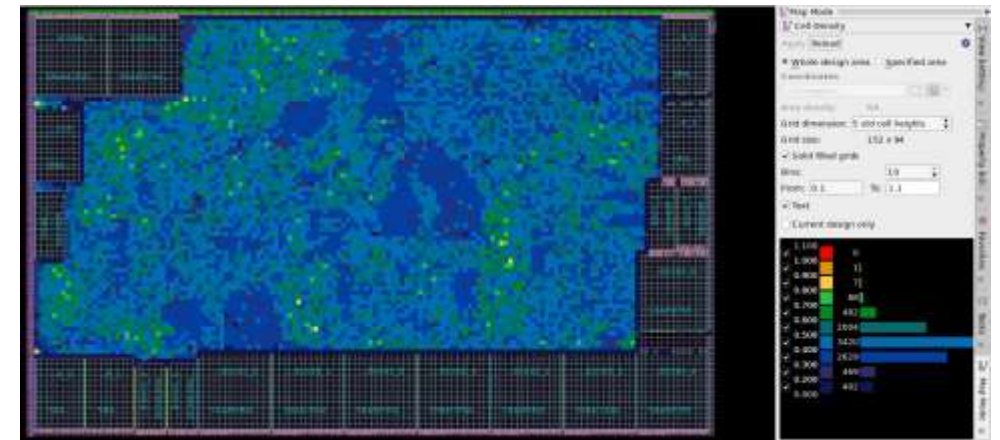
App option	Value	Usage
cts.common.max_net_length	150	Determine the allowable max length for the nets of the cts

After running cts we will find that :

- Hold violations would decrease if compared before cts
- Clock would be now propagated
- Cell density would increase



Before CTS



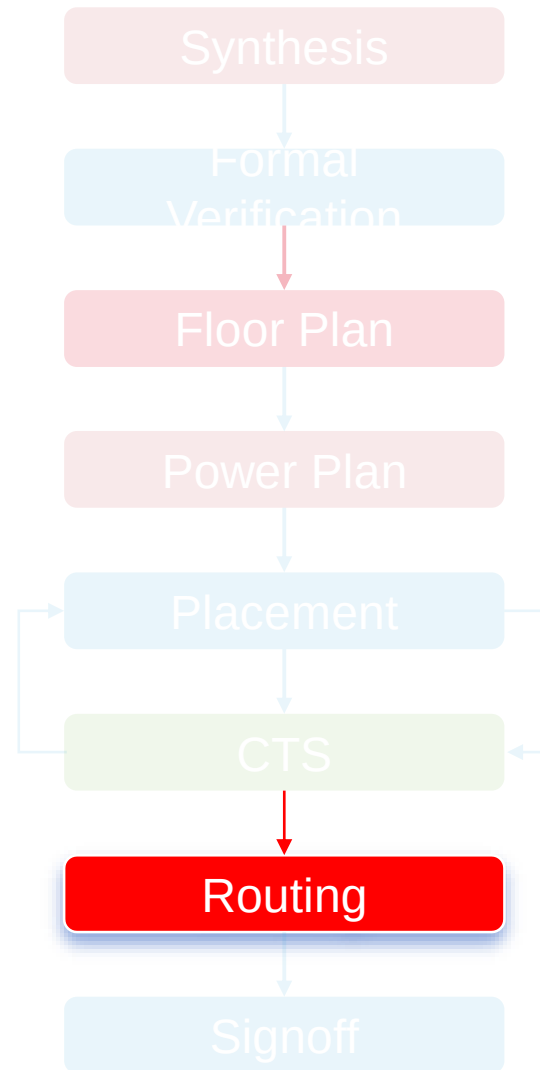
After CTS

For slow corner :

	Partition_C
Setup_before_CTS	13
Setup_after_CTS	3
Hold_before_CTS	0
Hold_after_CTS	0

For fast corner:

	Partition_C
Setup_before_CTS	0
Setup_after_CTS	0
Hold_before_CTS	17045
Hold_after_CTS	125



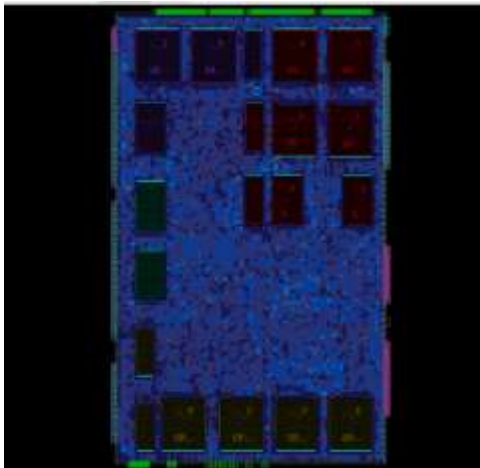
1. Routing Post-Rout ECOs
2. Chip Finishing

Routing and Post-Route ECOs

Post-CTS

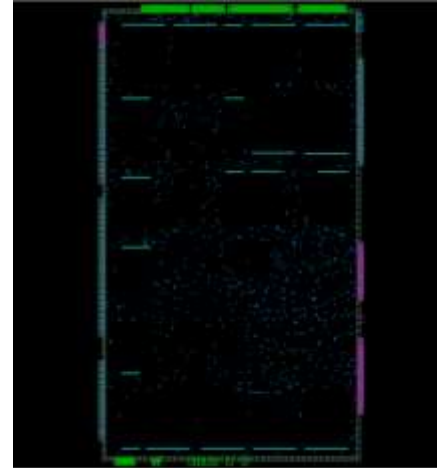
- **The Ultimate Test:** After we finished building the clock tree, we proceeded to the next step. This step ultimately determines the quality and viability of all the previous physical design steps.

Pin density map



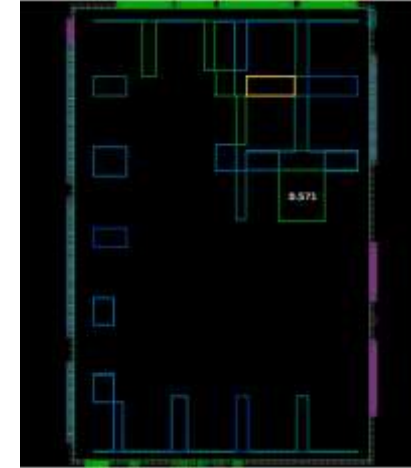
Maximum density 0.4

Global route congestion map



Maximum density 0.2

channel congestion map



Maximum density 0.6

Post-Route QoR Analysis

- **First Assessment:** After the routing was completed, we performed a QoR analysis to evaluate the physical and timing quality of the design before starting the ECO phase.
- The report showed that although the routing was completed, we still had several maximum capacitance and transition violations that needed to be fixed before final sign-off.
- This analysis gave us a clear starting point for the optimization and ECO iterations.

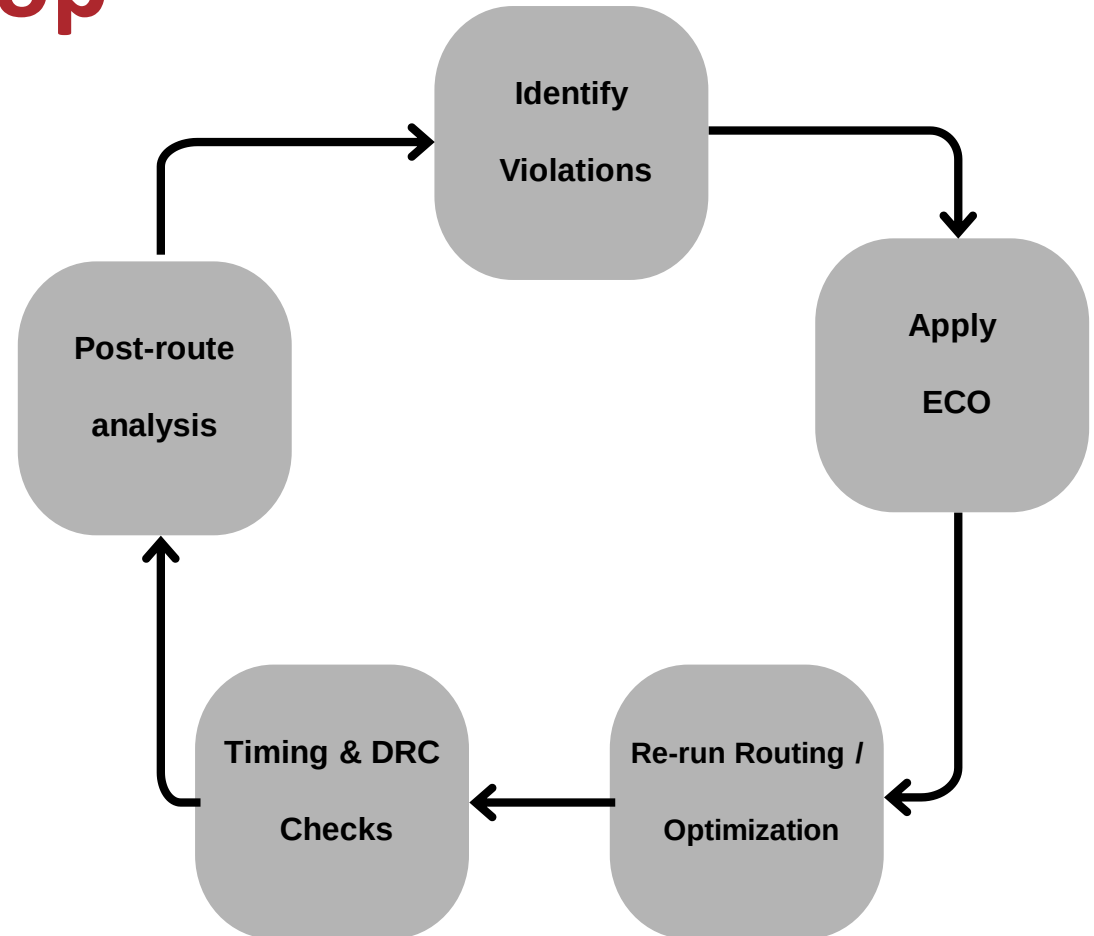
Violation type	Before ECOs
Nets with Violations	840
Max Trans Violations	44
Max Cap Violations	837
Setup	Met
Hold	Violations

- Finally, once the physical routing was clean, we started the Engineering Change Orders (ECOs) phase to meet our stringent setup and hold timing requirements.
- All timing requirement must be met for all sign-off corners
- This screen shows us that we have met the setup requirement for all scenarios, But the hold requirement is not met.



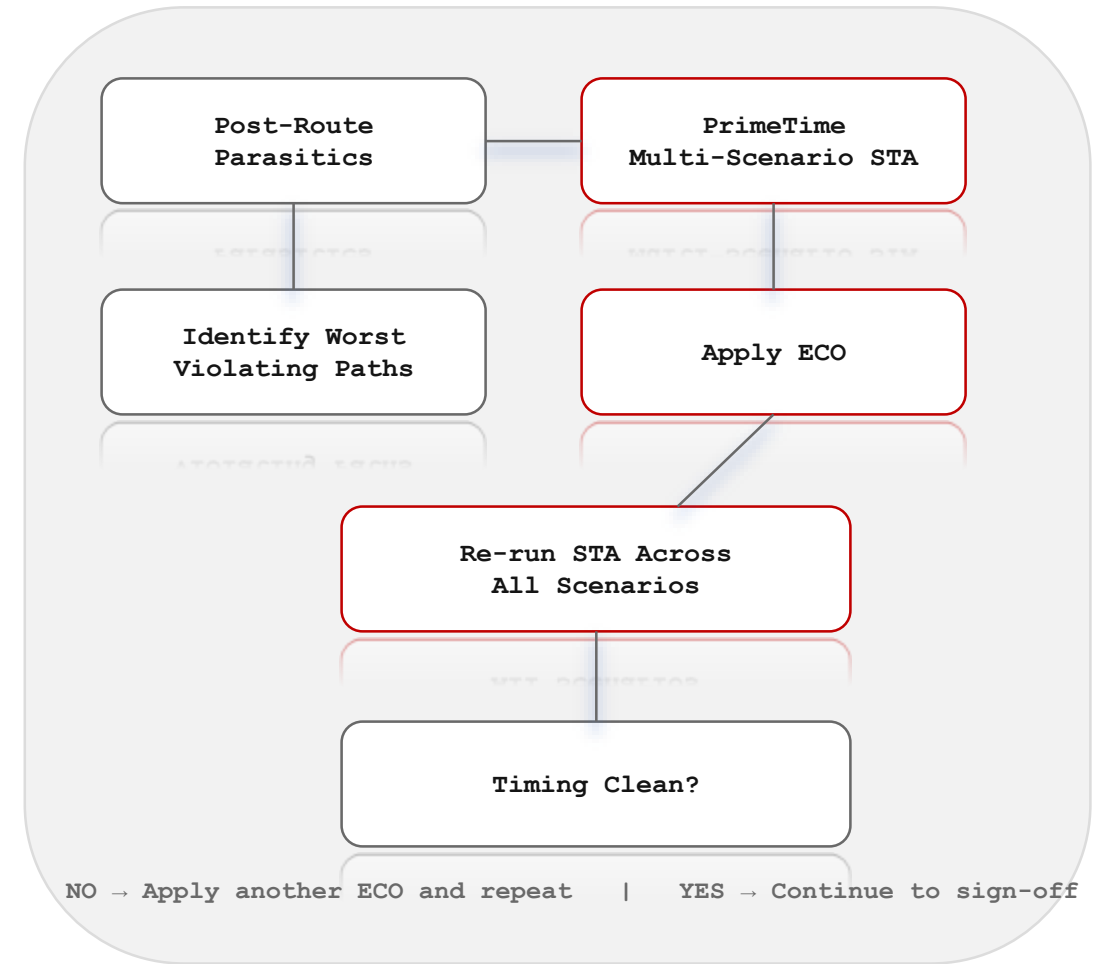
Post-Route Optimization Loop

- **Iterative Process:** Post-route optimization was not a single step; every change required us to re-check the physical and timing results to make sure that the design was still clean.
- We repeatedly analyzed the remaining violations, applied the required ECOs, and then re-ran the corresponding checks until all the requirements were satisfied.
- The main goal was to achieve timing closure while maintaining a clean physical design.



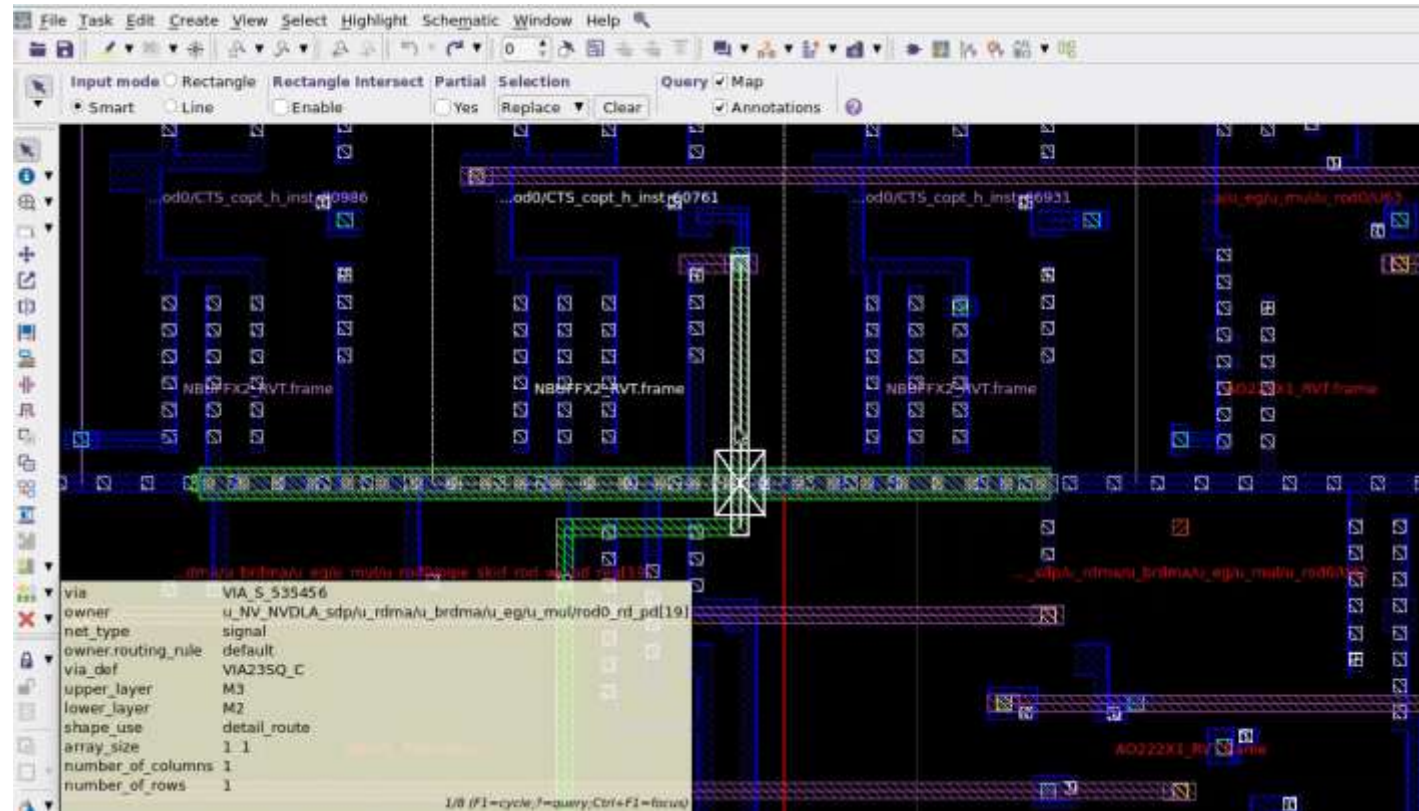
ECO Optimization with Multi-Scenario Analysis

- **PrimeTime Analysis:** After extracting the post-route parasitics, PrimeTime was used to analyze setup and hold timing across multiple scenarios and identify the paths that required optimization.
- **ECO Implementation:** Based on the worst violations, ECOs such as cell up-sizing, down-sizing, buffering, and repeater insertion were selected and applied while considering their impact on all active scenarios.
- **Multi-Scenario Check:** After each ECO iteration, timing was re-analyzed across all scenarios using Distributed Multi Scenario Analysis (**DSMA**) to make sure that fixing a violation in one scenario did not create a new violation in another scenario.



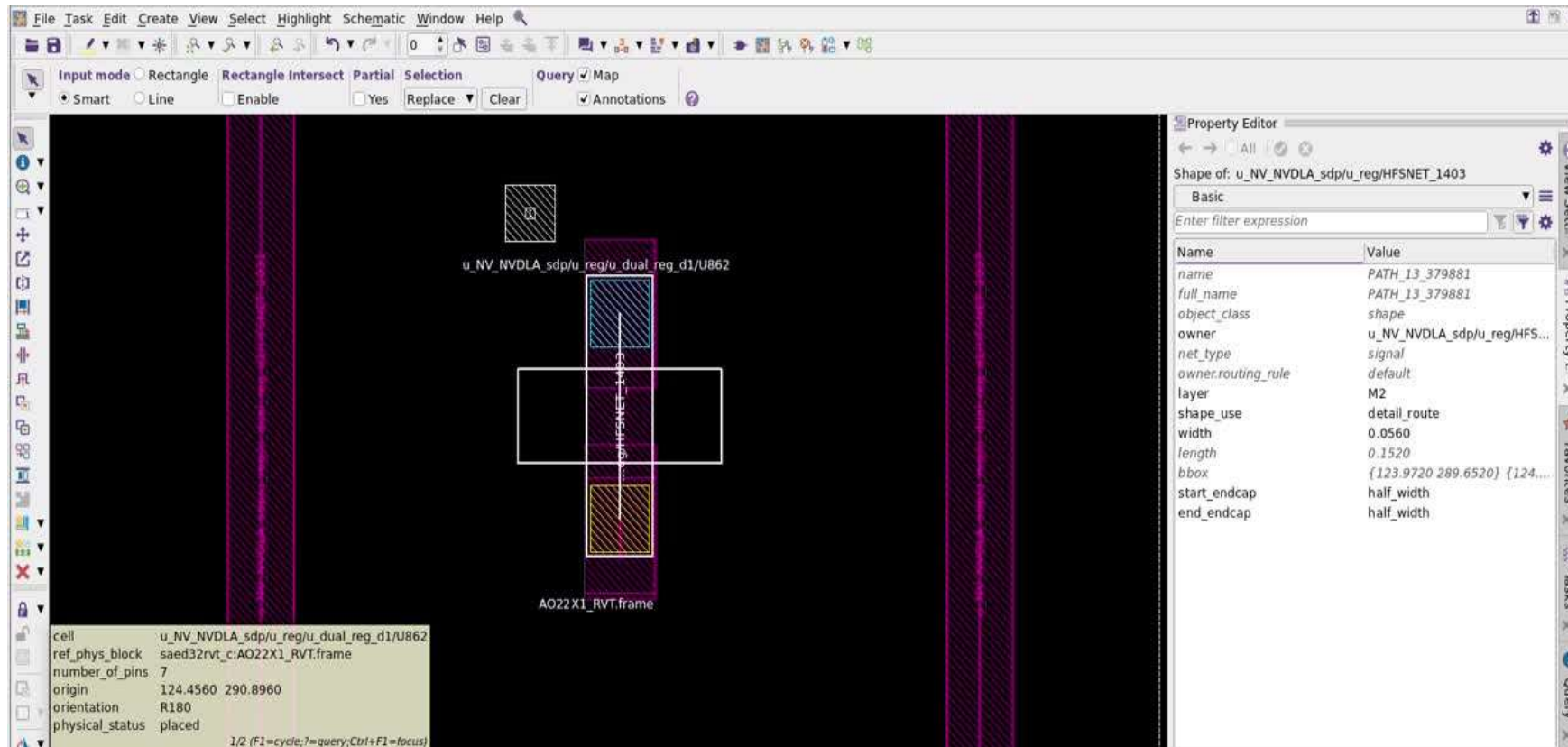
Routing DRCs

- Short between two deferent nets



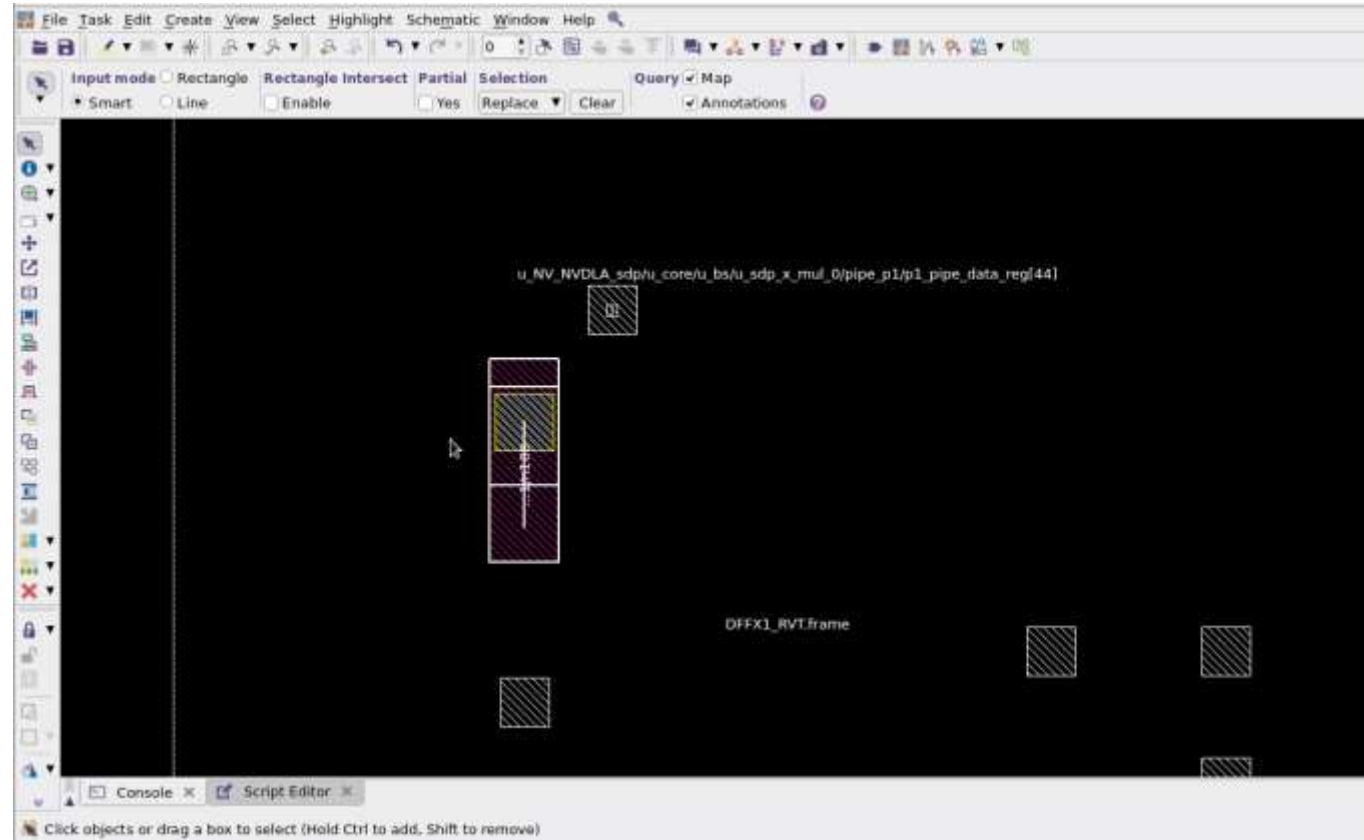
Routing DRCs

- Deferent net spacing



Routing DRCs

- Less than minimum area



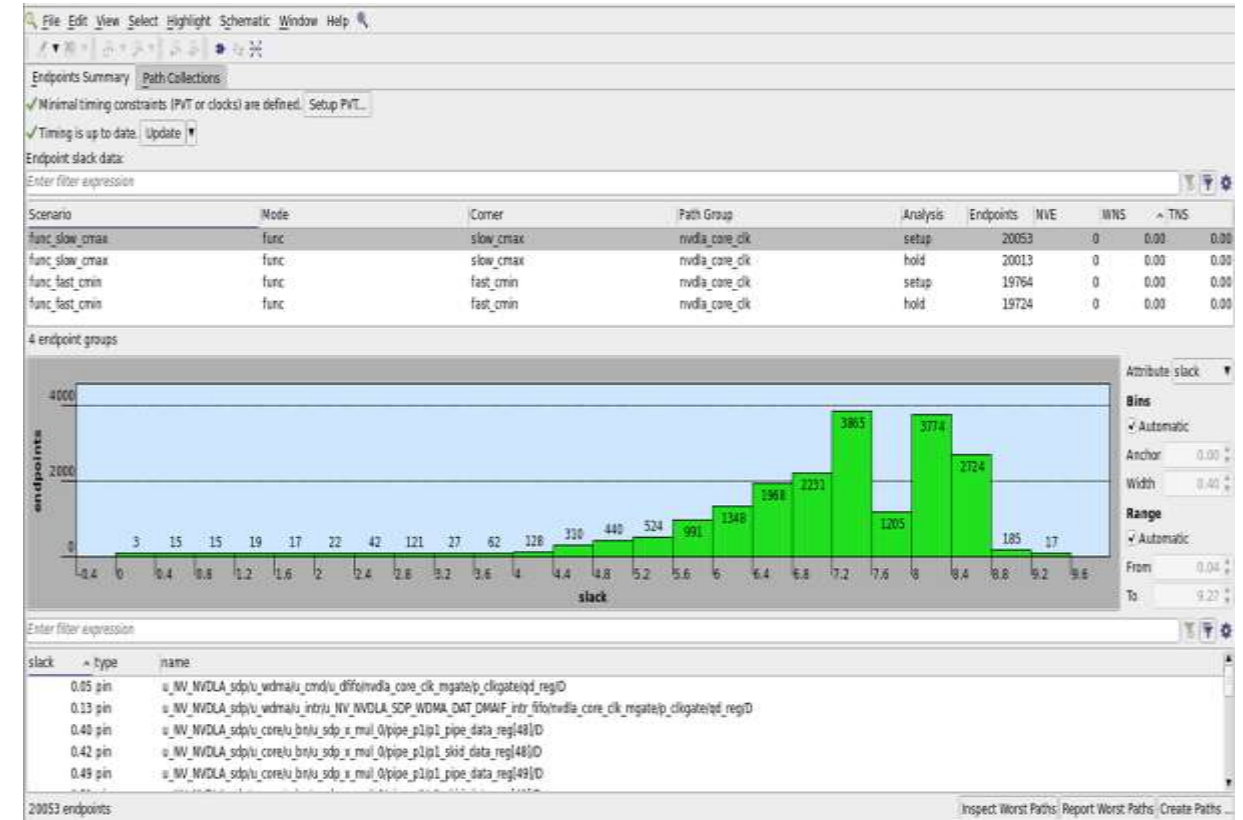
QoR Improvement

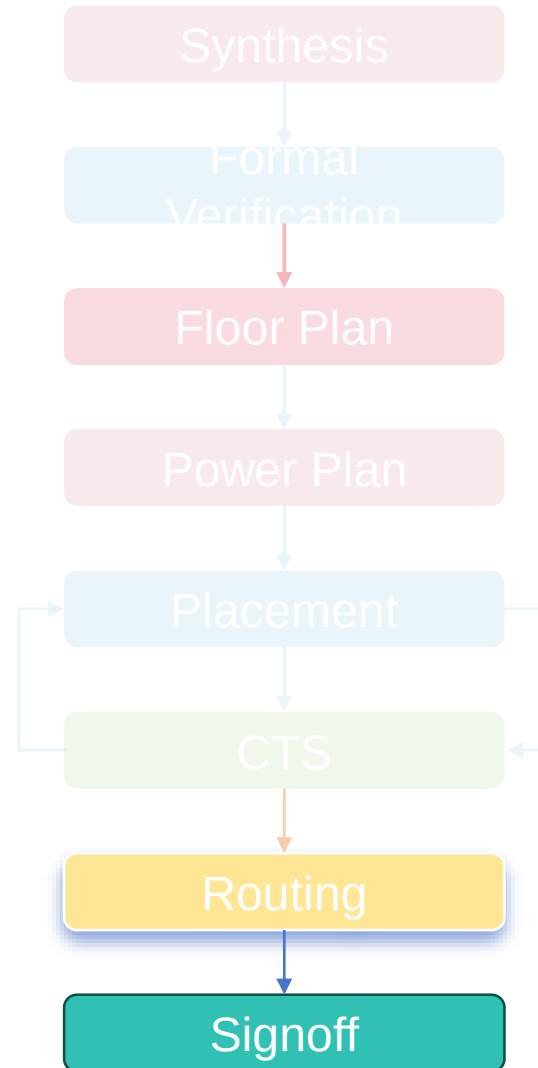
- **Final Optimization:** After several iterations of fixing the physical and timing violations, we compared the QoR results before and after the optimization process.
- The number of nets with violations was reduced from 840 to 0, while both maximum transition and maximum capacitance violations were completely resolved.
- The final timing report also shows that all setup and hold requirements were met for all sign-off corners.

Violaiton type	Before ECOs	Final
Nets with Violations	840	0
Max Trans Violations	44	0
Max Cap Violations	837	0
Setup	Violations	Met
Hold	Violations	Met

Timing Closure

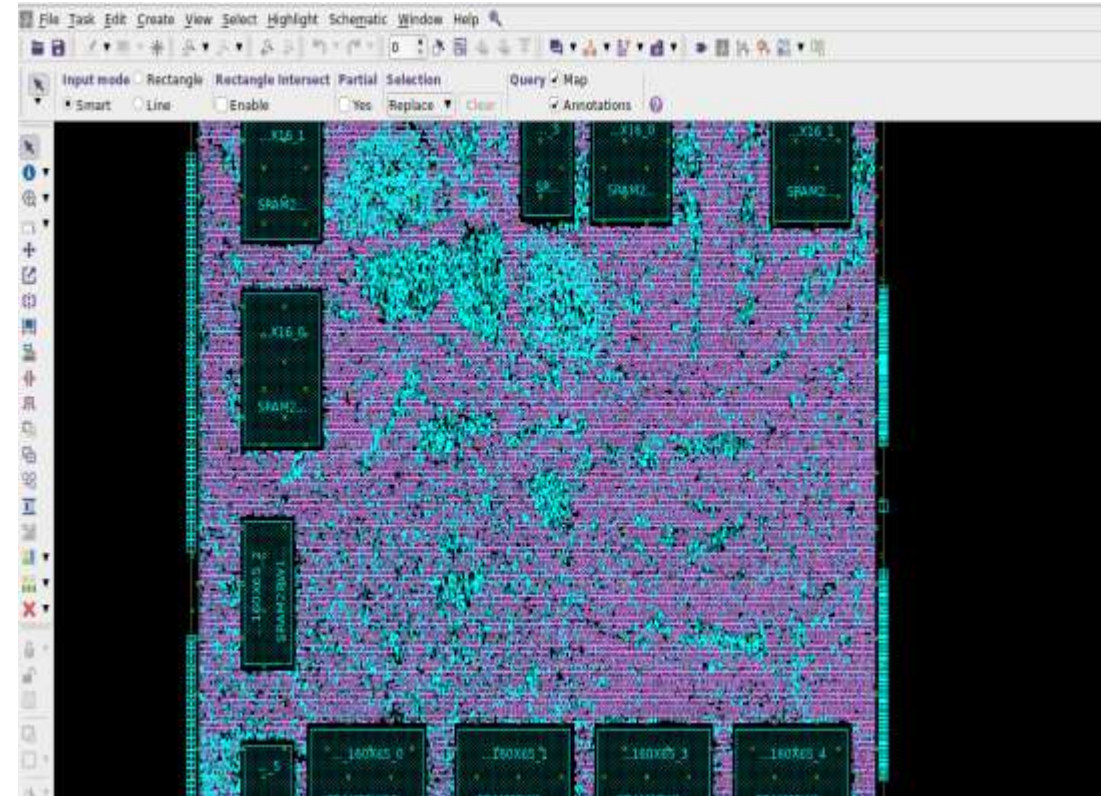
- After some iterations of optimizing the data paths and fixing all transition violations and capacitance violations with PrimeTime, this was the timing report for all sign-off corners, all requirements met.
- We also solved maximum capacitance and transition DRCs either by Up-sizing or adding reports to the violated path, and we ended up with 0 DRCs.





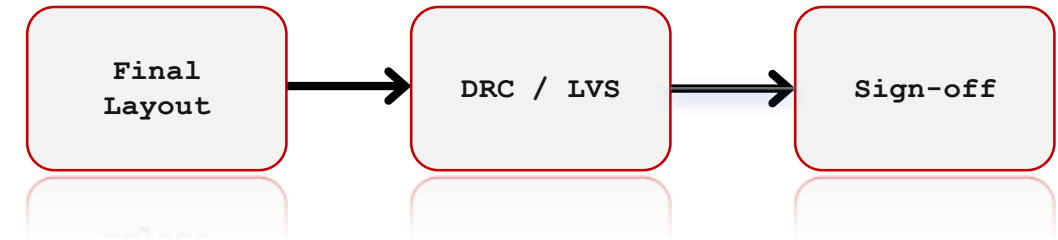
DCAP and Filler Cell Insertion

- Power Integrity: After completing the main optimization steps, DCAP cells were inserted to provide local decoupling capacitance and improve the stability of the power network.
- Filler Cells: Filler cells were then inserted into the remaining gaps between standard cells to maintain continuous well and implant regions and complete the standard-cell rows.
- After insertion, the design was checked again to make sure that the added cells did not introduce placement, routing, or physical verification violations.



Chip Finishing

- **Final Checks:** After achieving timing closure and resolving the physical violations, the design must pass the final physical verification checks before tape-out.
- At this stage, we verify the final layout for DRC and LVS correctness, connectivity, and other required sign-off checks.
- Once all checks are clean, the final database and sign-off reports can be delivered for the next stage of the chip manufacturing flow.



Final Sign-Off

- ✓ Timing requirements met
- ✓ Routing DRCs clean
- ✓ DRC / LVS checks passed
- ✓ Final design database generated



NVIDIA®

Thanks

